

Instituto Politécnico Nacional

UNIDAD PROFESIONAL INTERDISCIPLINARIA EN INGENIERÍA Y TECNOLOGÍAS AVANZADAS

Proyecto Terminal I

Prototipo de un sistema telemático para el monitoreo de la fermentación artesanal del kéfir de leche

Presenta:

Garcia Ledezma Miicxitl Kin

Asesores:

Dra. Bella Citlali Martínez Seis
Dra. Iclia Villordo Jiménez
Ing. Jonas Martínez Limón

4 de noviembre de 2025

upiita-ipn

Resumen

El presente proyecto tiene como objetivo desarrollar una herramienta tecnológica que permita a los productores de kéfir artesanal optimizar el tiempo de recolección del kéfir y determinar si su fermentación se llevó a cabo de manera óptima, con el fin de garantizar la calidad del producto. Para ello, se desarrollarán cuatro módulos. El primer módulo se encargará de obtener los datos del proceso de fermentación mediante sensores. Estos datos serán procesados en el segundo módulo, donde se realizará un análisis utilizando un modelo basado en reglas de lógica difusa para evaluar el estado de la fermentación. Luego, el tercer módulo almacenará tanto los datos obtenidos de los sensores como los resultados del análisis en una base de datos. Finalmente, el cuarto módulo permitirá visualizar toda esta información a través de una página web. Este sistema proporcionará a los productores de kéfir artesanal la posibilidad de analizar sus fermentaciones basándose en datos estructurados, en lugar de depender exclusivamente de conocimientos empíricos.

Palabras clave: kéfir, fermentación, sensores, análisis de datos, lógica difusa, base de datos, página web.

Abstract

The objective of this project is to develop a technological tool that allows artisanal kefir producers to optimize kefir harvesting time and determine whether its fermentation was optimal, in order to ensure product quality. To this end, four modules will be developed. The first module will collect data from the fermentation process using sensors. This data will be processed in the second module, where an analysis will be performed using a fuzzy logic rule-based model to assess the state of the fermentation. The third module will then store both the data obtained from the sensors and the analysis results in a database. Finally, the fourth module will allow all this information to be viewed via a website. This system will provide artisanal kefir producers with the ability to analyze their fermentations based on structured data, rather than relying solely on empirical knowledge.

Keywords: kefir, fermentation, sensors, data analysis, fuzzy logic, database, website.

Índice

1. Capítulo 1 **7**
1.1. Introducción . . . 7
1.2. Planteamiento del Problema . . . 8
1.3. Propuesta de Solucion . . . 9
1.3.1. Módulo 1:Sistema de Sensores . . . 9
1.3.2. Módulo 2:Análisis de Datos . . . 10
1.3.3. Módulo 4:Base de Datos . . . 11
1.3.4. Módulo 4:Página Web . . . 12
1.3.5. Productor en jefe (Administrador) . . . 12
1.3.6. Productores Inicial y Raíz (Usuarios) . . . 12
1.4. Objetivo . . . 13
1.5. Objeivos específicos . . . 14
1.6. Alcances . . . 14

2. Capítulo 2: Estado del Arte **15**
2.1. Internacional . . . 15
2.1.1. Sistema Embebido para el seguimiento y control en línea de un fermentador aplicado a la conversión de azúcares en etanol . . . 15
2.1.2. Modelado de la producción de kéfir con lógica difusa . . . 16
2.1.3. Diseño de fermentador de yogur con sistema de control lógico difusa usando el microcontrolador ATMEGA32 . . . 16
2.2. Nacional . . . 17
2.2.1. Sistema de aprendizaje automático para la predicción del avance de fermentación en cervecería artesanal . . . 17
2.2.2. Diseño de un proceso automatizado de fermentación de tepache 17
2.2.3. Desarrollo de un sensor de CO_2 y etanol para el monitoreo en procesos biológicos empleando una plataforma de código abierto 18
2.2.4. Análisis de la innovación de proceso y producto de un sistema de control y monitoreo de la fermentación alcohólica en vino . . . 19
2.3. Institucional IPN-UPIITA . . . 20
2.3.1. Sistema Telemático para el monitoreo y control de huertos urbanos basado en una red inalámbrica de sensores . . . 20
2.3.2. Prototipo de sistema inálambrico de monitoreo y advertencia para visibilizar fugas de mónoxido de carbono en un entorno limitado mediante técnicas de realidad aumentada . . . 20
2.4. Tabla comparativa . . . 20

3. Capítulo 3: Marco Teórico **23**
3.1. Kéfir . . . 23
3.2. Sensores . . . 24
3.2.1. Sensor de temperatura . . . 24
3.2.2. Sensor de gas . . . 25
3.2.3. Sensor de pH . . . 25
3.3. Modelado de Reglas basado en Lógica difusa . . . 25
3.4. Sistema Telemático . . . 26
3.4.1. Arquitectura . . . 27

3.4.2. Protocolo de comunicación . . . 27
3.4.3. Transmisión de datos . . . 28
3.4.4. Microcontrolador . . . 29
3.4.5. Backend . . . 29
3.4.6. Base de datos . . . 29
3.4.7. ACID . . . 30
3.4.8. BASE . . . 30
3.4.9. CAP . . . 30
3.4.10. Front End . . . 30
3.4.11. Framework . . . 31
3.4.12. Servidor . . . 31

4. Capítulo: Requerimientos Funcionales y no Funcionales 32
4.1. Módulo 1 . . . 32
4.2. Módulo 2 . . . 36
4.3. Módulo 3 . . . 37
4.4. Módulo 4 . . . 38
4.5. Servidor . . . 42

5. Capitulo: Análisis 45
5.1. Análisis del Módulo 1 . . . 45
5.1.1. Sensor de pH . . . 45
5.1.2. Sensor de Temperatura . . . 47
5.1.3. Sensor de CO_2 . . . 48
5.1.4. Formato de Datos . . . 49
5.1.5. Topología de red . . . 50
5.1.6. Protocolo de Comunicación . . . 52
5.1.7. Microcontrolador . . . 53
5.1.8. Fuente de alimentación . . . 55
5.2. Análisis del Módulo 2 . . . 57
5.3. Análisis del Módulo 3 . . . 59
5.3.1. Modelo de base de datos . . . 59
5.3.2. Base de datos NoSQL . . . 59
5.3.3. Estimación del volumen de la base de datos . . . 61
5.4. Análisis del Módulo 4 . . . 64
5.5. Framework . . . 66
5.6. Servidor . . . 67
5.6.1. Servidor en la nube o local . . . 67
5.6.2. MQTT Servidor . . . 67
5.6.3. Servidor en la nube . . . 68

6. Capítulo: Diseño 70
6.1. Diseño Módulo 1 . . . 70
6.1.1. Diagrama Módulo 1 . . . 70
6.2. Diseño Módulo 3 . . . 72
6.2.1. Modelo entidad relación . . . 72

Índice de figuras

1. Diagrama a Bloques de la interacción de los módulos del sistema 9
2. Vista Inferior y superior de la tapa del frasco 10
3. Vista completa del frasco 10
4. Diagrama a bloques de la integración del módulo 2 con el sistema . . . 11
5. Interacción del Administrador con la Página Web 12
6. Interacción del Productor con el sistema 12
7. Relación usuarios raíz con su origen 13
8. Evolución de pH y temperatura para la fermentación de lactosa. A. Evolución de pH a lazo abierto. B. Control de Temperatura utilizando un control PID. 15
9. Módelo de lógica difusa para el Cálculo de valores promedio ponderados 16
10. Esquema del sistema 16
11. Flujo de información 17
12. Flujo de información 18
13. pH durante la fermentación de piña. 18
14. Temperatura durante la fermentación de piña. 18
15. Pantalla principal del sistema. 19
16. Granos de kéfir de leche. 23
17. Diferentes procesos de fermentación iniciando desde glucólisis. 24
18. Diagrama del proceso de fermentación del kéfir. 26
19. El modelo TCP/IP frente al modelo OSI 28
20. Modo prueba del módulo 1 RF01. 35
21. Modo monitoreo del módulo 1 RF01. 36
22. Iniciar Sesón Página Web RF08. 42
23. Iniciar/Terminar una fermentación RF09. 42
24. visualizar historial de fermentación RF11. 42
25. Diagrama de administrador RF11. 43
26. Diagrama del módulo 1 71
27. Modelo Relación. 72

Índice de cuadros

1. Tabla comparativa de los proyectos con semejanza al prototipo. 20
2. Sensores de pH. 46
3. sensores de temperatura. 47
4. Sensores de CO_2 48
5. Formatos de Datos (Continúa en la página siguiente) 49
6. Topologías de conexión 51
7. Protocolos de Acceso a la Red 52
8. Protocolos de Transporte 53
9. Microcontroladores 54
10. Consumos de corriente de los componentes del Módulo 1. 55
11. Modelos de Baterías 57
12. Lenguajes de programación: C#, Python y R 58
13. Modelo de base de datos. 59
14. Bases de datos NoSQL 60

15. Atributos del módulo 1 61
16. Atributos del Usuario 62
17. Atributos del módulo 2 63
18. Atributos del administrador 63
19. Comparación de Frameworks. 64
20. Librerias de graficos para React 64
21. Comparación de Frameworks para el módulo 4. 66
22. Comparación de Servidor en la nube o local. 67
23. Comparación de servicios MQTT. 68
24. Comparación de Servidores en la nube. 68

1. Capítulo 1

1.1. Introducción

El kéfir (del turco *kief*, que significa "sentirse bien"ó .ªgradable sensación") es una bebida fermentada a partir de leche y un cultivo madre de microorganismos presentes en los granos de kéfir, conocidos en México como búlgaros. Esta bebida representa una de las prácticas de fermentación artesanal más antiguas, considerada incluso uno de los primeros productos pecuarios utilizados por el ser humano, con orígenes en las montañas del Cáucaso y Europa del Este [1] [2] Con el paso del tiempo, su producción y consumo se han extendido hacia diversas regiones del mundo, incluyendo México, donde fue adoptada tanto por sus propiedades nutricionales como por su valor probiótico. Además, ha sido adaptada por productores artesanales que han incorporado métodos propios, influenciados por la diversidad cultural y gastronómica del país [3] [4].
En la actualidad, la elaboración artesanal de kéfir en México ha cobrado gran relevancia debido a su creciente demanda comercial y al interés científico, lo cual ha derivado en una variedad de métodos de producción [5]. Estos métodos se ven condicionados por múltiples factores, como el tipo de leche utilizada, la proporción de búlgaros, la temperatura, el tiempo de fermentación y la fuente del inóculo (es decir, el equipo o recipiente utilizado) [6].
Siendo uno de los aspectos más importantes en el proceso de fermentación es la recolección del kéfir en el momento adecuado, ya que el tiempo de fermentación influye directamente en el sabor de la bebida. Sin embargo, en base a los ingredientes usados varía el punto óptimo de recolección lo que añade incertidumbre al proceso, obligando a los pequeños y medianos productores a definir sus propios parámetros iniciales para sus fermentaciones mediante prueba y error para definir el tiempo adecuado de recolección del kéfir. Esto no solo genera una gran variabilidad en la calidad del producto final, sino que también implica pérdidas económicas significativas, ya que lotes completos pueden resultar inadecuados para el consumo o la venta [7].
Por esta razón el monitoreo y control de los parámetros del proceso de fermentación se vuelve fundamental para garantizar una producción adecuada del kéfir, de lo contrario se obtendría una bebida avinagrada o muy ácida, siendo no apta para el consumo humano. Con el fin de atender esta problemática, el presente proyecto propone el diseño de un prototipo de monitoreo para la fermentación artesanal del kéfir, proporcionando una herramienta tecnológica que ayude a determinar el tiempo adecuado para la recolección del kéfir. De esta manera, contribuir a la mejora de la producción artesanal de kéfir, ofreciendo a los productores un mayor monitoreo sobre su proceso de producción, garantizando la calidad del producto final.

1.2. Planteamiento del Problema

La producción artesanal de kéfir en la Ciudad de México ha experimentado un crecimiento notable debido a su popularidad como bebida probiótica. No obstante, al tratarse de una actividad artesanal, el kéfir no es elaborado por una sola empresa, sino por diversos productores independientes que aplican métodos distintos. Estos métodos varían en aspectos clave como el tipo o la marca de leche comercial utilizada la cual, según la Norma Oficial Mexicana NOM-155-SCFI-2012, suguiere un pH inicial entre 6.6 y 6.8 [8], la cantidad de cultivos madre (búlgaros) empleados, y las condiciones ambientales, especialmente la temperatura, que debe mantenerse dentro de un rango recomendado de 18°C a 30°C. Esta diversidad de prácticas genera inconsistencias tanto en los tiempos de fermentación, que pueden ir de 12 a 48 horas, como en la calidad del producto final [9].

Uno de los principales retos que enfrentan los productores es asegurar que el kéfir sea seguro para el consumo, lo que implica identificar el momento preciso para su recolección. Este momento ocurre cuando el pH alcanza un intervalo saludable, de entre 4.2 y 4.6, y la temperatura se ha mantenido estable dentro del rango adecuado durante todo el proceso. Si bien puede alcanzarse el pH requerido, si la temperatura durante la fermentación no ha sido la apropiada, el producto puede resultar inadecuado para el consumo, ya que las desviaciones térmicas afectan negativamente el desarrollo de los microorganismos presentes en los búlgaros [10]. De igual forma, si el kéfir se recolecta y consume antes de alcanzar el pH mínimo (por encima de 4.6), los microorganismos no habrán completado su desarrollo, comprometiendo sus beneficios para la salud. Por otro lado, si se consume cuando el pH ya ha descendido por debajo del umbral recomendado (por debajo de 4.2), el proceso fermentativo puede volverse excesivo, generando un sabor avinagrado que indica la degradación de los microorganismos; en este estado, su consumo puede representar un riesgo sanitario. Por lo tanto, es fundamental que el kéfir se consuma dentro del intervalo adecuado de pH y que la temperatura permanezca dentro de los márgenes recomendados, a fin de garantizar el desarrollo correcto de los microorganismos y preservar sus propiedades probióticas [11].

En consecuencia, el monitoreo constante del proceso fermentativo se vuelve una tarea exigente para los productores, quienes con frecuencia recurren a métodos empíricos para evaluar el estado del producto, como abrir el frasco para olerlo o probarlo esporádicamente. Sin embargo, estas prácticas no solo interrumpen la fermentación, sino que también pueden comprometer la calidad del kéfir [12].

Ante esta problemática surge la siguiente pregunta: ¿Cómo desarrollar un sistema para automatizar el monitoreo del kéfir para determinar si la fermentación alcanzo el pH adecuado y se mantuvo en la temperatura adecuada para el consumo humano?, ya que de mejorar este proceso de monitoreo permitirá a los productores transitar de métodos empíricos e invasivos hacia un sistema basado en datos estructurados y confiables.

1.3. Propuesta de Solucion

Con base en la problemática presentada, para mejorar el proceso de monitoreo del kéfir se propone el desarrollo de un Prototipo de un sistema telemático para el monitoreo de la fermentación artesanal del kéfir mediante sensores y un análisis de datos. El diseño del sistema busca proporcionar a los productores artesanales de kéfir una herramienta eficiente que les permita evaluar el estado del kéfir y determinar si este alcanzo el rango de pH ideal(4.2-4.6) y determinar si la temperatura durante toda la fermentación se mantuvo de igual manera en el rango ideal de 18°C a 30°C, mejorando su proceso de producción, reduciendo la variabilidad en sus resultados al tomar decisiones basadas en datos precisos y estructurados haciendo comparaciones entre los diferentes parámetros de la fermentación a través de un historial detallado de fermentaciones previas. El sistema propuesto se compondrá de cuatro módulos principales como se puede observar en la fig. 1, cada uno diseñado para abordar un aspecto específico del proceso de monitoreo de la fermentación. A continuación, se describe cada uno de los módulos

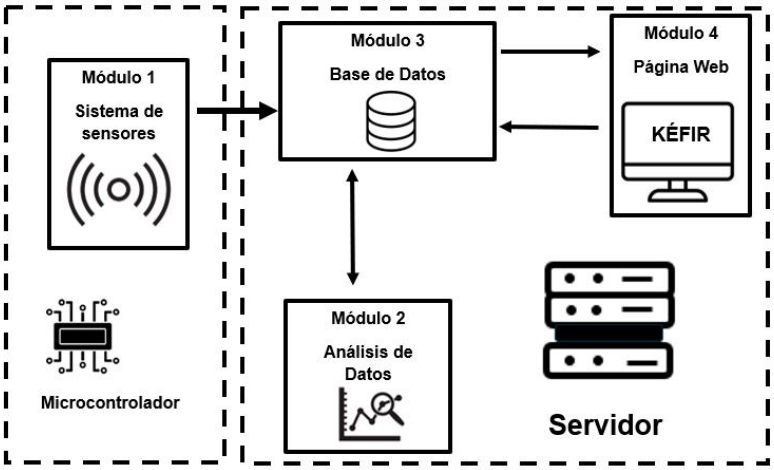


Figura 1: Diagrama a Bloques de la interacción de los módulos del sistema

del sistema:

1.3.1. Módulo 1:Sistema de Sensores

Para el desarrollo de este módulo, se integrarán tres sensores, cuya implementación dentro del proceso de fermentación es permitida por la NOM-251-SSA1-2009 [13], siempre y cuando no resulten invasivos para la fermentación.Los sensores se colocarán en la parte interna de la tapa de un frasco de vidrio hermético, mientras que en la parte superior de dicha tapa se instalará un microcontrolador, conectado a través de un circuito que enlazará los sensores con este componente, como se muestra en la fig. 2. Dado que los sensores de pH y temperatura estarán en contacto directo con el medio de fermentación, deberán ubicarse cuidadosamente en el interior del frasco, posicionados a una altura intermedia dentro del líquido. Esta ubicación es crucial, ya que en la parte superior del kéfir suele formarse una capa de grasa que podría interferir con las mediciones, mientras que en el fondo se acumulan los búlgaros. Por tanto, mantenerlos en la zona media del líquido garantiza una lectura representativa y confiable del proceso de fermentación. En contraste, el sensor de gas se colocará en el exterior del frasco,

conectado mediante una salida especializada que permitirá la liberación controlada del dióxido de carbono (CO_2) generado durante la fermentación. Esta salida evitará la acumulación de gas dentro del recipiente, lo cual podría generar sobrepresión, comprometiendo la seguridad del sistema y alterando las condiciones del proceso, como se muestra en la fig. 3.

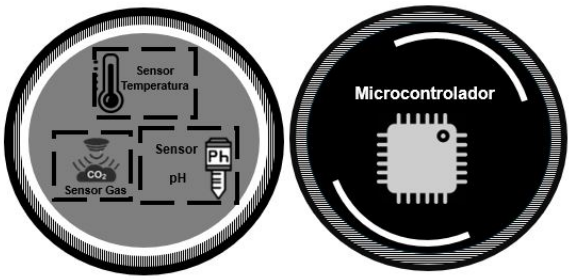


Figura 2: Vista Inferior y superior de la tapa del frasco

Si bien el prototipo se diseñará inicialmente para un frasco de 1 litro, el sistema podrá adaptarse a frascos de diferente capacidad, siempre que sean compatibles con la misma tapa, lo cual garantiza flexibilidad sin comprometer la funcionalidad del módulo de monitoreo. Dichos sensores se describen a continuación:

- **Sensor de pH:**Censar la disminución gradualmente del pH de la leche durante la fermentación y evaluar el pH inicial que este dentro del rango de 6.6 a 6.8.
- **Sensor de Gas (CO_2):**Censar la concentración de dióxido de carbono (CO_2) liberado durante la fermentación, lo cual es un indicador adicional de la actividad microbiológica y el progreso de la fermentación. Siendo que niveles constantes de CO_2 reflejan una fermentación activa y saludable.
- **Sensor de Temperatura:**Censar la temperatura durante el proceso de la fermentación y la temperatura inicial de la fermentación.

1.3.2. Módulo 2:Análisis de Datos

El módulo de análisis de datos se implementará a través de un modelo reglas basado en un algoritmo de lógica difusa. Su función será evaluar el estado del proceso de

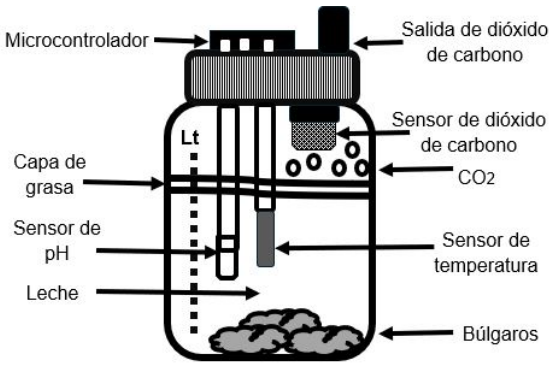


Figura 3: Vista completa del frasco

fermentación mediante un modelo de reglas previamente definidas en las cuales no se requiere una gran cantidad de datos de parte de los sensores y de la base de datos. Dicho funcionamiento de este módulo se muestra en la fig. 4 y se describe a continuación: Con base en los datos recolectados por los sensores, el módulo analizará si se ha alcanzado el pH ideal ubicado entre 4.2 y 4.6, junto con el comportamiento de la temperatura y la liberación de gas. De esta manera, el sistema no solo verificará si el pH final es adecuado, sino también si las condiciones de temperatura y producción de gas han sido consistentes con una fermentación exitosa. En caso contrario, podrá detectar si, a pesar de que el pH se haya alcanzado, las condiciones de temperatura o de gas no fueron las adecuadas, lo que indicaría una fermentación no óptima. Además, si durante el proceso la temperatura se sale del rango ideal, el sistema emitirá alertas. Todas estas alertas se enviarán a la base de datos para su registro y visualización en la página web.

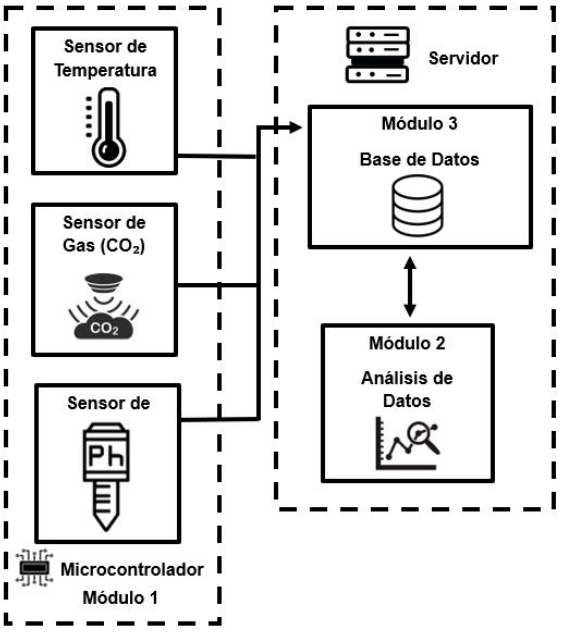


Figura 4: Diagrama a bloques de la integración del módulo 2 con el sistema

1.3.3. Módulo 4:Base de Datos

Este módulo es crucial para almacenar y gestionar la información relacionada con las fermentaciones. Sus funciones principales incluyen:

- **Almacenamiento de Datos**Los datos obtenidos por los sensores serán almacenados en una base de datos. Además, de las alertas emitidas por el análisis de datos.
- **Historial de Fermentaciones**La base de datos mantendrá un historial de todas las fermentaciones realizadas por cada productor, lo que permitirá realizar un seguimiento detallado de la evolución de cada proceso a lo largo del tiempo.
- **Gestión de Usuarios**El sistema almacenará información sobre los administradores (productores en jefe) y los productores (usuarios), permitiendo una gestión adecuada de los permisos y accesos al sistema llevados a cabo en la Página Web.

1.3.4. Módulo 4:Página Web

La página web es la interfaz de usuario que permitirá a los productores y administradores interactuar con el sistema. Los usuarios tendrán diferentes niveles de acceso según su rol, ya sea como productor en jefe (administrador) o como productores Inicial o Raíz (usuarios). Siendo que estos usuarios tendrán las siguientes funciones de la página web:

1.3.5. Productor en jefe (Administrador)

La interacción de este usuario con el sistema se puede observar en la fig. 5, así como se describe a continuación sus principales funciones:

- **Gestión de Usuarios:**El administrador podrá dar de alta o suspender a usuarios dentro del sistema, y visualizar la productividad de cada productor inicial y raíz así como su relación entre ellos, facilitando la gestión de permisos y accesos.
- **Monitoreo de Fermentaciones:**El administrador podrá visualizar el historial de fermentaciones de cada productor, así como la fermentación en curso de cada uno y recibir alertas si alguna fermentación en curso no está en condiciones óptimas. Asimismo, podrá manipular visualmente el historial de fermentaciones para poder comparar de manera estadística las fermentaciones.

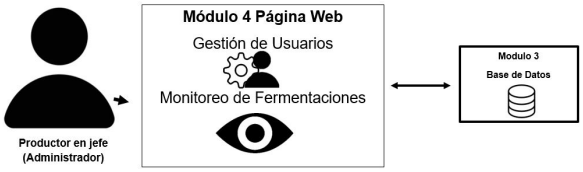


Figura 5: Interacción del Administrador con la Página Web

El productor tendra dos categorias, productor inicial y productor raiz, mismos que se describen a continuación:

1.3.6. Productores Inicial y Raíz (Usuarios)

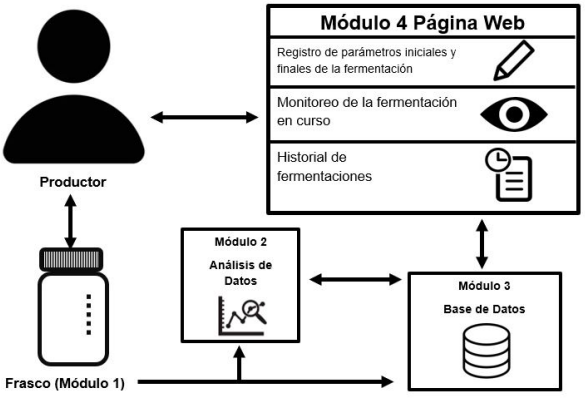


Figura 6: Interacción del Productor con el sistema

El productor, es decir, el usuario registrado en el sistema será responsable de llevar a cabo el proceso de fermentación del kéfir. Para ello, es indispensable que cada productor cuente con el frasco descrito en el Módulo 1. La interacción de este usuario con el sistema se ilustra en la fig. 6, y sus funciones en la plataforma web se describen a continuación:

- **Registro de parámetros iniciales y finales de la fermentación:**Una vez que el sistema detecte el inicio de la fermentación en el frasco, se solicitará al productor que ingrese los datos iniciales: tipo y marca de la leche utilizada, así como la cantidad de búlgaros empleados. Al finalizar la fermentación, el productor deberá registrar la cantidad final de búlgaros y proporcionar una breve descripción del sabor del kéfir obtenido.
- **Monitoreo de la fermentación en curso:**Una vez iniciado el proceso, el productor podrá visualizar los datos recopilados por los sensores conectados al frasco de fermentación. Así, como las alarmas emitidas por el módulo 2 (análisis de datos) sobre el estado de la fermentación.
- **Historial de Fermentaciones:**El productor podrá consultar en cualquier momento su historial de fermentaciones previas, visualizando los registros completos de cada proceso.

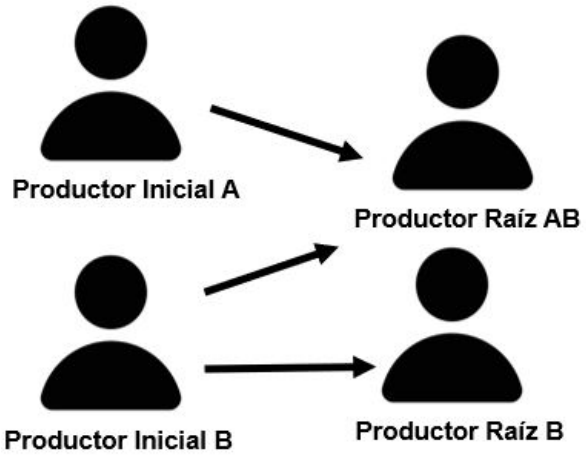


Figura 7: Relación usuarios raíz con su origen

Mientras que el prodcutor raíz será otra categoría la cual contará con las mismas funciones que el productor inicial; sin embargo, se considerará un usuario derivado de uno o más productores como se ejemplifica en la fig. 7, ya que en el proceso de fermentación del kéfir los búlgaros se reproducen y pueden ser compartidos con otros productores. De esta forma, el nuevo usuario hereda los cultivos del productor original, pero se considera independiente, ya que sus parámetros de fermentación como el tipo de leche, la temperatura o el tiempo pueden diferir. Esta relación entre usuarios permite trazar el origen de los búlgaros y analizar su trayectoria, lo cual puede ser útil para los productores al conocer el linaje o procedencia de sus cultivos.

1.4. Objetivo

Diseñar e implementar un prototipo de un sistema telemática de monitoreo no invasivo para la fermentación de kéfir artesanal de leche de vaca, mediante la recopilación,

análisis y visualización de los datos obtenidos por parte de los sensores.

1.5. Objeivos específicos

1. Seleccionar e integrar un microcontrolador y sensores no invasivos de temperatura, pH y CO_2 mediante un circuito que permita su conexión y el envío de datos hacia la base de datos.

2. Desarrollar una interfaz web enfocada a cada usuario (administrador, productor inicial y raíz), que permita la visualización de los datos recopilados por los sensores y del análisis de datos, así como de las alarmas generadas por el sistema.

3. Diseñar e implementar un modelo de reglas basado en criterios definidos que interprete los datos de los sensores y evalúe el estado de la fermentación, verificando que el pH alcance el rango ideal (4.2 a 4.6) y que la temperatura se mantenga dentro del intervalo óptimo durante todo el proceso.

4. Analizar, estructurar e implementar una base de datos que permita almacenar los datos del monitoreo, así como las alarmas emitidas por el sistema, y gestionar la autenticación de los usuarios de la página web

5. Realizar pruebas de fermentación con distintos parámetros iniciales para validar el funcionamiento del sistema.

1.6. Alcances

1. El sistema va dirigido únicamente a pequeños y medianos productores de kéfir artesanal elaborado con leche de vaca de origen comercial que cumplan con la Norma Oficial Mexicana NMX-F-446-1984, ubicados en la ciudad de México que requieran tener un mejor monitoreo en los parámetros de sus fermentaciones, los cuales deberán contar con una computadora para acceder al sistema.

2. Dado que es un prototipo, el sistema se enfocará únicamente en la visualización de los datos recolectados por los sensores e interpretados por el análisis de datos a través de la página web con el único fin de monitorear. El sistema no contara con funciones de manipulación o control de los parámetros en los sensores ni del módulo 2(análisis de datos).

3. El sistema estará limitado a un frasco por usuario (productor), con el fin de facilitar un monitoreo más preciso y controlado durante las pruebas del prototipo. Asimismo, el sistema solo permitirá una sesión activa por usuario, evitando la duplicación de datos o posibles inconsistencias.

4. Se establecerán los parámetros específicos que debe cumplir el proceso de fermentación, con base en normativas técnicas y condiciones óptimas del kéfir, los cuales serán utilizados por el análisis de datos para evaluar el proceso y emitir alertas si es necesario.

5. Debido a la sensibilidad y variabilidad que pueden presentar los sensores, deberán calibrarse y limpiarse manualmente después de cada proceso de fermentación, para esto se le incorporará al sistema una función de prueba para realizar dicha calibración, esto con el fin de tener una mejor medición por parte de los sensores.

2. Capítulo 2: Estado del Arte

En esta sección se presentan investigaciones y proyectos previos, tanto a nivel nacional como internacional, así como trabajos desarrollados dentro del Instituto Politécnico Nacional (IPN), en la carrera de Telemática de UPIITA, que están directamente relacionados con el prototipo propuesto. Estos estudios y proyectos destacan la relevancia del uso de sensores para la recolección de datos y la aplicación de algoritmos de inteligencia artificial en el análisis de estos datos, con el fin de optimizar la calidad y eficiencia en procesos de fermentación o en el monitoreo de alimentos. Las soluciones revisadas en este contexto están estrechamente vinculadas con los módulos propuestos en el sistema de monitoreo, particularmente en lo que respecta a la adquisición de datos y su posterior análisis.

2.1. Internacional

2.1.1. Sistema Embebido para el seguimiento y control en línea de un fermentador aplicado a la conversión de azúcares en etanol

En este proyecto, se desarrolló un sistema embebido basado en Arduino y Node-RED para supervisar y ajustar en tiempo real parámetros críticos como temperatura y pH en un biorreactor. El sistema incorporo un controlador PID que mantintuvo la temperatura dentro de los límites deseados con mínima variación, permitiendo un seguimiento remoto del proceso fermentativo mediante una aplicación móvil y visualización web, lo que mejora la eficiencia, reproducibilidad y flexibilidad operativa. Los resultados obtenidos muestraron una evolución estable de estas variables, con la validación realizada en una primera fermentacion con 10 g de granos de kéfir a 35 °C durante 18 horas, como se observa en la fig. 8 [14], destacando su utilidad para controlar procesos con levaduras y bacterias en aplicaciones alimentarias o bioenergéticas. Esto confirma que los sistemas embebidos son una herramienta clave para el monitoreo y control preciso de fermentaciones [14].

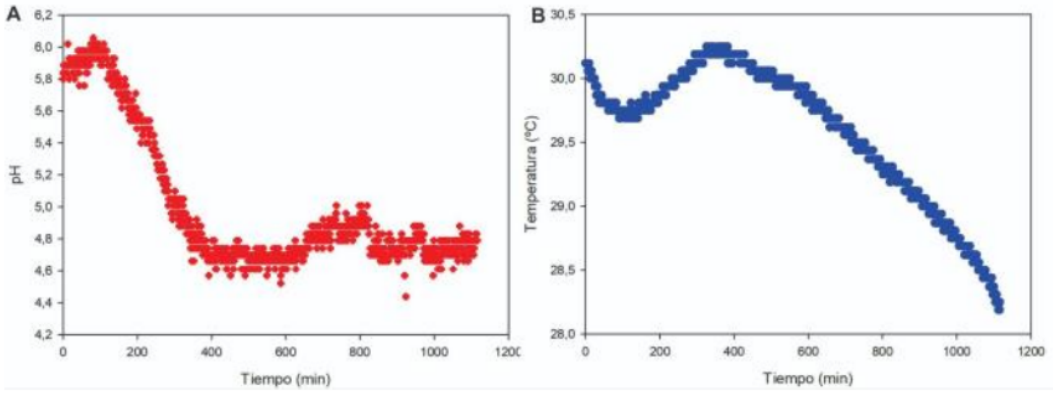


Figura 8: Evolución de pH y temperatura para la fermentación de lactosa. A. Evolución de pH a lazo abierto. B. Control de Temperatura utilizando un control PID.

2.1.2. Modelado de la producción de kéfir con lógica difusa

Este estudio propuso un modelo de lógica difusa para optimizar la producción industrial de kéfir, evaluando variables como temperatura de incubación, tiempo de fermentación y proporción de inóculo. Empleando reglas Mamdani y funciones de pertenencia triangulares y trapezoidales como se muestran en la fig. 9 [15], se construyó un sistema capaz de predecir si la fermentación alcanzaría el valor de pH ideal (4.6). La lógica difusa redujo la dependencia del operador y el tiempo de prueba-error, mejorando la eficiencia y reduciendo costos. Este modelo demostró que es posible automatizar y controlar procesos fermentativos complejos sin requerir modelos matemáticos precisos, lo cual es especialmente útil en sistemas alimentarios con variabilidad biológica [15].

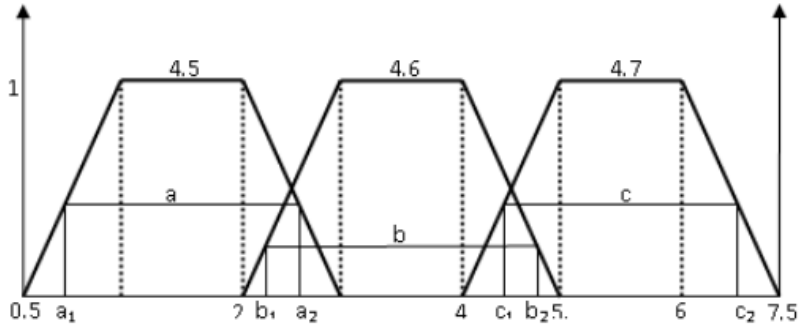


Figura 9: Módelo de lógica difusa para el Cálculo de valores promedio ponderados

2.1.3. Diseño de fermentador de yogur con sistema de control lógico difusa usando el microcontrolador ATMEGA32

Este proyecto desarrolló un fermentador de yogurt equipado con un sistema de control basado en lógica difusa implementado en un microcontrolador ATMega32 como se muestra en la fig. 10 [16]. El objetivo fue estabilizar la temperatura de fermentación entre 40°C y 45°C, rango óptimo para el crecimiento de Lactobacillus y Streptococcus. A diferencia del control ON-OFF tradicional, el sistema con lógica difusa ofreció un perfil térmico más estable, reduciendo errores de medición y mejorando la calidad del producto final. El yogurt resultante mostró valores óptimos de pH (3.66) y una mayor concentración de bacterias, evidenciando la efectividad del enfoque difuso en sistemas no lineales y biológicos como la fermentación [16].

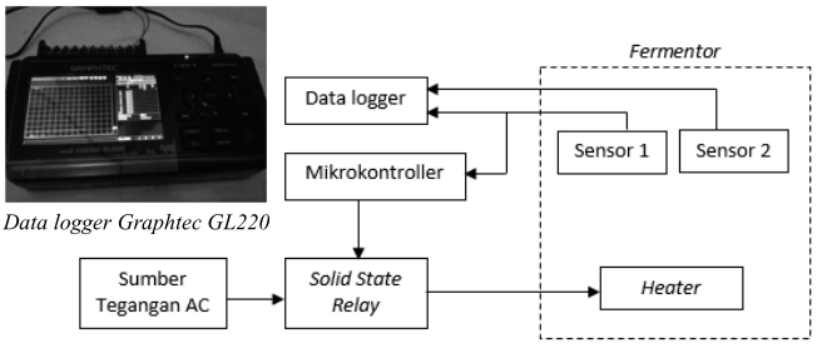


Figura 10: Esquema del sistema

2.2. Nacional

2.2.1. Sistema de aprendizaje automático para la predicción del avance de fermentación en cervecería artesanal

Este proyecto, realizado en colaboración con la cervecería artesanal Loba en Guadalajara, Jalisco,se implementó un sistema basado en la nube para obtener monitorear la temperatura durante la fermentación de la cerveza,a travez del diseño de un sistema de adquisición de datos y una interfaz web para el análisis de los datos en tiempo real, como se observa en la fig. 11 [17]. Haciendo uso de una IA, el sistema predice la densidad del mosto y clasifica las etapas de fermentación, mejorando la consistencia entre lotes y reduciendo pérdidas por variaciones no detectadas de temperatura. Demostrando ser una solución accesible y efectiva para pequeñas cervecerías artesanales [17].

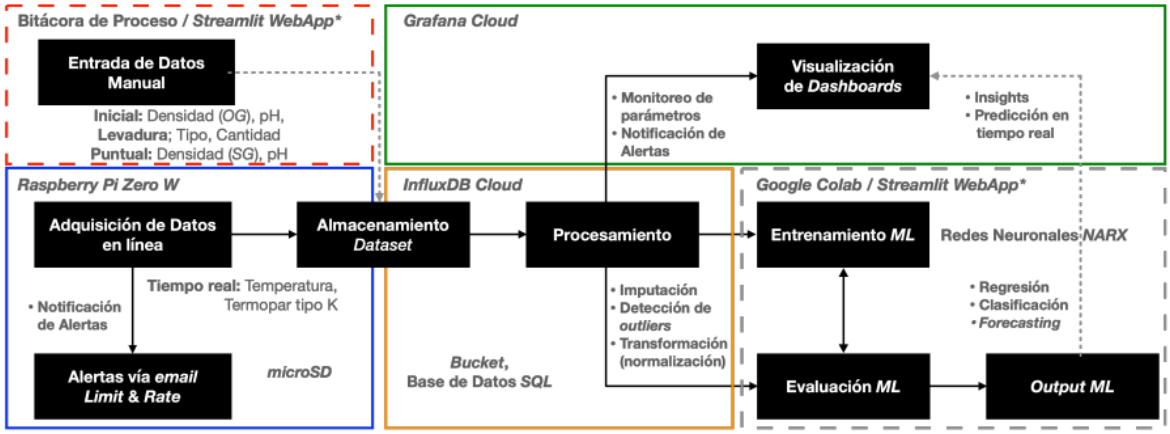


Figura 11: Flujo de información

2.2.2. Diseño de un proceso automatizado de fermentación de tepache

En colaboración con el Tecnológico Nacional de México, en este proyecto se diseñó e implementó un prototipo automatizado para la fermentación de tepache utilizando kéfir. Como se muestra en la fig. 11 [18], el sistema permite seleccionar entre distintos métodos de fermentación y, en base a ello, controlar el proceso seleccionado mediante el monitoreo del pH y la regulación de la temperatura, deteniendo automáticamente la fermentación al alcanzar los parámetros establecidos. Los resultados, observados en las fig. 12 y fig. 13 [18], establecieron un pH objetivo de 4.5, logrando un descenso controlado del pH ajustando la temperatura a 27 °C. Este enfoque busca estandarizar la producción de tepache, mejorando su calidad y seguridad alimentaria, y facilitando su escalabilidad [18].

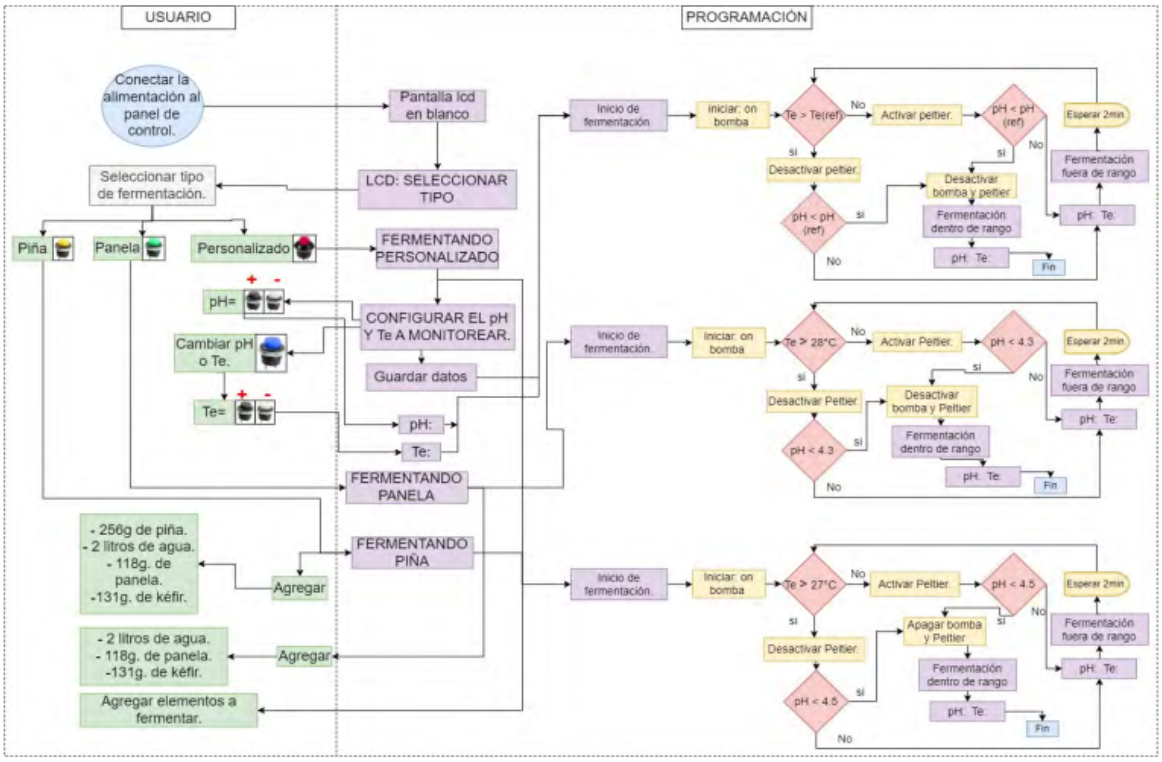


Figura 12: Flujo de información

2.2.3. Desarrollo de un sensor de CO_2 y etanol para el monitoreo en procesos biológicos empleando una plataforma de código abierto

Investigadores de la Universidad Autónoma Metropolitana, unidad Azcapotzalco, desarrollaron un prototipo de sensor basado en una plataforma de código abierto para monitorear en tiempo real las concentraciones de CO_2 y etanol durante procesos de

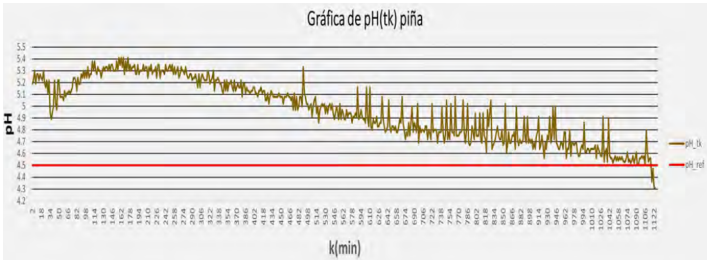


Figura 13: pH durante la fermentación de piña.

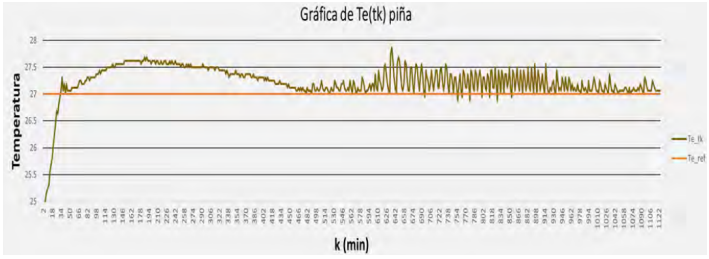


Figura 14: Temperatura durante la fermentación de piña.

fermentación. En el sistema se implementó una tarjeta Arduino con microcontrolador Atmega 328 y sensores electroquímicos, ofreciendo una alternativa económica para el monitoreo en bioprocesos. Demostrando su eficacia en la detección de cambios en la concentración de gases, permitiendo un seguimiento preciso del proceso fermentativo [19].

2.2.4. Análisis de la innovación de proceso y producto de un sistema de control y monitoreo de la fermentación alcohólica en vino

Este proyecto, realizado en CETYS Universidad, evaluó la viabilidad de implementar un sistema para controlar y monitorear parámetros clave como la temperatura durante la maceración de la uva o el pH en las diferentes etapas de la fermentación del vino, de manera remota a través de una interfaz web, como se observa en la fig. 15 [20]. Se identificaron y evaluaron parámetros esenciales del proceso de calidad, proponiendo mejoras tecnológicas para optimizar la producción y reducir costos. El estudio concluyó que la automatización del monitoreo de la fermentación puede mejorar la calidad del vino y aumentar la competitividad de las bodegas mexicanas en los mercados nacional e internacional [20].

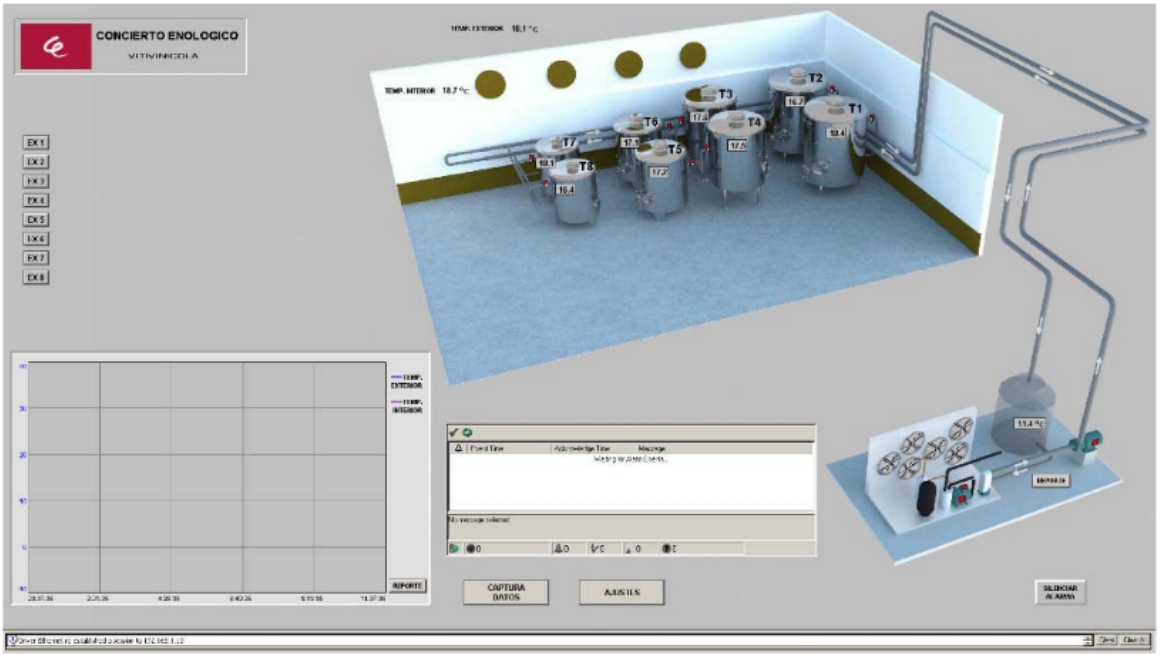


Figura 15: Pantalla principal del sistema.

2.3. Institucional IPN-UPIITA

2.3.1. Sistema Telemático para el monitoreo y control de huertos urbanos basado en una red inalámbrica de sensores

En este trabajo, realizado por estudiantes de Telemática de la UPIITA, se desarrolló un sistema telemático de monitoreo para el control de huertos, implementando tres módulos. El primer módulo se encarga de una red de sensores y actuadores interconectados de manera inalámbrica mediante el protocolo Zigbee hacia un nodo concentrador, responsable de comunicar la red de sensores con el servidor. El segundo módulo gestiona el servidor y la base de datos, mientras que el tercer módulo, una página web, permite al usuario seguir el estado de los huertos a través de una interfaz gráfica. Este trabajo destaca la importancia del monitoreo en los huertos para mantener un control de calidad en los cultivos finales [21].

2.3.2. Prototipo de sistema inálambrico de monitoreo y advertencia para visibilizar fugas de mónoxido de carbono en un entorno limitado mediante técnicas de realidad aumentada

En este proyecto realizado e implementado por estudiantes de Telemática de la UPIITA, se desarrolló un prototipo de una red de sensores inalámbricos para monitorear y advertir sobre fugas de monóxido de carbono (CO_2) en un entorno limitado.A traves de una pagina wev el usuario podra visualizar en tiempo real los niveles de CO_2 y recibir alertas mediante técnicas de realidad aumentada cuando se detecten niveles peligrosos del gas. El sistema utiliza sensores de CO_2 conectados a nodos inalámbricos que transmiten datos a un servidor central, donde se procesan y analizan para generar las alertas visuales en la interfaz de realidad aumentada [22].

2.4. Tabla comparativa

A continuación, se presenta la Tabla No. 1 cual muestra tanto las principales características, así como los módulos en donde presentan similitudes marcados con un ✓, los proyectos, investigaciones y estudios antes mencionados con el sistema.

Tabla 1: Tabla comparativa de los proyectos con semejanza al prototipo.

Proyecto	Módulo 1	Módulo 2	Módulo 3	Módulo 4	Tipo de relación
	Sistema de Sensores	Análisis de datos	Base de datos	Página Web	
2.1.1	✓ 1)Sensor de pH 2)Sensor de temperatura 3)Arduino			✓ Node Red	Tecnología implementada y problemática a resolver

2.1.2		✓ Modelo de Lógica difusa			Técnicas de análisis implementada y problemática a resolver
2.1.3	✓ 1)Sensor de temperatura 2)ATMega32	✓ Sistema con lógica difusa			Tecnología y técnicas de análisis implementada
2.2.1	✓ 1)Sensor de pH 2)Raspberry Pi Zero	✓ Red Neuronal NARX	✓ 1)Influxdb cloud 2)SQL	✓ Grafana	Tecnología y técnicas de análisis implementada
2.2.2	✓ 1)Sensor de pH 2)Sensor de temperatura 3)Arduino Mega				Tecnología implementada
2.2.3	✓ 1)Sensor de CO_2 3)Arduino Nano			✓ LABVIEW 2014	Tecnología implementada
2.2.4	✓ 1)Sensor de pH 2)Sensor de temperatura 3)Raspberry Pi 3				Tecnología implementada

2.3.1	✓ 1)Sensor de pH 2)Sensor de temperatura 3)ESP32		✓ Servidor de Azure Database para MySQL	✓ Framework express	Tecnología implementada
2.3.2	✓ 1)Sensor de CO_2 2) Raspberry PI	✓ 1) Amazon Sumerion	✓ AWS Cloud Dynamo DB	✓ 1) Amazon Cloudfront	Tecnología implementada

Como se observa en la tabla anterior, los trabajos revisados implementan diversos enfoques de monitoreo y análisis de datos aplicados a la producción de alimentos y bebidas fermentadas, lo que evidencia tanto la complejidad como la relevancia de estos sistemas. Cada propuesta fue diseñada con características particulares, adaptadas al medio y producto específico a monitorear.
Sin embargo, aunque varios de estos trabajos comparten una problemática similar a la abordada en el presente proyecto, como los proyectos 2.1.1 y 2.1.2, se identifican limitaciones importantes en el desarrollo integral de cada módulo propuesto. Por otro lado, aunque el proyecto 2.2.1 contempla módulos similares, su enfoque está dirigido a una problemática distinta y una solución más industrializada. Este último es un sistema muy robusto y mucho más completo, pero su naturaleza conlleva una implementación más costosa, ya que requiere equipo e infraestructura muy especializados.
Siendo que este proyecto tiene un enfoque más artesanal, esto resalta la necesidad de desarrollar un sistema completo, con todos los módulos interconectados, orientado específicamente al monitoreo de la fermentación artesanal del kéfir de leche, con el fin de mejorar la precisión, trazabilidad y control del proceso.

3. Capítulo 3: Marco Teórico

En este apartado, se abordan las teorías relevantes relacionados con cada módulo del sistema a través de fundamentos teóricos y conceptos clave.

3.1. Kéfir

El kéfir es una bebida fermentada probiótica obtenida a partir de la leche y un cultivo madre conocido como granos de kéfir, que miden entre 20 y 30 mm y son de color blanco o amarillentos con forma de coliflor, como se muestra en la fig. 16 [23]. Este proceso se lleva a cabo gracias al metabolismo anaeróbico de una comunidad de microorganismos, principalmente bacterias lácticas y levaduras, que viven en simbiosis dentro de los granos de kéfir. Dicho metabolismo provoca que el pH de la leche empiece a bajar gradualmente, en el que el punto ideal para su consumo es de 4.2-4.6 [24].

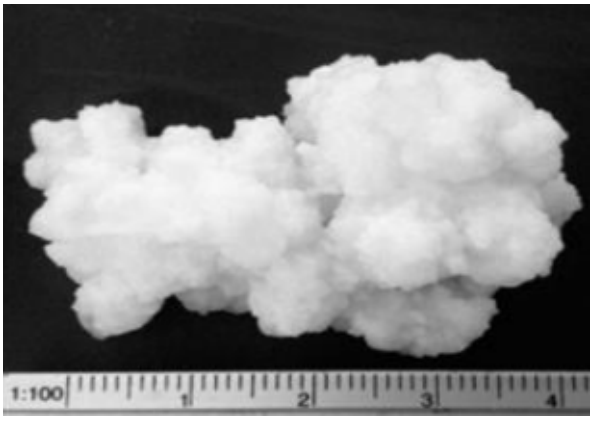


Figura 16: Granos de kéfir de leche.

El kéfir es de suma importancia en la actualidad debido a su alto valor probiótico, gracias a las bacterias presentes en él. Según un estudio sobre la bioactividad multifuncional del kéfir, realizado por José Carlos y otros, los granos de kéfir están constituidos por un consorcio de microorganismos llamados kefirian, que es un exopolisacárido en el cual se desarrollan bacterias ácido-lácticas, como varias especies de Lactobacillus, siendo especialmente importante el Lactobacillus kefiranofaciens, que únicamente se encuentra en los granos de kéfir [23].
A pesar de que existen numerosos estudios sobre la elaboración del kéfir, la mayoría destacan la importancia del desarrollo de los granos de kéfir en el proceso de fermentación. Por ejemplo, el estudio de Janet Araujo y otros aborda el desarrollo del kéfir en diferentes tipos de leche [11], mientras que el estudio de Fitrianingsih Fitrianingsih y colaboradores investiga el desarrollo de los granos de kéfir a diferentes temperaturas, señalando cómo estos factores también influyen en las propiedades únicas del proceso de fermentación [9]. Es así como en ambos estudios se destaca la relevancia del monitoreo durante la fermentación. No obstante, para llevar a cabo un monitoreo efectivo, es esencial comprender el proceso químico de la fermentación del kéfir, en el cual ciertos microorganismos, como bacterias y levaduras, transforman compuestos orgánicos, principalmente carbohidratos, en sustancias más simples, como ácidos, gases o alcoholes, con el fin de obtener energía. En el caso del kéfir de leche,la femerntación inicia

con la glucólisis en ausencia de oxígeno, convirtiendo la glucosa en ácido pirúvico para aceptar los electrones del NADH. Esta molécula orgánica se reduce para actuar como oxidante,y dependinedo de dicha molécula, puede seguir diferentes rutas metabólicas, como se muestra en la fig. 16 [20] .Siendo que en el kéfir predominan una femerntación mixta entre la alcohólica, en la que el ácido pirúvico se convierte en etanol y dióxido de carbono(CO_2),y la láctica, en la que en ausencia de oxigeno,el ácido pirúvico lleva acabo la transformación de azúcares como la glucosa, lactosa o fructosa en ácido láctico, gracias a las bacterias lácticas presentens en el kéfir.En algunos casos, cuando el pH disminuye demasiado, también puede ocurrir una fermentación acética, en la cual ciertas bacterias convierten el etanol en ácido acético (vinagre), siendo esta última etapa de la femerntación indeseable en la elaboración del kéfir [25] [26]. Puesto que monitorear cada etapa de la fermentación en el proceso de elaboración del kéfir es crucial para asegurar su viabilidad y la producción continua de kéfir de calidad.

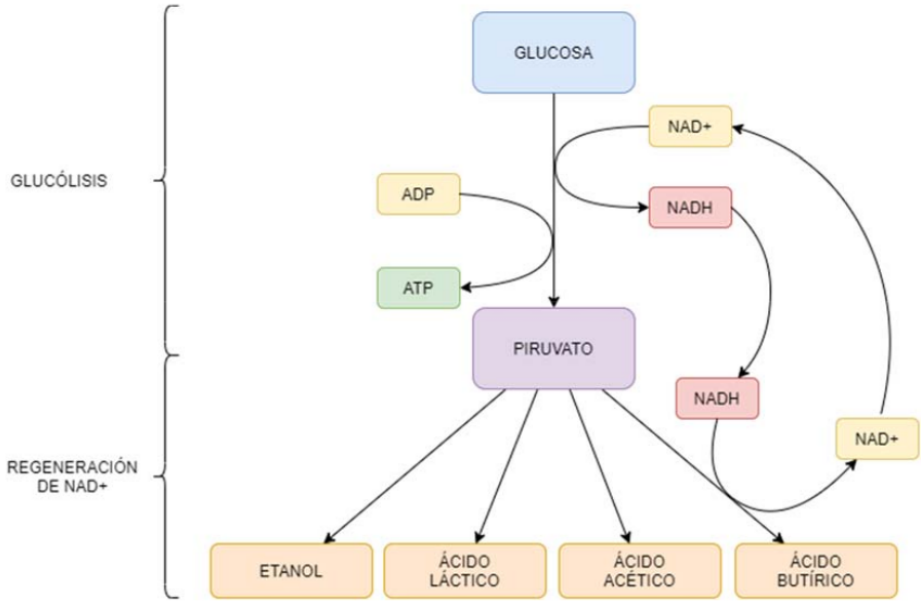


Figura 17: Diferentes procesos de fermentación iniciando desde glucólisis.

3.2. Sensores

Para monitorear el proceso de fermentación del kéfir, es esencial utilizar sensores que detecten cambios físicos y químicos en variables clave, como la temperatura, el pH y la concentración de dióxido de carbono. Estos sensores compuestos por por uno o más transductores son capaces de transformar las señales originales en datos interpretables, permitiendo un análisis preciso del proceso [27]. En la fermentación del kéfir, se destacan la fermentación alcohólica, que libera dióxido de carbono, y la fermentación láctica, que reduce el pH de la leche. Además, la temperatura juega un papel crucial, ya que las condiciones térmicas afectan directamente la actividad microbiana.

3.2.1. Sensor de temperatura

Un sensor de temperatura es un dispositivo compuesto por uno o más transductores capaces de detectar variaciones térmicas en entornos físicos o químicos y convertirlas en

una señal eléctrica o física comprensible para un sistema de control o de monitoreo. La precisión de estos sensores depende en gran medida del entorno en el que se utilicen, por lo que existen diversos tipos, cada uno con características específicas. Entre los más comunes se encuentran los termopares, los termistores, los RTD (detectores de temperatura resistivos) y los sensores bimetálicos, de silicio o monolíticos, Termistores, piros eléctricos, entre otros [28].

3.2.2. Sensor de gas

Un sensor de gas es un dispositivo diseñado para detectar la presencia y concentración de gases específicos en el ambiente, convirtiendo esta información en señales que pueden ser interpretadas por un sistema de control. Existen distintos tipos, como los sensores electroquímicos, que generan una corriente eléctrica proporcional a la concentración del gas mediante reacciones químicas, y los sensores infrarrojos (NDIR), que miden la absorción de radiación infrarroja por parte de los gases [29].

3.2.3. Sensor de pH

Un Sensor de pH es un dispositivo que permite medir el grado de acidez o alcalinidad de una solución mediante una señal de salida analógica proporcional al valor de pH. Generalmente, está compuesto por un electrodo de vidrio sensible al pH y un electrodo de referencia; la interacción de estos genera un potencial eléctrico que el transmisor convierte en un valor numérico. Este potencial se produce por la diferencia de carga en las capas de gel formadas en el vidrio, como resultado del contacto con los iones de hidronio presentes en la solución. Para asegurar la fiabilidad de las mediciones, es esencial realizar calibraciones periódicas, ya que el desgaste o la obstrucción del sensor puede afectar su precisión [30]. La integración de estos sensores de manera no invasiva permite un monitoreo constante al recolectar los datos obtenidos para su posterior análisis

3.3. Modelado de Reglas basado en Lógica difusa

Varios estudios como el de Wei, C., y Chen, L. en las fermentaciones, han determinado que estas se caracterizan por su naturaleza no lineal y variante en el tiempo, por lo que se debe implementar un modelo matemático especifico en base a los variables de cada fermentación [31]. En el caso del kéfir la fermentación se puede representar por medio de la fig. 18, en donde la salida del proceso genera cambios en el pH y la producción de CO_2 siendo que ambos tienen un comportamiento exponencial y este puede variar dependiendo de la temperatura y el tiempo de fermentación, lo que esta misma salida afecta los bulgaros siendo estos la variables de entrada de los procesos futuros, de manera que el proceso de fermentación no responden de manera proporcional o predecible a las variables de entrada, como la temperatura o el tiempo de fermentación. Además, su variabilidad en el tiempo es debida a varios factores que pueden cambiar durante el proceso, como el comportamiento de los microorganismos, las condiciones ambientales y las variaciones en los insumos utilizados (por ejemplo, la leche y los búlgaros). Siendo que la fermentación no sigue un patrón completamente predecible, lo que significa que el tiempo necesario para alcanzar el punto óptimo de recolección (pH entre 4.2 y 4.6) puede variar de un lote a otro, lo cual representa un reto para su monitoreo y control mediante técnicas tradicionales, lo que dificulta la

aplicación de modelos estáticos.
Dada la complejidad y variabilidad de este proceso, un modelado de una serie de reglas basada en lógica difusa permite un análisis de variables imprecisas y relaciones no lineales, ya que la lógica difusa es un modelo que permite representar matemáticamente la incertidumbre y la vaguedad de un proceso mediante conjuntos difusos (como unión, interacción o complemento) o un razonamiento aproximado por medio de reglas difusas [32]. Siendo que ya se han propuestos modelos basado en lógica difuso, como el modelo propuesto por Hüseyin Nail Akgül y asociados para el monitoreo del kéfir, lo que demuestra la importancia la aplicación de este, en el proceso de la elaboración del kéfir [15]. Y a su vez, es importante llevar un registro de estos análisis para hacer comparaciones entre lote y lote.

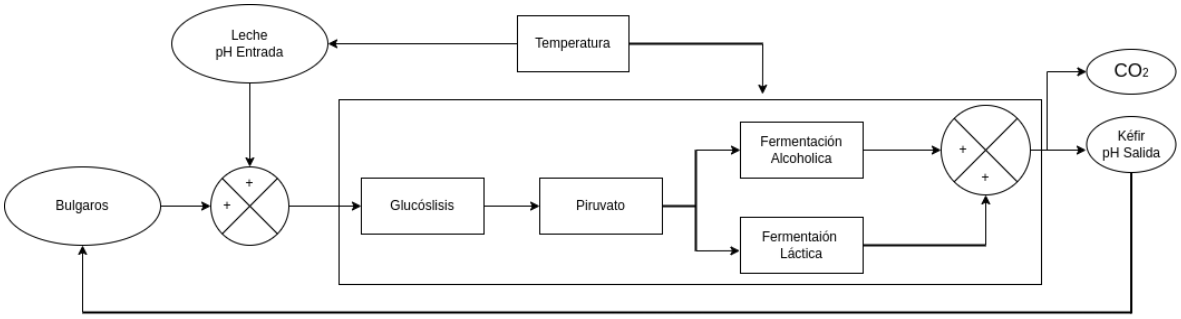


Figura 18: Diagrama del proceso de fermentación del kéfir.

3.4. Sistema Telemático

Según la definición de Santos González, un sistema telemático es el conjunto de elementos interconectados que permiten la transmisión, ,recepción y procesamiento de información a distancia. Para ello, se emplean tanto las telecomunicaciones con el fin de establecer la comunicación entre nodos dispersos geográficamente, como la informática que permite el análisis o el procesamiento de los datos involucrados en dichos nodos. En otras palabras, la télematica representa toda la infraestructura tecnológica especializada que posibilita multiples servicios y aplicaciones que requieran de dichas caracterisiticas para facilitar la comunicación, el almacenamiento e intercambio de datos a distancia de manera eficiente y segura. [33].

Por otro lado, Pascual Viñé define a los sistemas telemáticos como la asosiación de técnicas propias de las telecomunicaciones y la informática, siendo la columna de estos sistemas la transmisión y el procesamiento de datos a distancia, con el objetivo de desarrollar aplicaciones que faciliten la transmisión y el procesamiento de datos a distancia proporcionando a personas no especializadas la posibilidad de acceder a servicios de información y comunicación de manera sencilla y eficiente. [34].

En resumen de ambas definiciones, de un sistema telemático se caracterizan por ser un conjunto de elementos interconectados que integran las telecomunicaciones y la informática para facilitar la transmisión y procesamiento de datos a distancia. Estos sistemas se caracterizan por su capacidad para manejar información de manera eficiente y segura, permitiendo a los usuarios acceder a servicios y aplicaciones sin necesidad de

ser expertos en tecnología. Los sistemas telemáticos permiten la comunicación y el intercambio de datos entre diferentes ubicaciones geográficas, facilitando la conectividad y el acceso a la información en tiempo real. Estos sistemas son el conjunto de herramientas que permiten la integración de tecnologías de comunicación y procesamiento de datos atraves de arquitecturas y protocolos específicos.

Por lo que para desarrollar un sistema telemático es esencial considerar tanto los aspectos de comunicación como los de procesamiento de datos, asegurando una integración efectiva de ambos componentes para lograr una funcionalidad óptima y una experiencia de usuario satisfactoria.

3.4.1. Arquitectura

En base a lo anterior, Pascual Viñé menciona que un sistema telemático debe definir primero su arquitectura, la cual puede ser centralizada lo que le permite gestionar y controlar todos los recursos desde un único punto, , distribuida que reparte las tareas y recursos entre varios nodos conectados en red o mixta que combina elementos de ambas arquitecturas, dependiendo de la distribución de los recursos y la gestión de la información. Siendo que esta elección influye directamente en el diseño y la implementación del sistema telemático. Por otro lado, es fundamental definir el tipo de red que se utilizará, ya que se puede clasificar en una red de área local (LAN) lo que permite la comunicación entre dispositivos dentro de un área geográfica limitada, o una red de área amplia (WAN) que permite la comunicación entre dispositivos en ubicaciones geográficas dispersas, y las MAN (Metropolitan Area Network) que cubren áreas más extensas como ciudades o regiones metropolitanas.
Una vez definida la arquitectura y el tipo de red, es crucial seleccionar la topología de red adecuada, que puede ser en estrella, donde todos los nodos se conectan a un punto central; en bus, donde todos los nodos comparten un canal de comunicación común; en anillo, donde cada nodo se conecta al siguiente formando un circuito cerrado; o en malla, donde cada nodo se conecta directamente a varios otros nodos para aumentar la redundancia y la fiabilidad de la red. [34].

3.4.2. Protocolo de comunicación

En base a lo anterior el siguiente paso es definir los protocolos de comunicación que regirán el intercambio de datos entre los diferentes componentes del sistema telemático. Teniendo en cuenta el modelo OSI (Open Systems Interconnection) que divide las funciones de red en siete capas como se puede observar en la fig. 19 de [34], cada una con responsabilidades específicas en el proceso de comunicación. En donde la capa física se encarga de la transmisión de bits a través del medio físico, la capa de enlace de datos gestiona la comunicación entre nodos adyacentes, la capa de red se encarga del enrutamiento y la entrega de paquetes, la capa de transporte garantiza la entrega confiable de datos entre extremos, la capa de sesión establece y mantiene las conexiones entre aplicaciones, la capa de presentación se encarga de la representación y el formato de los datos, y la capa de aplicación proporciona servicios de red directamente a las aplicaciones del usuario final.

O TCP/IP (Transmission Control Protocol/Internet Protocol) que es el conjunto de protocolos más utilizado en redes modernas, que propone la división en cuatro capas, siendo la primera capa de acceso a la red que combina las funciones de la capa física y la de enlace de datos, la segunda capa de Internet que se encarga del enrutamiento y

la entrega de paquetes, la tercera capa de transporte que garantiza la entrega confiable de datos entre extremos, y la cuarta capa de aplicación que proporciona servicios de red directamente a las aplicaciones del usuario final.

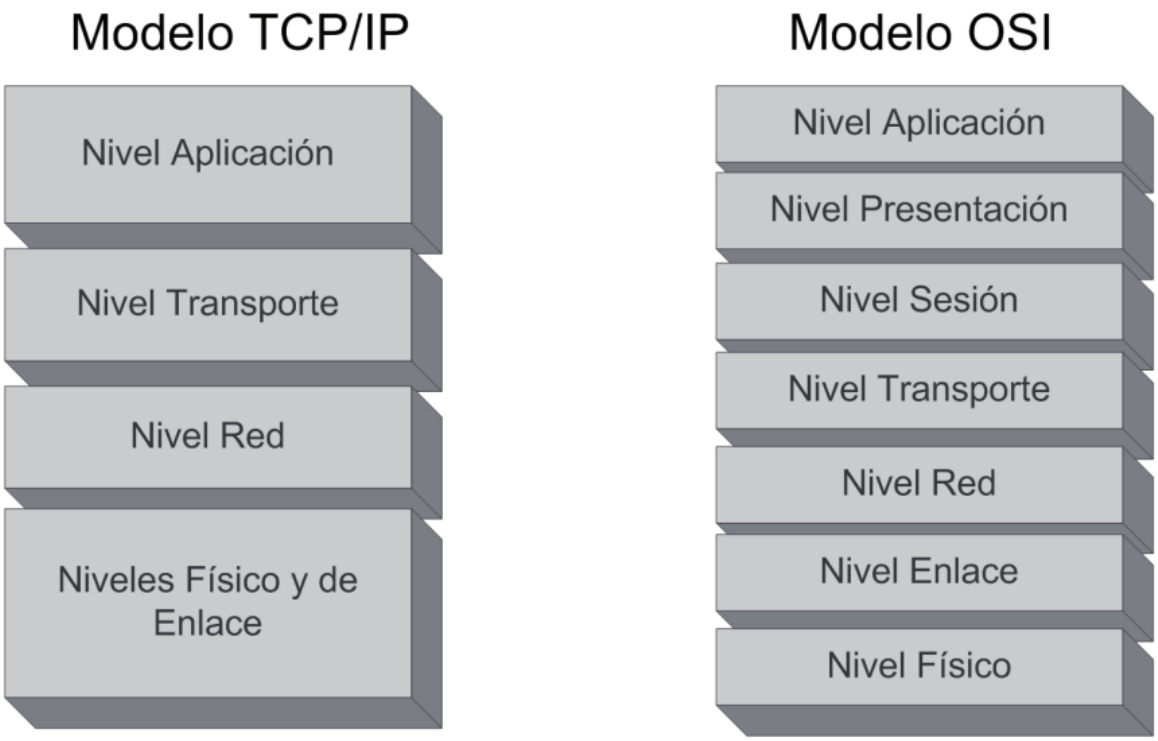


Figura 19: El modelo TCP/IP frente al modelo OSI

Siendo que ambos modelos definen un conjunto de protocolos específicos para cada capa, como TCP, IP, UDP, HTTP, FTP, entre otros, que facilitan la comunicación entre dispositivos y aplicaciones a través de la red. Estos protocolos operan en las diferentes capas del modelo OSI o TCP/IP, asegurando la correcta transmisión, enrutamiento y entrega de datos entre los nodos del sistema telemático. Por ejemplo, en la capa física se utilizan protocolos como Ethernet o Wi-Fi para la transmisión de datos, mientras que en la capa de enlace de datos se emplean protocolos como PPP o HDLC. En la capa de red, el protocolo IP es fundamental para el enrutamiento de paquetes, mientras que en la capa de transporte,TCP y UDP son los principales protocolos utilizados. Finalmente, en la capa de aplicación, protocolos como HTTP, FTP y SMTP facilitan la interacción entre aplicaciones y servicios. Estos protocolos trabajan en conjunto para garantizar una comunicación eficiente y segura en el sistema telemático, permitiendo la interoperabilidad entre diferentes dispositivos y aplicaciones. En resumen, la elección de los protocolos de comunicación adecuados es esencial para el diseño y la implementación de un sistema telemático eficiente y confiable, asegurando una transmisión de datos efectiva entre los diferentes componentes del sistema.

3.4.3. Transmisión de datos

Asimismo,es importante conciderar la transmisión de datos, la cual puede ser síncrona, que permite la transferencia continua de datos en tiempo real, asíncrona que permite la transferencia de datos en intervalos no regulares, o en tiempo real que garantiza la entrega inmediata de datos con baja latencia, dependiendo de los requisitos

específicos del sistema telemático y las necesidades de los usuarios finales. [34]
En base a estas características, se puede diseñar e implementar un sistema telemático eficiente y confiable que facilite la comunicación y el intercambio de datos a distancia. Aunque es importante destacar que la elección de cada uno de estos elementos dependerá de los requisitos específicos del sistema telemático y las necesidades de los usuarios finales. Es por eso que estos sistemas son ampliamente utilizados en diversas aplicaciones debido a su capacidad de integrar soluciones tecnológicas especializadas adapatandose a las necesidades de cada caso en particular.
Pero es importante destacar que sistema telematico tambien se conforma por un hardware y un software, siendo el hardware el conjunto de componentes físicos y tangibles que conforman un sistema telemático, incluyendo dispositivos de comunicación, equipos de procesamiento y almacenamiento, y otros elementos necesarios para su funcionamiento. Mientras que el software es el conjunto de programas, aplicaciones y sistemas operativos que permiten la gestión, control y operación del sistema telemático, facilitando la interacción entre los usuarios y el hardware. Siendo que los componentes presentes en este sistema se describen a continuación:

3.4.4. Microcontrolador

Un microcontrolador es un circuito integrado de gran escala (VLSI), considerado un sistema cerrado y autónomo, siendo diseñados principalmente para sistemas embebidos con funcionalidades específicas y predefinidas donde su programación y configuración determinan completamente su comportamiento. Esto es posible gracias a que está compuesto por una unidad central de procesamiento (CPU), memorias (RAM y ROM), líneas de entrada/salida, temporizadores, conversores analógicos/digital (ADC), generador de reloj y otros módulos periféricos [35].

3.4.5. Backend

Para el desarrollo de cualquier sistema es importante el Back end el cual es la parte lógica y estructural de un sistema o aplicación que opera del lado del servidor, es decir, la sección del software que no es visible ni accesible para el usuario final. Es responsable de gestionar y procesar el acceso a la base de datos, la seguridad, la funcionalidad interna y la optimización de recursos del sistema. Actualmente, existen diversos lenguajes de programación del lado del servidor para desarrollar el back end, cada uno con su propia estructura y enfoque. La arquitectura del back end dependerá en gran medida del lenguaje y del entorno de desarrollo elegido [36].

3.4.6. Base de datos

Una base de datos es un sistema que permite almacenar, organizar y gestionar datos de manera estructurada, lo que facilita su acceso, actualización y recuperación de forma eficiente. Estas se administran mediante un Sistema de Gestión de Bases de Datos (DBMS), el cual permite realizar consultas, modificaciones y análisis, apoyando así la toma de decisiones basada en la información almacenada. Dependiendo del tipo de datos, del lenguaje o framework utilizado, y de los requerimientos del sistema, se puede optar por dos arquitecturas principales: bases de datos relacionales (SQL) y no relacionales (NoSQL) [36].

3.4.7. ACID

En base a lo anteior la elección de una u otra influye directamente en la estructura y organización del proyecto.En la cual esta definida la bases de datos ACID (Atomicity, Consistency, Isolation, Durability) para las bases de datos relacionales, en donde la atomicidad garantiza que todas las operaciones de una transacción se completen correctamente o ninguna lo haga, la consistencia asegura que una transacción lleve a la base de datos de un estado válido a otro estado válido, el aislamiento garantiza que las transacciones concurrentes no interfieran entre sí, y la durabilidad asegura que una vez que una transacción ha sido confirmada, sus cambios se mantendrán incluso en caso de fallos del sistema.

3.4.8. BASE

Por el contrario, BASE (Basically Available, Soft state, Eventual consistency) para las bases de datos no relacionales permiten garantizar que el sistema esté disponible para operaciones de lectura y escritura la mayor parte del tiempo sin necesidad de que un usuaario espere a que otro finalice su operación, esto tambien les permite tener flexibilidad que el sistema pueda cambiar con el tiempo sin necesidad de una actualización inmediata, y la consistencia eventual asegura que, aunque los datos puedan no estar inmediatamente consistentes en todas las réplicas, eventualmente todas las copias de los datos convergerán a un estado consistente [37].

3.4.9. CAP

Sin embargo, independientemente del tipo de base de datos, es esencial diseñar una estructura adecuada que facilite el acceso y la gestión eficiente de los datos, la cual esta estrictamente definida por el teorema CAP (Consistency, Availability, Partition tolerance) que establece que un sistema distribuido pueda ser consistente lo que implica que todos los nodos vean los mismos datos al mismo tiempo, disponible lo que significa que cada solicitud recibe una respuesta, ya sea exitosa o fallida, y tolerante a particiones lo que asegura a el sistema continuar funcionando a pesar de las fallas de comunicación entre nodos. Por lo que solo se pueden garantizar dos de las tres propiedades al mismo tiempo. Por lo tanto, es fundamental para el diseño y la implementación de sistemas distribuidos, en la cual se debe tomar en cuenta si una base de datos prioriza la consistencia y disponibilidad (CA), consistencia y tolerancia a particiones (CP), o disponibilidad y tolerancia a particiones (AP) [38].

3.4.10. Front End

El frontend es la capa de desarrollo de software responsable de la presentación visual de una aplicación o sitio web, es decir, todo aquello con lo que el usuario final puede ver e interactuar. Se encarga de estructurar y mostrar los datos procesados por el backend mediante interfaces gráficas, formularios, botones, elementos multimedia, animaciones y otros componentes visuales. Su objetivo es proporcionar una experiencia de usuario atractiva, accesible, funcional e intuitiva. Dependiendo del framework utilizado, varía la manera en que se organiza y desarrolla esta capa, ya que cada uno propone una arquitectura y jerarquía distinta para construir interfaces, influenciando directamente la lógica y estructura del código visual [36] [39].

3.4.11. Framework

Un framework es un marco de trabajo que simplifica la estructura base para el desarrollo de software mediante herramientas y recursos reutilizables desarrollados por otros programadores, como módulos, librerías y fragmentos de código. Estos elementos permiten resolver problemas comunes de forma más rápida y eficiente. Su objetivo principal es facilitar y agilizar la programación, ayudando a organizar el código, estandarizar procesos y mejorar el trabajo en equipo, ya sea en proyectos generales o en tareas específicas. Actualmente, existen dos tipos principales de frameworks, y la elección de uno u otro influye directamente en la arquitectura y la organización del proyecto: siendo los frameworks frontend (como React o Angular) que permiten crear interfaces visuales e interactivas con las que el usuario final puede interactuar. Y los Frameworks backend (como Django o Express)que se encargan de la lógica del servidor, el manejo de bases de datos y aspectos relacionados con la seguridad [36].

3.4.12. Servidor

El término *servidor* presenta dos significados principales en el ámbito informático. En primer lugar, se refiere al equipo físico que comparte recursos a través de una red; en segundo lugar, al programa que ejecuta servicios en dicho equipo [40].

Por una parte, un servidor basado en *hardware* también denominado *host* es una máquina integrada en una red informática donde, además del sistema operativo, operan uno o varios servidores de software. Por otra parte, un servidor basado en *software* consiste en un programa que ofrece servicios específicos a clientes, ya sea localmente o en red, mediante el modelo cliente-servidor y protocolos de transmisión especializados. Además, los servidores pueden gestionarse de forma física (*on-premise*) o mediante servicios de terceros en la nube. Siendo el despliegue local (*on-premise*), el software que se instala en una infraestructura propia.
A diferencia de las soluciones en la nube, este enfoque garantiza acceso físico, control total sobre configuración, seguridad y datos, sin depender directamente de conexión a internet. En consecuencia, una interrupción en los enlaces no afecta la operatividad de la organización.
Por el contrario, los servidores en la nube (*cloud*) permiten gestionar y acceder a datos y aplicaciones mediante internet de forma ágil. Esto elimina la necesidad de infraestructura física local y se caracteriza por mejorar la movilidad, reducir costos de inversión y mantenimiento, acortar tiempos de implementación y ofrecer escalabilidad inmediata.
En síntesis, mientras el modelo *on-premise* prioriza control y autonomía, el enfoque en la nube destaca por flexibilidad, eficiencia económica y rapidez de adaptación.

4. Capítulo: Requerimientos Funcionales y no Funcionales

En este capítulo se especifican los requerimientos funcionales y no funcionales según el modelo IEEE Std 830-1998, en los cuales se detallan las funcionalidades del sistema, así como los criterios de diseño e implementación. Asimismo, se detallan los casos de uso segun correspanda al requerimiento funcional para garantiza el correcto diseño, implementación y validación del sistema propuesto.

4.1. Módulo 1

1. **RF01-Modos de operación**

   1.1 **Validez de las entradas**

   1.1.1 Se debe implementa un sensor de pH debe estar fabricado de un material de grado alimenticio que no contamine ni interfiera con el proceso de la fermentación, capaz operar en líquidos dentro de un rango de temperatura de 18 °C a 30 °Cz un rango de medición de pH de 4.0 a 7.0 con un tiempo de respuesta de máximo 1 minuto .

   1.1.2 Se debe implementa un sensor de temperatura debe estar fabricado de un material de grado alimenticio capaz de trabajar en líquidos , sin contaminar ni interferir con la fermentación del kéfir, atraves de un rango de medición de 18°C a 30°C.

   1.1.3 Se debe implementa un sensor de CO_2 debe ser capaz de medir en un rango de 0-10,000ppm en el aire dentro del frasco de fermentación, operando en el mismo rango de temperatura (18°C a 30°C).

   1.1.4 Se debe implementa frascos con capacidad minima de 1 litro para asegurar que los sensores de pH y temperatura esten completamente sumergidos en la leche durante la fermentación .

   1.1.5 Se debe implementa un formato de datos deberá integrar las lecturas de los sensores en un solo paquete, incluyendo un identificador único para cada conjunto de datos y una marca de tiempo.

   1.1.6 Se debe implementa un una salida para la liberación del gas CO_2 en las tapas de los frascos, la cual estará conectada mediante una manguera a un segundo frasco con agua (no mayor a 1 litro), destinado a disolver el CO_2 generado durante el proceso.

   1.1.7 Se debe implementa dos modos de operación(modo prueba y modo monitoreo), activados mediante dos botones físicos en el microcontrolador que, al presionarse, inicien el modo correspondiente y permitan salir del mismo al presionar nuevamente el botón.

   1.2 **Secuencia exacta de operaciones**

   1.2.1 El sistema al entrar en modo prueba, deberá verificar el estado de los sensores al enviar los datos de los sensores cada 1 minuto exclusivamente a la página web para visualización temporal, sin almacenarlos en la base de datos.

1.2.2 En modo monitoreo, el sistema deberá verificar que el pH inicial de la leche esté entre 6.6 a 6.8 y su temperaturaesté entre 18°C a 30°C, en caso contrario, no iniciar el monitoreo y enviar una alerta a la base de datos.

1.2.3 En modo monitoreo, el sistema deberá verificar la conexión con la base de datos antes de iniciar la recolección de datos, y en caso de fallo, no iniciar el monitoreo y enviar una alerta.

1.2.4 En modo monitoreo, el sistema deberá iniciar la recolección y envío de datos a la base de datos.

1.2.5 En modo monitoreo, el sistema deberá enviar los datos recopilados por los sensores de pH y CO_2 cada 10 minutos si el pH está en el rango de 6.8 a 5.6.

1.2.6 En modo monitoreo, el sistema deberá enviar los datos recopilados por los sensores cada de pH y CO_2 cada 5 minutos si el pH está en el rango de 5.5 a 5.0.

1.2.7 En modo monitoreo, el sistema deberá recopilar y enviar los datos de los tres sensores (pH, CO_2 y temperatura) cada 1 minuto si el pH está en el rango de 5.0 a 3.9 en adelante.

1.2.8 En modo monitoreo, el sistema deberá recopilar y enviar los datos del sensor de temperatura cada 2.5 minutos de manera independiente, y ajustar la frecuencia a cada 1 minuto de todos los sensores si la temperatura supera los 28°C.

1.3 Respuestas a situaciones anormales

1.3.1 El sistema genera una alerta y la registra en la base de datos si la comunicación con cualquier sensor (pH, temperatura, CO) se interrumpe ya sea en modo prueba o monitoreo.

1.3.2 El sistema retrasmite los datos recolectados por los 3 sensores en caso de un fallo en la conexión con la base de datos.

1.3.3 Si el pH supero el parametro de 3.9 , se genera una alerta y el sistema deja de sensar.

1.4 Efecto de los parámetros

1.4.1 El formato de datos soporta diferentes tipos de datos (números decimales para pH y temperatura, enteros para CO2, cadenas para ID y timestamp) en una estructura anidada, permitiendo flexibilidad para futuros sensores.

1.5 Relación entre las salidas y las entradas

1.5.1 Los sensores proporcionan valores crudos y el formato de datos integra dichos valores de forma anidada.

2. RF02-Cónexion del Microcontrolador a la Base de Datos

2.1 Validez de las entradas

2.1.1 Se debe implementa un protocolo de conexión que garantize la conexión de multiples microcontroladores de manera simultanea e independiente.

2.1.2 Se debe implementa un protocolo de conexión que permita la identificación única de cada microcontrolador mediante un ID.

2.2 **Secuencia exacta de operaciones**

2.2.1 El microcontrolador establece la conexión con la base de datos al encenderse y verificar la conexión antes de enviar cualquier dato.

2.3 **Respuestas a situaciones anormales**

2.3.1 El sistema registra en un log todos los intentos de conexión fallidos al servidor de base de datos.

2.4 **Efecto de los parámetros**

2.4.1 El protocolo de conexión mantiene escalabilidad para soportar un aumento en el número de microcontroladores conectados sin degradar el rendimiento.

2.5 **Relación entre las salidas y las entradas**

2.5.1 El protocolo de conexión del microcontrolador asegura compatibilidad con el servidor de base de datos para asegurar la correcta transmisión de datos.

2.5.2 El protocolo de conexión incluye que los datos enviados a la base de datos incluyan una marca de tiempo para cada lectura, permitiendo un seguimiento preciso de la evolución de los parámetros monitoreados.

2.5.3 El protocolo de conexión garantiza que los datos enviados a la base de datos estén en el formato requerido por el módulo de análisis de datos para su procesamiento posterior.

3. **RF03-Microcontrolador**

3.1 **Validez de las entradas**

3.1.1 Se debe implementa un microcontrolador que disponga las entradas analógicas necesarias para conectar los sensores de pH, temperatura y CO_2.

3.1.2 Se debe implementa un microcontrolador que cuente con memoria suficiente para almacenar datos temporalmente en caso de fallos de conexión con la base de datos.

3.1.3 Se debe implementa un microcontroladror que incluya la conexión requerida para hacer uso del protocolo de conexión.

3.1.4 Se debe implementa un microcontrolador que gestione la lógica de control para los dos modos de operación (prueba y monitoreo) y ajustar la frecuencia de recolección de datos según los parámetros medidos.

3.2 **Secuencia exacta de operaciones**

3.2.1 El microcontrolador adquiere los datos de los sensores de manera secuencial (primero pH, luego CO_2 , luego temperatura) en cada ciclo de medición para optimizar el procesamiento.

3.3 **Respuestas a situaciones anormales**

3.3.1 El sistema emite una alerta y se registra en la base de datos ,si la comunicación con cualquier sensor (pH, temperatura, CO) se interrumpe ya sea en modo prueba o monitoreo.

3.3.2 El sistema retransmite los datos recolectados por los 3 sensores en caso de un fallo en la conexión con la base de datos.

3.3.3 El sistema documenta en un log todos los intentos de conexión fallidos al servidor de base de datos.

3.4 **Efecto de los parámetros**

3.4.1 El microcontrolador modifica la frecuencia de recolección de datos en función del rango de pH medido y la temperaura, como se especifica en la secuencia de operaciones.

3.4.2 La alimentación del microcontrolador mantiene estabilidad y adecuación para garantizar el funcionamiento continuo durante el proceso de fermentación, considerando al menos un intervalo de 12 a 48 horas de operación ininterrumpida.

3.5 **Relación entre las salidas y las entradas**

3.5.1 El microcontrolador procesa lecturas crudas(haciendo la conversión de datos analogicos a digitales) de los sensores y genera paquetes de datos estructurados junto con un identificador del microcontrolador para su envío a la base de datos.

4. **RNF01**

5. 5.1 El sistema debe ser capaz de manejar múltiples conexiones simultáneas de microcontroladores sin degradar el rendimiento.

5.2 El sistema debe ser escalable para soportar un aumento en el número de microcontroladores en base al crecimiento de usuarios.

5.3 El sistema debe perimitir analizar diferentes cepas de bulgaros y diferentes tipos de leches.

A continuación se muestran los casos de uso en las fig. 20 y fig. 21 de los casos de uso del RF01.

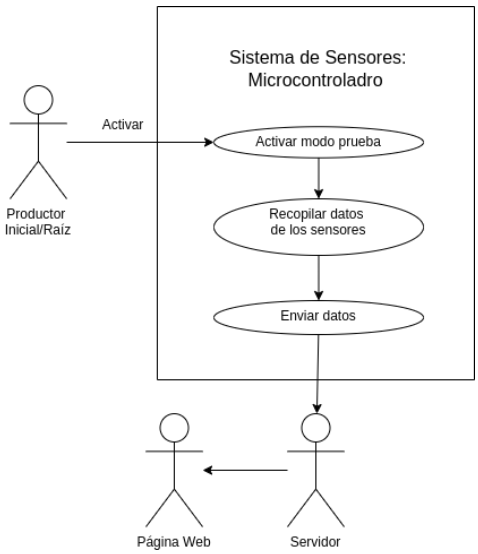


Figura 20: Modo prueba del módulo 1 RF01.

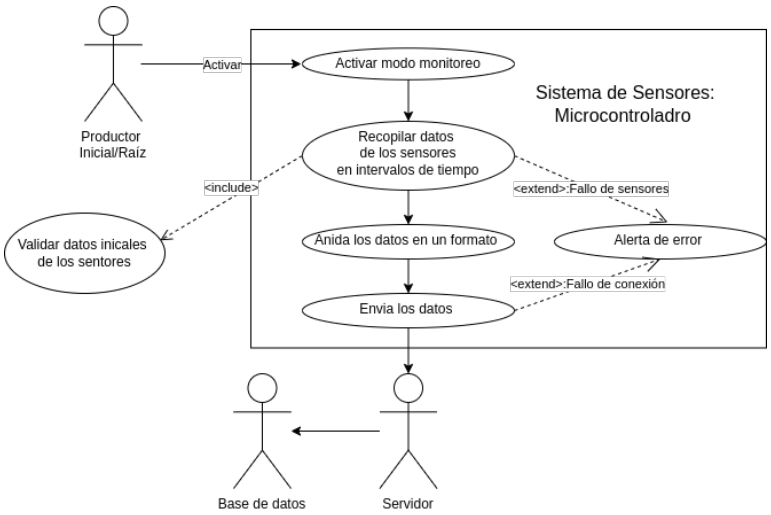


Figura 21: Modo monitoreo del módulo 1 RF01.

4.2. Módulo 2

1. **RF04-Análisis de datos**

   1.1 **Validez de las entradas**

   1.1.1 Se valida que los datos recibidos de la base de datos estén completos y no contengan valores nulos.

   1.1.2 El módulo 2 recibe los datos en el formato especificado por el módulo 1 con su respectivo ID.

   1.1.3 El módoulo 2 realiza el análisis de los datos de acuerdo a los datos de cada usuario.

   1.2 **Secuencia exacta de operaciones**

   1.2.1 El sistema inicia el analisis automáticamente los datos conforme estos sean recibidos, ejecutando el proceso de análisis en tiempo real a medida que llega nueva información desde los sensores.

   1.2.2 El sistema procesa los datos en el orden en que fueron recibidos.

   1.2.3 El sistema produce un informe de análisis después de procesar un lote completo de datos.

   1.3 **Respuestas a situaciones anormales**

   1.3.1 El sistema emite una alerta si se detectan valores fuera del rango esperado en cualquiera de los parámetros (pH, temperatura, CO).

   1.3.2 El sistema registra en un log cualquier error encontrado durante el análisis de datos.

   1.4 **Efecto de los parámetros**

   1.4.1 Se asegura que los datos incluyan un timestamp válido y estén ordenados cronológicamente para garantizar la correcta secuencia temporal del análisis.

1.4.2 El módulo 2 se adapta la frecuencia de los datos recibidos desde la base de datos paara optimizar el análisis de datos.

1.5 **Relación entre las salidas y las entradas**

1.5.1 El módulo 2 actualiza la base de datos con los resultados del análisis para su posterior consulta.

2. **RNF02**

3. 3.1 El mpdulo 2 debe ser capaz de manejar múltiples solicitudes de análisis de datos de diferentes usuarios sin degradar el rendimiento.

3.2 El módulo 2 debe ser escalable para soportar un aumento en el volumen de datos a analizar.

4.3. Módulo 3

1. **RF05-Base de datos**

1.1 **Validez de las entradas**

1.1.1 La Base de datos valida que los datos recibidos del módulo 1 estén completos y no contengan valores nulos.

1.1.2 La Base de datos deberá comprobar que todos los paquetes de datos provenientes de los sensores incluyan la marca de tiempo asociada y su ID único del microcontrolador para asociarlo con su usuario correspondiente.

1.1.3 La Base de datos valida las claves de acceso de los usuarios por medio de la página web.

1.2 **Secuencia exacta de operaciones**

1.2.1 La Base de datos almacena los datos recibidos en el orden en que fueron recibidos.

1.2.2 La Base de datos permite consultas y recuperaciones de datos por parte del módulo de análisis de datos.

1.2.3 La Base de datos envia los datos almacenados a la interfaz web para su visualización cuando sea solicitado por parte del usuario.

1.3 **Respuestas a situaciones anormales**

1.3.1 La Base de datos emite una alerta si se detectan intentos no autorizados de acceso.

1.3.2 La Base de datos registra en un log cualquier error encontrado durante las operaciones de almacenamiento o recuperación de datos.

1.4 **Efecto de los parámetros**

1.4.1 La Base de datos garantiza la compatiblidad con el formato de datos del módulo 1 y 2.

1.4.2 La Base de datos adapta la frecuencia de datos recibidos por parte del módulo 1.

1.4.3 La Base de datos optimiza las consultas si se detectan patrones comunes en las solicitudes de datos.

1.4.4 La Base de datos garantiza una alta dispnibilidad y escalabilidad horizontal de cara a los usuarios.

1.4.5 La Base de datos solo debe permitir el acceso a los datos de los sensores a su usuario correspondientes y al admin, implementando mecanismos de autenticación y autorización adecuados.

1.5 **Relación entre las salidas y las entradas**

1.5.1 La Base de datos permite la entrada de datos desde el módulo 1 para su almacenamiento.

1.5.2 La Base de datos permite la entrada de solicitudes desde el módulo 2 y envía los datos necesarios para el análisis.

1.5.3 La Base de datos permite la entrada de solicitudes desde el módulo 4 y envía los datos correspondientes para su visualización en la página web.

2. **RNF03**

3. 3.1 El mpdulo 3 debe ser capaz de manejar múltiples solicitudes de almacenamiento y recuperación de datos de diferentes usuarios sin degradar el rendimiento.

3.2 El módulo 3 debe ser escalable horizontalmente para soportar un aumento de usuarios y de datos almacenados.

3.3 La Base de datos proporciona acceso rápido y eficiente a los datos almacenados para su análisis y visualización.

3.4 La Base de datos garantiza una consistncia e integridad de los datos almacenados de cada usuario.

4.4. Módulo 4

1. **RF07-Página Web**

1.1 **Validez de las entradas**

1.1.1 El entorno de front de la página web debe permitir compatibilidad con el formato de datos del módulo 1,2 y 3 para su futura visualización.

1.1.2 El entorno de front end de la página web debe integrar una interfaz renderizada para gráficos y formularios interactivos para la visualización de datos en tiempo real.

1.2 **Secuencia exacta de operaciones**

1.2.1 La página web debe integrar un sistema de autenticación seguro para que los usuarios puedan iniciar sesión con sus credenciales.

1.2.2 La página web debe permitir a los usuarios visualizar los datos de sus fermentaciones en tiempo real.

1.3 **Respuestas a situaciones anormales**

1.3.1 El entorno de front de la página web debe registrar en un log cualquier intento fallido de conexion hacia la base de datos.

1.4 **Efecto de los parámetros**

1.4.1 El entorno de front de la págia web debe poder permitir tener múltiples usuarios conectados de manera simultanea sin degradar el rendimiento.

1.4.2 El entorno de front de la págia web debe escalar gráficos según el volumen de información obtenida de la base de datos.

1.5 **Relación entre las salidas y las entradas**

1.5.1 El entorno de front de la página web debe permitir una conexión bidireccional con la base de datos para enviar y recibir datos en tiempo real.

2. **RF08-Página Web:Iniciar Sesión**

2.1 **Validez de las entradas**

2.1.1 La página web solicita por medio de un solo formulario de registro las credenciales(nombre y contraseña) de acceso del usuario o admin para su validacíon en la base de datos.

2.1.2 La página web debe permitir recuperar la contraseña en caso de olvido por medio de un correo electrónico asociado a la cuenta.

2.2 **Secuencia exacta de operaciones**

2.2.1 El usuario ingresa sus credenciales en el formulario de inicio de sesión.

2.2.2 La página web envía las credenciales a la base de datos para su validación.

2.2.3 La base de datos verifica las credenciales y responde a la página web con el resultado.

2.2.4 Si las credenciales son válidas, la página web redirige al usuario a la página principal.

2.3 **Respuestas a situaciones anormales**

2.3.1 Si las credenciales son inválidas, la página web muestra un mensaje de error y permite al usuario intentar iniciar sesión nuevamente.

2.4 **Efecto de los parámetros**

2.4.1 La página web debe cifrar las credenciales del usuario o admin antes de enviarlas a la base de datos para garantizar la seguridad.

2.5 **Relación entre las salidas y las entradas**

2.5.1 La página debe generar un token de sesión único para el usuario o admin al iniciar sesión exitosamente, que se utilizará para mantener la sesión activa.

3. **RF09-Página Web:Iniciar/terminar una fermentación**

3.1 **Validez de las entradas**

3.1.1 La página web por medio de un formulario solicita los datos (cantidad de búlgaros, tipo de leche y cantidad) para iniciar una fermentación.

3.1.2 La página web muestra la fermentación activa en el módulo 1 del usuario por medio de un grafico.

3.1.3 La página web permite al usuario registrar la finalización de la fermentación en el módulo 1.

3.2 **Secuencia exacta de operaciones**

3.2.1 El ususario debe inicar la fermentación en el módulo 1 por medio del modo monitoreo y posteriormente registrar los datos en la página web por medio del formulario.

3.2.2 La página web muestra el estado de la fermentación en tiempo real.

3.2.3 El usuario debe terminar la fermentación en el módulo 1 y posteriormente registrar la finalización en la página web.

3.2.4 La página web actualiza el estado de la fermentación a "terminadaz muestra el analisis de los datos recopilados durante el proceso.

3.3 **Respuestas a situaciones anormales**

3.3.1 La fermentación no puede ser iniciada si el módulo 1 no está encendido o este determina si la fermentación esta en malas condiciones desde el inicio.

3.4 **Efecto de los parámetros**

3.4.1 La página web adapta la visualización del estado de la fermentación según los datos recibidos del módulo 1 en tiempo real.

3.5 **Relación entre las salidas y las entradas**

3.5.1 La página web muestra los datos de la fermentación en tiempo real de pH, temperatura y CO2.

4. **RF10-Página web:Historial**

4.1 **Validez de las entradas**

4.1.1 La página web por medio de un formulario solicita la o las fermentaciónes específicas del usuario para mostrar o comparar en su historial.

4.1.2 La página web debe mostrar el historial por medio de gráficos y tablas de las fermentaciones pasadas del usuario.

4.2 **Secuencia exacta de operaciones**

4.2.1 La página web por medio de la base de datos valida que el usuario tenga permisos para acceder al historial solicitado.

4.3 **Respuestas a situaciones anormales**

4.3.1 La página web muestra un mensaje de error y redirige a la página principal si no se encuentran fermentaciones pasadas asociadas al usuario

4.4 **Efecto de los parámetros**

4.4.1 La página web adapta la visualización del historial según los datos recibidos de la base de datos.

4.4.2 La página web permite comparar múltiples fermentaciones segun el parametro especificado por el usuario.

4.5 **Relación entre las salidas y las entradas**

4.5.1 Los gráficos y tablas deben mostrar los datos históricos de pH, temperatura y CO2 de las fermentaciones pasadas segun lo solicite el usuario.

5. **RF11-Página web:administrador**

5.1 **Validez de las entradas**

5.1.1 El administrador por medio de un formulario que le solicite nombre, edad, correo,contraseña, municipio y categoria(productor inicial o raíz). puede crear, suspender o activar usuarios del sistema.

5.1.2 El administrador puede visualizar todas las fermentaciones activas y pasadas de todos los usuarios.

5.2 **Secuencia exacta de operaciones**

5.2.1 El administrador ingresa los datos del usuario en el formulario para crear un nuevo usuario.

5.2.2 La página web envía los datos a la base de datos para crear el nuevo usuario.

5.2.3 La base de datos confirma la creación del usuario y responde a la página web con el resultado.

5.2.4 Si el administrador desea suspender un usuario, selecciona el usuario de una lista y confirma la suspensión.

5.2.5 La página web envía la solicitud de suspensión a la base de datos.

5.3 **Respuestas a situaciones anormales**

5.3.1 El administrador recibe un mensaje de error si intenta crear un usuario con un nombre o correo ya existente.

5.3.2 El administrador recibe un mensaje de confirmación al suspender un usuario exitosamente.

5.4 **Efecto de los parámetros**

5.4.1 En caso de crear un usuario con categoria "productor raíz", la página debe solicitar información sobre de que otor usuario esta asociados.

5.5 **Relación entre las salidas y las entradas**

5.5.1 El formulario debe solicitar los datos necesarios para crear un nuevo usuario.

5.5.2 La página web debe mostrar una lista actualizada de todos los usuarios del sistema, incluyendo su estado (activo o suspendido).

6. **RNF04**

7. 7.1 El mpdulo 4 debe ser capaz soportar multiples usuarios conectados de manera simultanea sin degradar el rendimiento.

7.2 El módulo 4 debe ofrecer una interfaz amigable e intuitiva de cara al usuarios.

A continuación se muestran los diagramas de caso de uso de los requerimientos funcionales correspondientes de la página web:

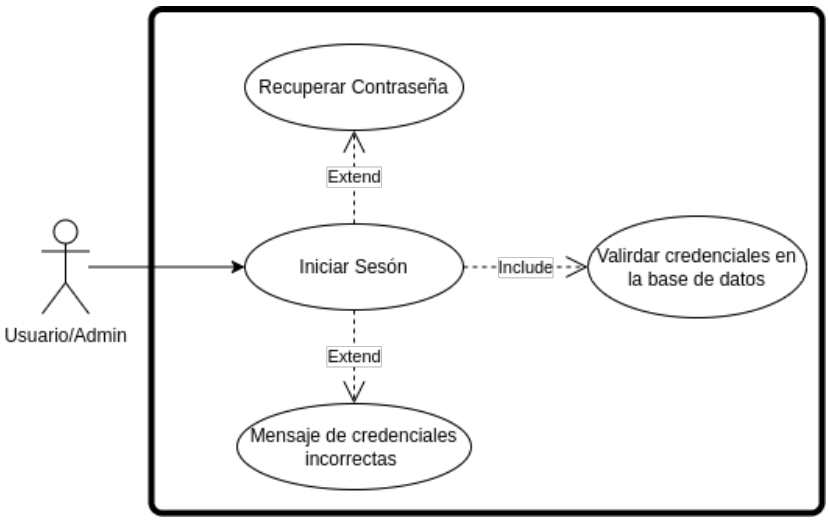


Figura 22: Iniciar Sesón Página Web RF08.

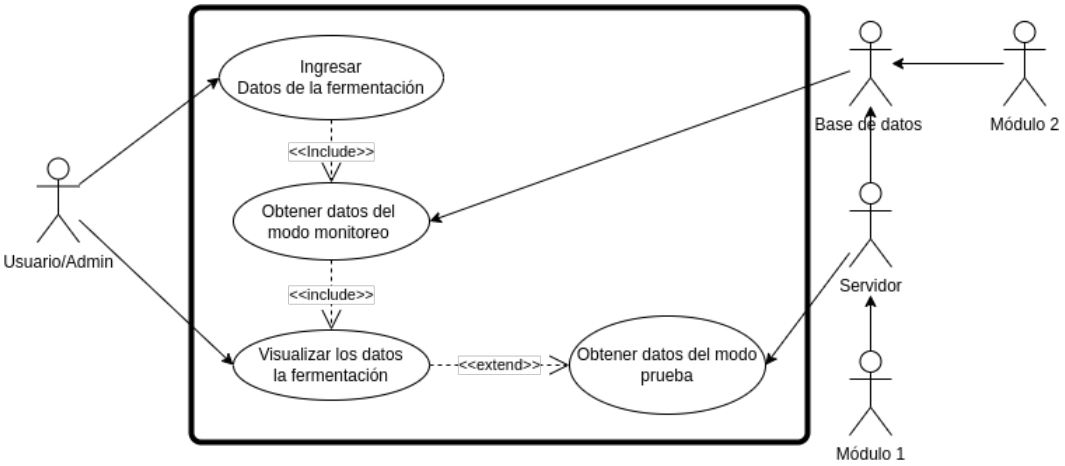


Figura 23: Iniciar/Terminar una fermentación RF09.

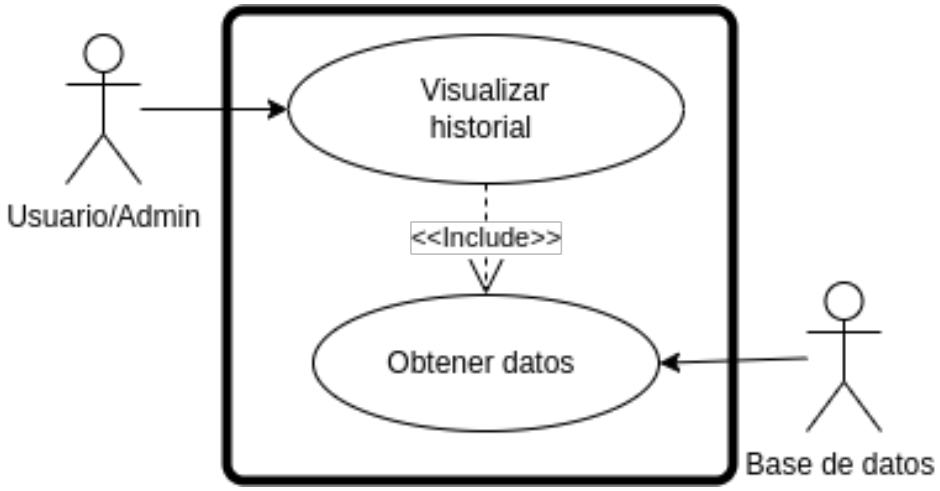


Figura 24: visualizar historial de fermentación RF11.

4.5. Servidor

1. **RF12-Servidor**

1.1 **Validez de las entradas**

1.1.1 El entorno de desarrollo del servidor debe permitir compatibilidad con el formato de datos del módulo 1,2,3 y 4 para su futura integración.

1.1.2 El entorno de desarrollo del servidor debe gestionar las solicitudes de los diferentes módulos de manera eficiente.

1.1.3 El entorno de desarrollo del servidor debe implementar mecanismos de seguridad para proteger los datos transmitidos entre los módulos.

1.1.4 El entorno de desarrollo del servidor debe manejar múltiples conexiones simultáneas sin degradar el rendimiento.

1.2 **Secuencia exacta de operaciones**

1.2.1 El servidor recibe datos del módulo 1 y los almacena en la base de datos del módulo 3.

1.2.2 El servidor recibe solicitudes de análisis de datos del módulo 2 y envía los datos necesarios desde la base de datos.

1.2.3 El servidor recibe solicitudes de la página web del módulo 4 y envía los datos correspondientes desde la base de datos.

1.3 **Respuestas a situaciones anormales**

1.3.1 El servidor emite una alerta si se detectan intentos no autorizados de acceso.

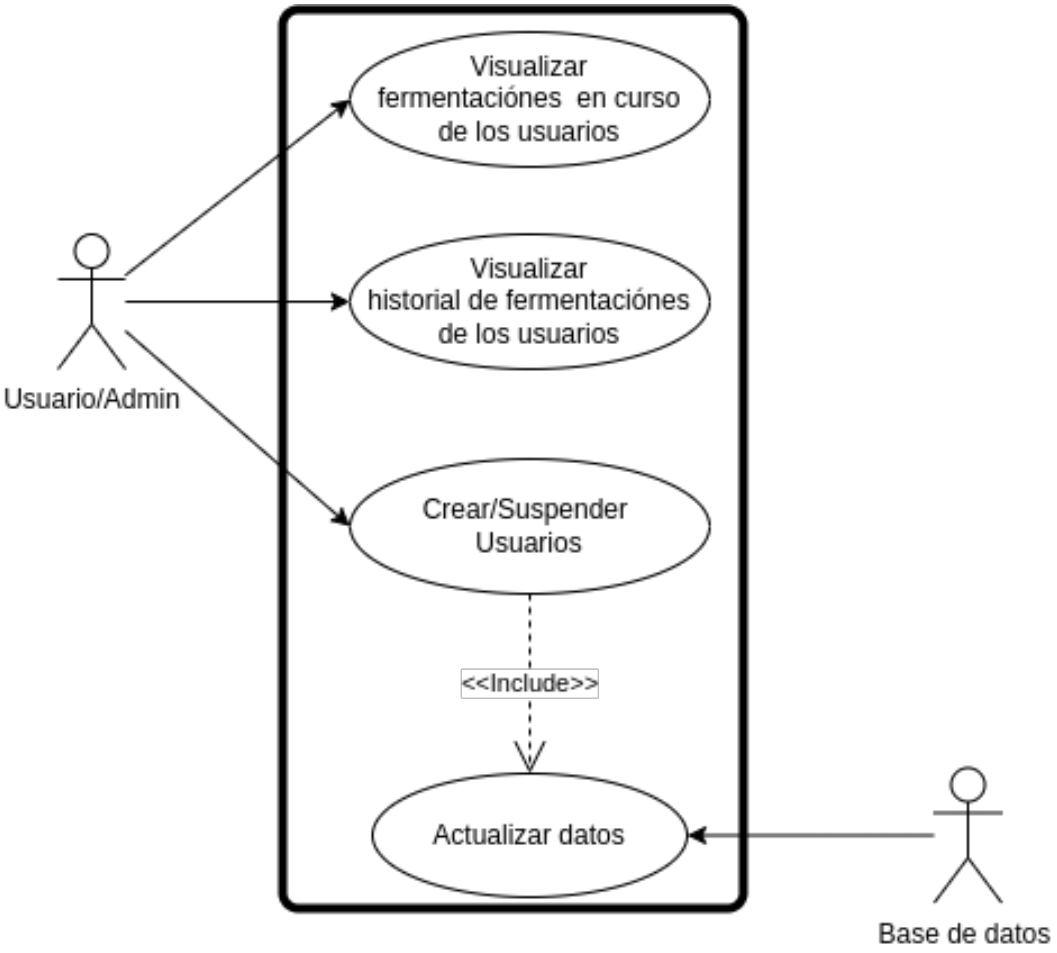


Figura 25: Diagrama de administrador RF11.

1.3.2 El servidor registra en un log cualquier error encontrado durante las operaciones de transmisión de datos entre los módulos.

1.4 **Efecto de los parámetros**

1.4.1 El servidor debe adaptarse a la gestión de solicitudes según el volumen de datos transmitidos entre los módulos.

1.5 **Relación entre las salidas y las entradas**

1.5.1 El servidor deber permitir la entrada de datos desde el módulo 1 y enviarlos al módulo 3 para su almacenamiento.

1.5.2 El servidor debe permitir la entrada de solicitudes desde el módulo 2 y enviar los datos necesarios desde el módulo 3.

1.5.3 El servidor debe permitir la entrada de solicitudes desde el módulo 4 y enviar los datos correspondientes desde el módulo 3.

2. **RNF05**

3. 3.1 El servidor debe garantizar la integración y comunicación entre todos lo módulos del sistema.

3.2 El servidor debe ser escalable para soportar un aumento en el volumen de datos y solicitudes entre los módulos.

3.3 El servidor debe implementar medidas de seguridad para proteger los datos transmitidos entre los módulos

5. Capitulo: Análisis

A continuacón se presenta un análisis, comparación y selecccion de las herramientas y tecnologías a utilizar en el desarrollo de este proyecto ,así como las razones que justifican su elección considerando aspectos técnicos, económicos y de disponibilidad en base a las necesidades del proyecto. Se incluyen herramientas de hardware, software y plataformas evalúando las ventajas y desventajas de cada una, así como su adecuación al proyecto.

5.1. Análisis del Módulo 1

En esta sección se presenta un análisis,comparacción y selección tanto de los sensores,como del microcontrolador a implementar en el Módulo 1 de este proyecto mediante una tabla comparativa de las diferentes opciones disponibles en el mercado que cumplen con los requerimientos funcionales del proyecto. A continuació se describen los componentes principales del Módulo 1:

5.1.1. Sensor de pH

Para medir el pH del kéfir de leche durante la fermentación, se utilizará un sensor de pH analógico. Este sensor debe ser capaz de proporcionar una salida de voltaje proporcional al valor de pH medido. Para la selección del sensor de pH, se consideraron los siguientes criterios en base a los requeriminetos funcionales RF01 del proyecto:

- **No contaminante:** El sensor debe ser adecuado para su uso en líquidos, en especial, en fermentaciones y no debe estar hecho de un material que pueda contaminar el kéfir por lo que es recomendado que este hecho de algun polimero de grado alimenticio, vidiro o de ceramica.
- **Rango de medición:**El sensor debe cubrir el rango de pH esperado durante la fermentación del kéfir de leche, que típicamente varía entre 6.8(inicial) y 4.6(final).
- **Precisión:**El sensor debe tener una precisión adecuada para detectar cambios pequeños en el pH, idealmente con una precisión de ± 0.1 pH.
- **Salida:**Se consideraron sensores con conectores estándar (como BNC) para facilitar una salida digital y una conexión con el microcontrolador.
- **Temperatura de operación:**El sensor debe ser capaz de operar en el rango de temperatura esperado durante la fermentación (18-30 °C).
- **Disponibilidad y costo:**Se consideraron sensores que sean fácilmente adquiribles y con un costo razonable para el proyecto.

A continuación se presentan la con diferentes sensores de pH considerados para el proyecto:

Tabla 2: Sensores de pH.

Modelo	**Rango**	**Voltaje**	**Referencia**	**Precisión**	**Tiempo de Respuesta**	**Temperatura de operación °C**	**Cuerpo**	**Punta**	**Tipo de Salida**	**Precio (MXN)**	**Imagen**
Hanna Instruments-FC200B [41]	0 a 12pH	3.3V-5.5V	Ag/AgC (Plata/cloruro de plata)	0,1	N/A	0 a 50°C	PVDF	Cónica de vidrio	BNC	$5233.92	
Milwaukee SE-220 [42]	0 a 14pH	3.3V-5.5V	doble unión Ag/AgC	0,02	N/A	-5 a 70°C	Resina-PEI	esférica	BNC	$1762	
Atlas Científic-ENV-40-pH [43]	0 a 14pH	3.3V-5.5V	doble unión Ag/AgC	0,001	1 seg.	5 - 99°C	Policarbonato	esférica	BNC	$3000-4000	
Atlas Científic-ENV-30-pH [44]	2 a 12pH	3.3V-5.5V	Ag/AgC	0.1	4 seg.	0-60°C	Policarbonato	esférica	BNC	$1000-2000	
PH-E201C [45]	0 a 14pH	5V	Ag/AgCl	0.02	5 seg.	0-80°C	Policarbonato	esférica	BNC	$250-500	

Tras un análisis comparativo detallado de los sensores presentados en la Tabla No. 2, se ha determinado que el electrodo PH-E201C [45] es la opción más viable para la integración en este proyecto. Si bien los modelos alternativos cumplen con la mayoría de los requerimientos funcionales, el PH-E201C ofrece una combinación óptima de rendimiento y coste. Este sensor destaca por su excelente precisión de 0.02 unidades de pH y un tiempo de respuesta rápido de 5 segundos. Además de estas características técnicas sólidas, su bajo coste y alta disponibilidad en el mercado lo convierten en la solución más accesible y eficiente frente a las opciones de mayor precio.

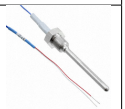
5.1.2. Sensor de Temperatura

Para medir la temperatura del kéfir de leche durante la fermentación, se utilizará un sensor de temperatura digital. Este sensor debe ser capaz de proporcionar una lectura precisa de la temperatura en grados Celsius. Para la selección del sensor de temperatura, se consideraron los siguientes criterios en base a los requeriminetos funcionales RF01 del proyecto:

- **No contaminante:** El sensor debe ser adecuado para su uso en líquidos, en especial, en fermentaciones y no debe estar hecho de un material que pueda contaminar el kéfir por lo que es recomendado que este hecho de acero inoxidable de grado alimenticio.
- **Rango de medición:**El sensor debe cubrir el rango de temperatura esperado durante la fermentación del kéfir de leche, que típicamente varía entre 18 y 30 °C.
- **Precisión:**El sensor debe tener una precisión adecuada para detectar cambios pequeños en la temperatura, idealmente con una precisión de ±0.5 °C.
- **Salida:**Se consideraron sensores con salida digital (como I2C o 1-Wire) para facilitar la integración con el microcontrolador.
- **Disponibilidad y costo:**Se consideraron sensores que sean fácilmente adquiribles y con un costo razonable para el proyecto.

A continuación se presentan la tabla diferentes sensores de temperatura considerados para el proyecto:

Tabla 3: sensores de temperatura.

Modelo	Rangos de medición	Voltaje	Material	Error	Tipo de salida	Precio (MXN)	Imagen
DS18B20 [46]	–55 °C a 125 °C	3.3–5.5 V	Acero inoxidable	–10 °C a 85 °C ±0.5 °C	1-Wire	$50–150	
E52-CA6DN 4M [47]	0–350 °C	3.3–5.5 V	Acero inoxidable	0–333 °C ±2.5 °C	1-Wire	$2104	

En base a los sensores presentados en la Tabla No. 3, el sensor DS18B20 [46] se integra mucho mejor al proyecto a diferencia de los otros sensores, ya que presenta un índice de error de solo +/-0.5C combinada con su bajo coste lo convierte en la elección más eficiente.

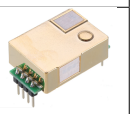
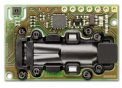
5.1.3. Sensor de CO_2

Para medir la concentración de dióxido de carbono (CO_2) en el ambiente circundante al kéfir de leche durante la fermentación, se utilizará un sensor de CO_2. Este sensor debe ser capaz de proporcionar una lectura precisa de la concentración de CO_2 en partes por millón (ppm). Para la selección del sensor de CO_2, se consideraron los siguientes criterios en base a los requeriminetos funcionales RF01 del proyecto:

- **Rango de medición:**El sensor debe cubrir el rango de concentración de CO_2 esperado durante la fermentación del kéfir de leche, que típicamente varía entre 0 y 5000 ppm.
- **Precisión:**El sensor debe tener una precisión adecuada para detectar cambios pequeños en la concentración de CO_2, idealmente con una precisión de ±50 ppm.
- **Salida:**Se consideraron sensores con salida digital (como I2C o UART) para facilitar la integración con el microcontrolador.
- **Temperatura de operación:**El sensor debe ser capaz de operar en el rango de temperatura esperado durante la fermentación (18-30 °C).
- **Disponibilidad y costo:**Se consideraron sensores que sean fácilmente adquiribles y con un costo razonable para el proyecto.

A continuación se presentan la tabla diferentes sensores de CO_2 considerados para el proyecto:

Tabla 4: Sensores de CO_2

Sensor	**Precisión**	**Rango de medición**	**Temp. de operación °C**	**Voltaje / Consumo**	**Precio aprox. (MXN)**	**Imagen**
MQ-135 [48]	N/A	10-1000 ppm (estimado, otros gases)	-20 a +50°C	5 V	$100-200	
MH-Z19 / MH-Z19B [49]	+/- (50ppm+5 %)	400-10000 ppm	-10 a 50°C	3.6-5.5 V	$700-1700	
SCD30 (Sensirion) [50]	+/- (30ppm+3 %)	0-40000 ppm	0 a 50°C	3.3-5.5 V	$780-1500	
MG-811 [51]	+/- (30ppm+5 %)	350-10000 ppm	20 a 50°C	5-6 V	$600-740	

Uno de los parámetros más difíciles de medir en la fermentación es el CO_2. Dado que la fermentación del kéfir libera grandes cantidades de este gas, se requiere un sensor con una excelente precisión y un amplio rango de medición. En base a los sensores presentados en la Tabla No. 4, el sensor **SCD30 (Sensirion)** [50] es la mejor opción para el proyecto. Esto se debe a que presenta un índice de error de solo +/-(30ppm+3 %) y un rango de medición superior (0-40,000 ppm). Aunque no es el más barato, su especialización en la medición de CO_2 evita los problemas de precisión que tendrían otros

sensores de menor calidad, como el MQ-135 (sensible a otros gases), o el rango limitado de los modelos MG-811 y MH-Z19 en el contexto específico de la fermentación del kéfir.

Para facilitar la elección del microcontrolador, se van analizar los siguientes puntos, esto con la finalidad de tener un mejor entendimiento de las capacidades y limitaciones de cada uno.

5.1.4. Formato de Datos

La elección del formato de datos es un aspecto crítico en el diseño del sistemas , ya que influye directamente en la eficiencia, la interoperabilidad y la escalabilidad del sistema. Con base en los formatos analizados en la Tabla No. 5 y los requerimientos funcionales RF01, se identifica a JSON (JavaScript Object Notation) como la opción más adecuada esto debido a que permite estructurar datos de manera jerárquica, agrupando valores y metadatos en objetos anidados, lo que facilita la integración entre los módulos del sistema, características que cumplen con los requerimientos funcionales de interoperabilidad, eficiencia y escalabilidad. En contraste, otros formatos presentan limitaciones significativas. CSV, aunque simple y compacto, carece de anidamiento y contexto, lo que lo hace inadecuado para datos complejos o flexibilidad en su estructura, ya que cualquier cambio en los datos implica modificaciones manuales en el esquema de lectura. Por su parte, XML, si bien es robusto para manejar metadatos complejos y ofrece validación estricta de esquemas, resulta demasiado pesado y verboso debido a su estructura de etiquetas, lo que incrementa el consumo de recursos en los sistemas.

Tabla 5: Formatos de Datos (Continúa en la página siguiente)

Nombre	**Tipo de Formato**	**Estructura Base**	**Caracterisit- icas**	**Enfoque**	**Ventaja Principal**	**Desventaja Principal**
JSON [52]	Clave-Valor	JSON objetos anidados y arrays	Elimina la necesidad de un diseño de esquemas predeterminados al serializar datos semiestructurados y transmitirlos.	Enfoque a desarrollo con documentos JSON.	Simplicidad y legibilidad lo que permite gran escalabilidad en diferentes lenguajes de programación	Mayor consumo de memoria dependiendo del formato JSON.
CSV [53]	Comma Separated Values	Valores separados por un delimitador (coma, |, etc.). Sin anidamiento.	Permite la iteración de grandes cadenas de valores	Enfoque más especializado a hojas de datos.	Los valores son obvios, sin etiquetas.	Al trabajar con solo valores y delimitadores lo que ocasiona escasa escalabilidad.

Continúa en la página siguiente

Tabla 5 – continuación de la página anterior

Nombre	Tipo de Formato	Estructura Base	Caracterisit- icas	Enfoque	Ventaja Principal	Desventaja Principal
XML [54]	Lenguaje de marcado extensible	Documento estructurado con etiquetas jerárquicas.	Documentos XML están compuestos por una estructura física la cual contiene entidades y una lógica como referencias de caracteres, instrucciones de procesamiento, etc.	Enfocado a facilitar la implementación para la interoperabilidad con SGML y HTML.	Permite metadatos complejos y validación estricta del esquema (*Schema Validation*).	Debido al cierre de etiquetas se vuelve demasiado pesado para microcontroladores.

5.1.5. Topología de red

En esta sección, se analiza y selecciona la topología de conexión a imple- mentar específicamente para la comunicación entre el microcontrolador y el servidor. Con base en los requerimientos funcionales RF02 establecidos y las características de las topologías de red presentadas en la Tabla No. 6 comparativa, se optó por la topología en estrella. Esta elección se fundamenta en su capacidad para permitir la conexión de múltiples microcontroladores de manera independiente, de modo que el fallo de uno no comprometa la comunicación de los demás con el servidor. Además, esta topología ofrece una notable escalabilidad, lo que facilita la incorporación de nuevos dispositivos sin afectar el rendimiento general del sistema.
En contraste, otras topologías, como la de bus o anillo, presentan limitaciones significativas en términos de tolerancia a fallos y capacidad de expansión, ya que un fallo en un punto crítico puede interrumpir toda la red o dificultar la adición de nuevos nodos. Por otro lado, la topología en malla, aunque ofrece alta redundancia, implica una complejidad y costo elevados debido a la necesidad de múltiples conexiones directas entre los dispositivos, lo que no es necesario para los requerimientos del proyecto. Y la topología en árbol jerárquico, aunque escalable, introduce una dependencia crítica en el nodo raíz, cuyo fallo puede afectar a toda la red, además de una mayor complejidad en su gestión.
Por lo tanto,la topología en estrella se considera la más adecuada ya que se ajusta a los requerimientos funcionales RF02 y no funcionales RNF01 del sistema, garantizando una comunicación robusta, eficiente y adaptable a futuras necesidades.

Tabla 6: Topologías de conexión

Topología [55]	**Descripción**	**Ventajas**	**Desventajas**
Bus o multiacceso	Todos los dispositivos están conectados a un único medio de transmisión compartido, formando una línea común.	Fácil de implementar y configurar, adecuado para redes pequeñas con costos iniciales bajos.	Un fallo en el medio principal afecta toda la red. Dificultad para diagnosticar problemas y rendimiento que disminuye con muchos dispositivos debido a colisiones.
Anillo	Cada dispositivo está conectado al siguiente, formando un círculo cerrado donde los datos fluyen en una dirección.	Transmisión ordenada que evita colisiones. Rendimiento predecible en redes pequeñas y fácil de gestionar en configuraciones simples.	Un fallo en un dispositivo o enlace puede interrumpir toda la red. Difícil de reconfigurar o escalar, y presenta retrasos en redes grandes debido al paso secuencial de datos.
Malla	Cada dispositivo está conectado directamente a todos los demás, formando múltiples enlaces.	Proporciona alta redundancia y tolerancia a fallos, gran capacidad de transmisión debido a conexiones directas y escalabilidad en redes críticas.	Configuración y mantenimiento complejos. Costos elevados debido a múltiples conexiones, poco práctica para redes muy grandes.
Estrella	Todos los dispositivos están conectados a un punto central, como un *switch* o *hub*.	Fácil de instalar y gestionar. Un fallo en un dispositivo no afecta a los demás y facilita el escalado al añadir nuevos dispositivos.	Depende del punto central, cuyo fallo afecta toda la red. Costos mayores debido al dispositivo central y mayor complejidad en su gestión.
Árbol jerárquico	Combina topologías en estrella organizadas en niveles, con un nodo raíz que conecta subredes.	Escalable para redes grandes y jerárquicas. Facilita la gestión segmentada de la red y permite una expansión modular.	Presenta complejidad en diseño y configuración. Depende del nodo raíz, cuyo fallo afecta niveles inferiores. Costos elevados por la infraestructura jerárquica.

5.1.6. Protocolo de Comunicación

En la Tabla No. 7 se presenta una comparación de los principales protocolos de acceso a la red inalámbrica considerados para la comunicación entre el Módulo 1 y el servidor. Y en base a los requerimientos funcionales RF01 y no funcionales RNF01 del proyecto, se selecciono Wi-Fi (IEEE 802.11) como el protocolo más adecuado para este proyecto,ya que ofrece altas tasas de transferencia de datos y aunque su consumo de energía es mayor en comparación con otros protocolos como Bluetooth Low Energy (BLE) o Zigbee, ofrece la ventaja de utilizar la infraestructura de red existente en la mayoría de los entornos domésticos y comerciales.
A diferencia de otros protocolos como LoRaWAN, que aunque ofrece una gran distancia de comunicación y bajo consumo de energía, su baja tasa de datos y un ciclo de trabajo limitado restringen su escalabilidad a más dispositivos ya que el ancho de bada se comparte.Al igual que Z-Wave y Zigbee, que requieren un *gateway* o *hub* dedicado para conectarse a la red IP (Internet). En contraste, Bluetooth Classic y BLE presentan limitaciones en alcance y número de nodos conectados, por lo que Wi-Fi se posiciona como la opción más equilibrada para cumplir con los requisitos del proyecto. Aunque no se descarta la posibilidad de usar Zigbee, ya que proporciona una red de malla escalable, robusta y de muy bajo consumo de energía, para múltiples dispositivos en una red local.

Tabla 7: Protocolos de Acceso a la Red

Protocolo	**Tipo de Red/Estándar**	**Protocolo MAC**	**Banda de Operación**	**Velocidad**	**Ventajas**	**Desventajas**
Wi-Fi (IEEE 802.11) [56]	LAN (Local Area Network)	CSMA/CA	2.4/5 GHz	11,54 y 600 Mbps	Ofrece altas tasas de transferencia de datos y utiliza infraestructura de red existente.	Alto consumo de energía y limitado número de nodos conectados a un solo router.
Bluetooth Classic (IEEE 802.15.1) [57]	PAN (Personal Area Network)	TDD-TDMA	2.4 GHz	1–3 Mbps	Ofrece alta tasa de datos en un enlace punto a punto para transmisión de voz o audio.	Alto consumo de energía y alcance limitado en la transferencia de datos.
Bluetooth Low Energy (BLE) (IEEE 802.15.1) [57]	PAN/Malla	CSMA/CA	2.4 GHz	1–2 Mbps	Permite una operación de muy bajo consumo de energía, ideal para dispositivos a batería.	Su velocidad de transferencia de datos más baja en comparación con otras tecnologías.
Zigbee (IEEE 802.15.4) [58]	Malla (*Mesh*)	CSMA/CA	2.4 GHz	250 Kbps	Proporciona una red de malla escalable,robusta y fiable.	Requiere un *gateway* o *hub* dedicado para conectarse a la red IP (Internet).
Z-Wave (ITU-T G.9959) [59]	Malla (*Mesh*)	CSMA/CA	Sub-GHz	100 Kpbs	Opera en frecuencias más bajas, evitando la congestión de la banda de 2.4 GHz y mejorando la penetración en paredes.	Es una tecnología de código cerrado y está limitado a menos dispositivos en la red de malla.
LoRaWAN (LoRa Alliance) [60]	LPWAN (Low Power Wide Area Network)	ALOHA	Sub-GHz	0.0003–0.05 Kbps	Ofrece la mayor distancia de comunicación con un consumo de energía extremadamente bajo.	Su baja tasa de datos y un ciclo de trabajo limitado restringen su escalailidad a más dispositivos ya que el ancho de bada se comparte.

Para garantizar una comunicación eficiente y confiable entre el Módulo 1 y el Servidor, se requiere un protocolo de aplicación que soporte la topología estrella definida anteriormente. Con base en la comparación de protocolos presentada en la Tabla No. 8, MQTT-WIFI (Message Queuing Telemetry Transport) se identifica como la opción óptima debido a su modelo de publicación/suscripción (Pub/Sub) sobre TCP, que se alinea perfectamente con la arquitectura propuesta.

En esta topología, cada microcontrolador publica datos de sensores (pH, CO2, temperatura) en tópicos específicos asociados a un usuario, y el broker central distribuye estos datos a suscriptores como los módulos de análisis y visualización. Esta estructura permite escalar el número de dispositivos y el volumen de datos sin sobrecargar la red, gracias a la ligereza de MQTT y sus niveles de QoS configurables, que aseguran entrega confiable incluso en conexiones inestables.

En contraste, HTTP, aunque compatible con APIs, genera un alto overhead debido a su modelo request-response, resultando ineficiente para transmisiones frecuentes desde dispositivos de bajo consumo. WebSocket, pese a su baja latencia, mantiene conexiones persistentes que incrementan el consumo energético, haciéndolo inadecuado para los microcontroladores. MQTT-Zigbee, optimizado para redes mesh y bajo consumo, requiere hardware específico y un gateway, lo que añade complejidad y limita su integración en entornos .

Tabla 8: Protocolos de Transporte

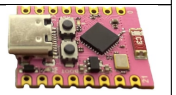
Protocolo	**Arquitectura**	**Transporte**	**Características**	**Ventajas**	**Desventajas**
MQTT-WiFi [61]	Pub/Sub	TCP	Mensajería ligera, basada en broker, en donde se envia un topic al cual uno o varios suscriptores pueden recibir dicho mensaje.	Ligero y con una gran escalabilidad en redes grandes de dispositivos atraves de QoS.	Dependencia de un broker.
HTTP [62]	Request-Response	TCP	Transferencia de hipertexto, solicitudes/respuestas mediante GET-POST.	Ideal para monitoreo esporadico, al ser universal es fácil la integración con APIs, ya que es estandarizado.	Tiene un alto overhead y no es apto para tiempo real.
Web-Socket [63]	Bidirec. Full-Duplex	TCP	Conexión bidireccional y en tiempo real mediante una unica solicitud-respuesta HTTP.	Permite una conexión continua lo que es ideal para tiempo real, bidireccional, baja latencia.	Alto consumo, conexiones persistentes, no apto para bajo consumo.
MQTT-Zigbee [58]	Mesh Networking	IEEE 802.15.4	Enfocada en comunicaciónes locales atraves de una Red de malla lo que permite un muy bajo consumo,	Baja interferencia, alta seguridad de los datos (AES-128) y una gran escalabilidad entre los dispositivos Zigbee	Hardware muy específico y dependencia de un gateway.

5.1.7. Microcontrolador

En base a los sensores , el protocolo de acceso ,el protocolo de transporte seleccionados y los requerimientos funcionales RF03 del proyecto, se consideraron los siguientes microcontroladores mostrados en la Tabla No. 9 para su análisis comparativo, de los cuales el más adecuado para este proyecto es el XIAO-ESP32C6 [64]. Esto se debe a que incorpora un procesador RISC-V de 32 bits con una frecuencia de hasta 160 MHz, lo que proporciona un rendimiento eficiente para manejar múltiples tareas,combinado con soporte para Wi-Fi 6, lo que asegura una comunicación robusta con WiFi-MQTT y de bajo consumo energético durante los ciclos de muestreo adaptativo de los sensores.

A diferencia de otros microcontroladores que a pesar de sus potentes arquitecturas (como Xtensa LX6 Dual-Core o ARM Cortex M0+ Dual-Core), incluyen características excesivas que incrementan innecesariamente el costo y el consumo energético. No obstante, no se descarta la posibilidad de usar el ESP32-DevKitC-32E [65], ya que tambien soporta multiples protocolos de acceso (Wi-Fi 6, BLE 5, Zigbee 3.0, Thread). Sin embargo, cuenta con un mayor consumo energético y un costo más elevado en comparación con el XIAO-ESP32C6.

Tabla 9: Microcontroladores

Modelo	**Procesador / Frecuencia**	**Memoria**	**Conectividad**	**Puertos**	**Voltaje de Operación**	**Precio (MXN)**	**Imagen**
XIAO ESP32C6 [64]	RISC-V 32-bit Single-Core / 160 MHz	ROM 320KB, SRAM 512KB, Flash 4MB	Wi-Fi 6, BLE 5, Zigbee 3.0, Thread	1xI2C, 1xUART, 1xSPI, 1xI2S, 11xGPIO	3.3V (5V VCC)	$100-200	
ESP32-C3 SuperMini [66]	RISC-V 32-bit Single-Core / 160 MHz	ROM 400KB, SRAM 400KB, Flash 4MB	Wi-Fi 4 (802.11n), BLE 5	2xUART, 2xI2C, 1xSPI, 13xGPIO	3.3V (5V VCC)	$100-200	
ESP32-C6-DEVKITC-1-N8 [67]	RISC-V 32-bit Single-Core / 160 MHz	ROM 320KB, SRAM 512KB, Flash 8MB	Wi-Fi 6, BLE 5, Zigbee 3.0, Thread	4xSPI, 3xUART, 2xI2C, 2xI2S, 23xGPIO	3.3V (5V Micro-USB)	$160-250	
ESP32-DevKitC-32E [65]	Xtensa LX6 Dual-Core / 240 MHz	ROM 448KB, SRAM 520KB, Flash 4MB	Wi-Fi 4 (802.11n), BT 4.2	3xSPI, 3xUART, 2xI2C, 2xI2S, 34xGPIO	3.3V (5V Micro-USB)	$160-250	
ESP32-C2 (Módulo) [68]	RISC-V 32-bit Single-Core / 120 MHz	ROM 320KB, SRAM 272KB, Flash 4MB	Wi-Fi 4 (802.11n), BLE 5	2xSPI, 2xUART, 1xI2C, 1xI2S, 14xGPIO	3.3V	$120-250	
Raspberry Pi Pico 2 W [69]	ARM Cortex M0+ Dual-Core / 133 MHz	SRAM 264KB, Flash 4MB	Wi-Fi 4 (802.11n), BT 5.2	2xSPI, 2xI2C, 2xUART, 16xPWM, 26xGPIO	3.3V (5V Micro-USB)	$200-280	

5.1.8. Fuente de alimentación

En base a los requerimientos funcionales RF03 del proyecto, es necesario mantener el microcontrolador y los sensores en funcionamiento continuo durante el proceso de fermentación del kéfir de leche, que puede durar varios días. Por lo tanto en base al siguiente análisis en donde se calcula un aproximado del consumo energético total del Módulo 1, se selecciona una fuente de alimentación adecuada para garantizar un suministro estable y confiable de energía.
En base a la Tabla No. 10, se muestran los consumos energéticos aproximados de cada componente del Módulo 1, tanto en modo activo como en modo de bajo consumo (sleep). Estos valores son estimados y pueden variar según las condiciones de operación específicas y la configuración del sistema.

Tabla 10: Consumos de corriente de los componentes del Módulo 1.

Componente	**Modo Activo (mA)**	**Modo Reposo(mA)**
PH-E201C (sensor pH)	5 (continuo en operación)	5 (continuo)
DS18B20 (sensor temperatura)	1.5 (durante conversión)	0.001 (<1 μA)
SCD30 (sensor CO_2)	19 (durante medición)	0.03 (standby)
XIAO ESP32C6 (microcontrolador)	50–100 (CPU activo); 150–300 (WiFi TX)	0.015 (deep sleep)

La fermetación del kéfir de leche típicamente dura entre 12 y 48 horas, durante este tiempo, los sensores y el microcontrolador estarán en modo activo durante los ciclos de muestreo , y en modo de bajo consumo el resto del tiempo.Para este análisis se toma como referencia fermentaciónes ya hehcas como las que se muestran en la fig. 8 del proyecto 2.1.1. Tomando el caso más extremo de una fermentación de 48 horas (172800 segundos), en la cual en las primeras 33 horas esta el ph en un rango de 6.8 a 5.6 ,de la hora 34 a 41 esta el pH en un rango de 5.5 a 5.0 y de la hora 41 a 48 el pH esta en un rango de 5.0 a 4.2. En base a los anteriores rangos de pH, el módulo 1 debe ajustar la frecuencia de muestreo para capturar adecuadamente los cambios en el pH durante la fermentación segun los RF01.
Para esto calculamos primero la carga en mA · s para cada fase del monitoreo (48 h, 3 fases), según consumos y requerimientos.

- Corrientes (mA): $I_{\text{medición12}} = 54$ (fases 1-2: MCU 30 + pH 5 + CO2 19), $I_{\text{medición3}} = 55{,}5$ (fase 3: +DS18B20 1.5), $I_{\text{TX}} = 250$, $I_{\text{reposo}} = 0{,}01$.
- Tiempos por ciclo (s): $t_{\text{medición}} = 5$, $t_{\text{TX}} = 5$, $t_{\text{activo}} = 10$.
- Períodos (s): Fase 1 = 600, Fase 2 = 300, Fase 3 = 60.
- Duraciones (s): Fase 1 (33 h) = 118800, Fase 2 (8 h) = 28800, Fase 3 (8 h) = 28800.

Para esto definimos las fórmulas a usar en este calculo: 1. Carga por ciclo (mA · s):

$$\text{Carga por ciclo} = (I_{\text{medición}} \times t_{\text{medición}}) + (I_{\text{TX}} \times t_{\text{TX}}) + (I_{\text{reposo}} \times t_{\text{reposo}}) \tag{1}$$

Con $t_{\text{reposo}} = \text{período} - t_{\text{activo}}$. Suma consumos en medición, transmisión y reposo.
2. Número de ciclos:

$$\text{Número de ciclos} = \frac{\text{Duración fase}}{\text{Período ciclo}} \tag{2}$$

Divide duración de la fase por intervalo de ciclo.
3. Carga por fase (mA · s):

$$\text{Carga fase} = \text{Número de ciclos} \times \text{Carga por ciclo} \tag{3}$$

Escala la carga unitaria al total de ciclos.
Calculamos la caga por ciclo de cada fase como se muestra a continuación:
Usamos (1).

- **Fase 1**: $t_{\text{reposo1}} = 600 - 10 = 590$.

$$\begin{aligned}\text{Carga por ciclo1} &= (54 \times 5) + (250 \times 5) + (0{,}01 \times 590) \\ &= 270 + 1250 + 5{,}9 = 1525{,}9\,\text{mA} \cdot \text{s}.\end{aligned}$$

  Medición (pH + CO2), WiFi, reposo prolongado.

- **Fase 2**: $t_{\text{reposo2}} = 300 - 10 = 290$.

$$\begin{aligned}\text{Carga por ciclo2} &= (54 \times 5) + (250 \times 5) + (0{,}01 \times 290) \\ &= 270 + 1250 + 2{,}9 = 1522{,}9\,\text{mA} \cdot \text{s}.\end{aligned}$$

  Igual que fase 1, pero reposo más corto.

- **Fase 3**: $t_{\text{reposo3}} = 60 - 10 = 50$.

$$\begin{aligned}\text{Carga por ciclo3} &= (55{,}5 \times 5) + (250 \times 5) + (0{,}01 \times 50) \\ &= 277{,}5 + 1250 + 0{,}5 = 1528\,\text{mA} \cdot \text{s}.\end{aligned}$$

  Incluye DS18B20, reposo mínimo.

Calculamos el número de ciclos de cada fase como se muestra a continuación:

- Fase 1: $\frac{118800}{600} = 198$.
- Fase 2: $\frac{28800}{300} = 96$.
- Fase 3: $\frac{28800}{60} = 480$.

Y por ultimo obtenemos la carga de cada fase:

- Fase 1: $198 \times 1525{,}9 = 301228{,}2$ mA · s.
- Fase 2: $96 \times 1522{,}9 = 146198{,}4$ mA · s.
- Fase 3: $480 \times 1528 = 733440$ mA · s.

obtenemos que la Fase 1: 301228.2 mA · s, Fase 2: 146198.4 mA · s, Fase 3: 733440 mA · s. Fase 3 en donde domina por su alta frecuencia.

Una vexz obtenidos la carga de cada fase , determinamos la capacidad de la batería en mAh para 48 h de monitoreo, sumando las cargas de las tres fases, convirtiendo a mAh y aplicando un factor de seguridad.

- Cargas por fase (mA · s): Fase 1 = 301228.2, Fase 2 = 146198.4, Fase 3 = 733440.
- Duración total: 48 h = $48 \times 3600 = 172800$ s.
- Factor de seguridad: 1.5 (para pérdidas por eficiencia, envejecimiento, variaciones).

Usamos las siguientes fórmulas : 1. Carga total (mA · s):

$$\text{Carga total} = \text{Carga fase1} + \text{Carga fase2} + \text{Carga fase3} \tag{4}$$

Suma las cargas de cada fase.

2. Capacidad sin seguridad (mAh):

$$\text{Capacidad sin seguridad} = \frac{\text{Carga total}}{3600} \tag{5}$$

Convierte mA · s a mAh (3600 s/hora).

3. Capacidad requerida (mAh):

$$\text{Capacidad requerida} = \text{Capacidad sin seguridad} \times \text{Factor de seguridad} \tag{6}$$

Ajusta por pérdidas.

Obtenemos la carga total sumanando las cargas obtenidas de cada fase , como se muestra a continuación:

$$\begin{aligned}\text{Carga total} &= 301228{,}2 + 146198{,}4 + 733440 \\ &= 1180866{,}6\,\text{mA} \cdot \text{s}.\end{aligned}$$

Al sumar las cargas de fase 1 (33 h, ciclos cada 10 min), fase 2 (8 h, ciclos cada 5 min) y fase 3 (8 h, ciclos cada 1 min).

Procedemos a obtener la carga sin seguridad

$$\text{Capacidad sin seguridad} = \frac{1180866{,}6}{3600} \approx 328{,}02\,\text{mAh}.$$

En donde dividimos por 3600 para convertir a mAh.
Y obtenemos la capacidad requerida

$$\text{Capacidad requerida} = 328{,}02 \times 1{,}5 \approx 492{,}03\,\text{mAh}.$$

Multiplica por 1.5 para cubrir ineficiencias (85 %) y variaciones.

Verificamos el calculo obteniendo la corriente promedio:

$$I_{\text{promedio}} = \frac{1180866{,}6}{172800} \approx 6{,}83\,\text{mA}.$$

Obtenemos que la capacidad alternativa es de $6{,}83 \times 48 \times 1{,}5 \approx 492{,}03$ mAh, confirmando (6).
Este calculo es una aproximación dado que puede variar los tiempos de fermentación, en este caso se tomo como ejermplo una fermentación de 48 horas pero pueden durar mucho menos por lo que gracias a este calculo obtenemos que la capacidad requerida ≈ 492 mAh, lo que nos permite implementar una batería siempre y cuando sea mayor a 500 mAh.
A continuación se muestra en la Tabla No. 11 las bateiras a considerar, de las cuales se selecciono la batería módelo 802040 ya que esta tiene cuenta con 650mAh lo que asegura la alimentación continua durante el proceso de monitoreo, aunque no se descarta el uso de las demás baterias o de otros modelos siempre y cuando cuenten con una capacidad mayor a 500mAh.

Tabla 11: Modelos de Baterías

Modelo de Batería	**Voltaje Nominal (V)**	**Capacidad (mAh)**	**Tipo**	**Precio Est. (USD)**	**Imagen**
802040 [70]	3,7 V	650	Li-Po	$75	
6022480 [71]	3,7 V	600	Li-Po	$100	
303450 [72]	3,7 V	600	Li-Po	$120	

5.2. Análisis del Módulo 2

En esta sección se presenta un análisis,comparacción y selección del lenguaje de programación considerado para llevar a cabo el desarrollo del analisis de datos.Tomando en cuenta los requeriminetos funcionales RF01 y el formato de datos (JSON) previamente seleccionado con el cual va a trabajar el módulo 1 , lo que es de suma importancia que el lenguaje de programación seleccionado tenga una integración nativa para el manejo de este formato.

De la Tabla No. 12, donde se comparan diferentes lenguajes de programación, de los cuales se seleccionó Python, ya que este lenguaje orientado a objetos cuenta con la caracterisitca principal de permitir la herencia de propiedades y métodos en subclases, facilitando la reutilización de código sin duplicación lo que simplfica mucho el código y el almacenamiento para implementarlo.
Además, su polimorfismo soporta procesos de análisis simultáneos sin afectar el rendimiento entre ellos, gracias a un amplio ecosistema con librerías para manejo nativo de JSON y librerias nativas de lógica difusa. Siendo que Python también ofrece prototipado rápido, integración sencilla con servicios, visualización avanzada y soporte para aprendizaje automático. Por ello, presenta la mejor relación entre facilidad de uso, herramientas y comunidad para el Módulo 2. En contraste, C# se limita al ecosistema .NET, complicando su implementación, mientras que R, aunque no se descarta su uso debido a se excelente rendimiento en secuencias de datos, se enfoca en análisis estadístico y academico limitando su flexibilidad hacia el desarrollo web.

Tabla 12: Lenguajes de programación: C#, Python y R

Aspecto	**C#** [73] [74]	**Python** [75] [76]	**R** [77]
Características	Lenguaje de programación visual orientado a objetos, tipado, estático y controlado por eventos mediante el uso de un IDE. Desarrollado e integrado para el ecosistema .NET	Lenguaje multiparadigma ya que soporta una programación orientada a objetos, imperativa y funcional con tipado dinámico y con una sintaxis muy fácil de aprender	Lenguaje especializado en análisis estadístico con un enfoque en manipulación de datos y generación de gráficos.
Rendimiento	Compilado en MSIL y luego optimizado mediante JIT en .NET, de nivel medio.	Interpretado, de alto nivel, lo que implica menor velocidad en tareas intensivas en CPU. Pero permite una gran portabilidad en diferentes entornos de programación.	Interpretado, de alto nivel, optimizado para operaciones estadísticas y matriciales, pero menos eficiente en tareas generales.
Escalabilidad	Compatible con JSON a través de bibliotecas siendo ampliamente eficiente en sistemas distribuidos y escalables como en ASP.NET.	Soporte nativo para JSON con la biblioteca json y escalable de forma nativa con diferentes *frameworks* para aplicaciones web y con bases de datos modernas.	Compatible con JSON mediante paquetes y escalable en entornos estadísticos de *big data*.
Ventaja	Permite una gran integración y soporte multiplataforma robusta con .NET Core al tener un rendimiento optimizado y un tipado estático lo que reduce errores en tiempo de ejecución.	Al tener una curva de aprendizaje suave su sintaxis es muy simple y clara, teniendo un ecosistema extenso y una comunidad activa lo que permite una amplia gama de bibliotecas para ciencia de datos y otras herramientas.	Permite una visualización muy especializada y un análisis estadístico muy avanzado, siendo que tiene un gran soporte en sus bibliotecas especializadas en el análisis de datos respaldadas por una gran comunidad académica.
Desventaja	Menor flexibilidad al tener mucha dependencia del ecosistema .NET Core lo que hace gran dependencia a este ecosistema. Esto también hace que algunas herramientas o bibliotecas puedan tener algún costo, aparte el ecosistema .NET puede ser algo robusto en cuestión de memoria.	Tiende a tener un rendimiento limitado debido a una concurrencia y un tipado dinámico al hacer la interpretación de línea por línea, lo que lo hace más lento en aplicaciones intensivas. Haciendo que tenga dependencia de bibliotecas externas para algunas aplicaciones.	Tiene una flexibilidad muy baja con otros entornos enfocados al desarrollo web, esto debido a su sintaxis única lo que hace que tenga una curva de aprendizaje más pronunciada.

5.3. Análisis del Módulo 3

En esta sección, se lleva a cabo un análisis y comparación entre bases de datos SQL y NoSQL, con base en sus características, propiedades y adoperación a los requerimientos del sistema. Asimismo, se realiza un cálculo inicial del peso de los atributos de las tablas o colecciones, considerando la estructura de datos JSON generada por los microcontroladores a través del protocolo MQTT. Por consiguiente, se justifica la selección del tipo de base de datos más idóneo, garantizando escalabilidad, flexibilidad para cambios en la estructura de datos y eficiencia en la gestión de flujos de datos en tiempo real, en función de las necesidades de asociar usuarios a dispositivos y permitir la incorporación de nuevos nodos sin afectar el sistema existente.

5.3.1. Modelo de base de datos

Basado en la Tabla No. 13, se seleccionó una base de datos NoSQL para la implementación del Módulo 3, dado que satisface los requerimientos no funcionales (RNF03) de escalabilidad horizontal, ya que permite gestionar múltiples nodos (microcontroladores) de manera simultánea y añadir nuevos dispositivos sin comprometer la integridad de los existentes. Por ello, la arquitectura distribuida de NoSQL, junto con su modelo BASE, asegura alta disponibilidad y tolerancia a particiones, lo cual es crucial para procesar flujos de datos MQTT en tiempo real con mínima latencia. Además, su soporte nativo para JSON facilita una integración directa y eficiente con el protocolo MQTT, porque permite adaptarse rápidamente a datos cuya estructura puede variar según la fermentación de cada usuario.
A pesar de que el sistema estará alojado inicialmente en un solo servidor, el diseño flexible de NoSQL posibilita una transición sencilla a entornos distribuidos en el futuro. Sin embargo, las bases de datos SQL, con su esquema fijo y enfoque en escalado vertical, resultan menos adecuadas, dado que su rigidez dificulta la adaptación a cambios dinámicos en los datos y su arquitectura monolítica compromete la escalabilidad en contextos IoT. Por consiguiente, NoSQL se consolida como la opción óptima para garantizar flexibilidad, eficiencia y escalabilidad en este proyecto.

Tabla 13: Modelo de base de datos.

Característica	**SQL (Relacional)** [78]	**NoSQL (No Relacional)** [79]
Modelo de Datos	**Esquema Fijo** (Tablas, Filas y Columnas predefinidas en una estructura).	**Esquema Dinámico/Flexible** (Documentos, Pares Clave-Valor, Grafos, etc.).
Escalabilidad	Escalabilidad principalmente Vertical (mejorar servidor) y horizontal (sharding).	Principalmente permite una escalabilidad horizontal (añadir más servidores o nodos), lo que permite mayor distribución
Transacciones	ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).	BASE (Básicamente Disponible, Estado Suave, Consistencia Eventual).
Arquitectura	Monolítica, centralizada en un único servidor o clúster.	Distribuida, diseñada para clústeres de nodos a lo largo de múltiples servidores.
Ventajas	Basado en el modelo relacional permite la integridad de datos,garantizada, consultas complejas con JOINs eficientes y modelado de relaciones claro.	Permite una alta velocidad y consistencia en inserción/lectura, adaptabilidad a cambios, y rendimiento en grandes volúmenes de datos.
Desventajas	Bastante robustas lo que las vuelve menos flexibles a cambios en el esquema. Teniendo un rendimiento limitado para escrituras y lecturas distribuidas.	Compromete la fiabilidad a cambio de rendimiento, y al tener no tener propiedades ACID tan estrucuturadas no permite relaciones complejas.

5.3.2. Base de datos NoSQL

Basado en la Tabla No. 14, se determinó que MongoDB es la base de datos NoSQL más adecuada para el Módulo 3, dado que cumple con los requerimientos funcionales (RF01) y no funcionales (RNF03). En efecto, su modelo de documentos con soporte nativo para JSON permite integrar directamente los datos enviados por los microcontroladores a través de MQTT, ya que se adapta a estructuras dinámicas que varían según la fermentación de cada usuario. Además, su propiedad CP (consistencia y tolerancia a particiones) asegura fiabilidad en el monitoreo, lo cual es crucial para asociar datos a usuarios específicos, mientras que su tolerancia a fallos garantiza que, a pesar de utilizar un único broker MQTT inicialmente, un fallo en la conexión de un microcontrolador no afecte a otros usuarios. Por otra parte, MongoDB ofrece escalabilidad horizontal, de modo que facilita la incorporación de nuevos dispositivos sin comprometer los existentes. Asimismo, su diseño flexible permite operar en un solo servidor, pero a su vez posibilita una transición fluida a entornos distribuidos en el futuro. En contraste, Cassandra prioriza disponibilidad sobre consistencia, por lo que puede introducir latencias en la sincronización de datos usuario-dispositivo. De igual manera, Redis, aunque rápido, está limitado por su naturaleza en memoria, lo que encarece la persistencia a largo plazo, DynamoDb permite una integración al tener un módelo basado en docuemntos pero se ve limitado ya que solo se puede integrar por medio del eosistema de AWS. Por su parte, InfluxDB, optimizado para series temporales, resulta menos adecuado para relaciones usuario-dispositivo no temporales. En consecuencia, MongoDB se consolida como la mejor opción, puesto que equilibra consistencia, flexibilidad y escalabilidad.

Tabla 14: Bases de datos NoSQL

Base	**Modelo de Datos**	**CAP**	**Características**	**Licencia**	**Propiedades ACID**	**Propiedades BASE**	**Implementación en la Nube o Local**
MongoDB [80]	Documento (nativo BSON/JSON)	CP	Permite esquemas flexibles, asignando cada documento a un objeto.	Código abierto SSPL/Apache	Media:Permite transacciones ACID entre documentos pero de forma limitada.	Fuerte (Consistencia Eventual en clústeres distribuidos).	Nube (MongoDB Atlas, AWS, GCP, Azure) y Local
Cassandra [81]	Columna Amplia (Wide-Column)	AP	Permite una escalabilidad lineal al optimiza las consultas por medio de CQL(Lenguaje de consulta de Cassandra) para grandes cantidades de datos.	Apache	Bajo (Garantías mínimas de consistencia y aislamiento).	Fuerte (Alta disponibilidad y particionamiento automático).	Nube (AWS, GCP, Azure) y Local
Redis [82]	Clave-Valor (Key-Value)	CP	Almacenamiento en memoria (*in-memory*), uso primario como caché y broker de mensajes.	BSD	Parcial (Transacciones atómicas básicas en comandos únicos).	Fuerte (Alta velocidad y disponibilidad por naturaleza *in-memory*).	Nube (ElastiCache, Memorystore) y Local
InfluxDB [83]	Series de Tiempo (TSDB)	AP	Optimizado para ingesta masiva de datos indexados por secuencias de tiempo.	MIT y Comercial	Bajo (Prioriza la velocidad de ingesta sobre la atomicidad).	Fuerte (Alta disponibilidad, optimizado para particionamiento temporal).	Nube (InfluxDB Cloud) y Local
DynamoDB [84]	Clave-Valor y Documento	AP	Escalable sin servidor, optimizado para lecturas/escrituras rápidas de datos IoT.	Propietaria (AWS)	Media: Soporta transacciones ACID limitadas para operaciones específicas.	Fuerte (Alta disponibilidad y particionamiento automático).	Nube (AWS)

5.3.3. Estimación del volumen de la base de datos

En esta sección, se describen y especifican, mediante las tablas correspondientes, los volúmenes estimados de datos generados [85] por los microcontroladores a través del protocolo MQTT, considerando la estructura de documentos JSON almacenados en la base de datos NoSQL seleccionada. Para ello, se calcula el peso de los atributos de cada colección, en función de la frecuencia de los mensajes, el número de dispositivos y las variaciones en los datos de fermentación por usuario. Descritors en las siguientes tablas:

Tabla 15: Atributos del módulo 1

Parámetro	Descripción	Tipo de dato	Peso en by-tes
Id del nodo	Identificador de cada microcontrolador	Tinyint	1
Time-set	Tiempo de registro del dato del sensor	Timestamp	4
pH	Dato del sensor de pH	Float	4
CO_2	Dato del sensor de CO_2	Float	4
Temperatura	Dato del sensor de Temperatura	Flaot	4
bandera de alarma	Dato que indica la alarma correspondientes	varchar(20)	longitud de la cadena +1

En base a la tabla Tabla No. 15, se describen los datos proporcionados por el módulo 1, dichos datos son pertenencientes a una fermentación, dado que estas pueden variar en el tiempo. Se toma el caso más extremo de una fermentación de 48 horas (172800 segundos), en la cual en las primeras 33(118800 segundos) horas esta el ph en un rango de 6.8 a 5.6 ,de la hora 34 a 41 esta el pH en un rango de 5.5 a 5.0 y de la hora 41 a 48 el pH esta en un rango de 5.0 a 4.2.Siguiendo los RF01 en los que especifican la frecuencia en que los sensores adquieren los datos , se puede obtener una estimación de la peso de estos atributos descritos a continuación:

1. **Rango 1 (primeras 33 horas)**: En este intervalo, los datos de pH y CO_2 se registran cada 10 minutos (600 segundos), generando, según la operación 7, un peso de 14 bytes por registro, dado que no se incluye la temperatura. Mediante la operación 8, se determinan 198 registros, lo que resulta en 2772 bytes conforme a la operación 9. Por otra parte, la temperatura se envía de forma independiente cada 2.5 minutos (150 segundos), con un peso de 10 bytes por registro, según la operación 10. Así, la operación 11 indica 792 registros, equivalentes a 7920 bytes, conforme a la operación 12. En consecuencia, el total de bytes para este rango es de 10692 bytes, calculado mediante la operación 13.

$$1 + 4 + 4 + 4 + 1 = 14bytes \tag{7}$$

$$118800 \div 600 = 198registros \tag{8}$$

$$198 \times 14 = 2772bytes \tag{9}$$

$$1 + 4 + 4 + 1 = 10bytes \tag{10}$$

$$118800 \div 150 = 792registros \tag{11}$$

$$792 \times 10 = 7920bytes \tag{12}$$

$$2772 + 7920 = 10692bytes \tag{13}$$

2. **Rango 2 (horas 34-41)**: En este intervalo, los datos de pH y CO_2 se registran cada 5 minutos (300 segundos), generando, según la operación 14, un peso de 14 bytes por registro, dado que no se incluye la temperatura. Mediante la operación 15, se determinan 96 registros, lo que resulta en 1344 bytes conforme a la operación 16. Por otra parte, la temperatura se envía de forma independiente cada 2.5 minutos (150 segundos), con un peso de 10 bytes por registro, según la operación 17. Así, la operación 18 indica 192 registros, equivalentes a 1920 bytes, conforme a la operación 19. En consecuencia, el total de bytes para este rango es de 3264 bytes, calculado mediante la operación 20.

$$1 + 4 + 4 + 4 + 1 = 14bytes \tag{14}$$

$$28800 \div 300 = 96registros \tag{15}$$

$$96 \times 14 = 1344bytes \tag{16}$$

$$1 + 4 + 4 + 1 = 10bytes \tag{17}$$

$$28800 \div 150 = 192registros \tag{18}$$

$$192 \times 10 = 1920bytes \tag{19}$$

$$1344 + 1920 = 3264bytes \tag{20}$$

3. **Rango 3 (horas 41-48)**: En este intervalo, los datos de pH, CO_2 y temperatura se registran cada 1 minuto (60 segundos), generando, según la operación 21, un peso de 18 bytes por registro, dado que se incluyen todos los atributos. Mediante la operación 22, se determinan 420 registros, lo que resulta en 7560 bytes conforme a la operación 23. Puesto que los envíos de temperatura están incluidos en estos registros, no se generan envíos independientes adicionales. En consecuencia, el total de bytes para este rango es de 7560 bytes, calculado mediante la operación 23.

$$1 + 4 + 4 + 4 + 4 + 1 = 18bytes \tag{21}$$

$$25200 \div 60 = 420registros \tag{22}$$

$$420 \times 18 = 7560bytes \tag{23}$$

Por consecuencia, al sumar los bytes generados en los tres rangos de la fermentación de 48 horas, se obtiene un total de 21516 bytes. Esto se calcula, según la operación 24, combinando los 10692 bytes del Rango 1, los 3264 bytes del Rango 2 y los 7560 bytes del Rango 3. Por consiguiente, esta estimación refleja el volumen de datos generado por los microcontroladores mediante MQTT, considerando este el caso más extremo, ya que este dato puede variar segun cada fermentación.

$$10692 + 3264 + 7560 = 21516bytes \tag{24}$$

Tabla 16: Atributos del Usuario

Atributo	**Descripción**	**Tipo de Dato**	**Peso en Bytes**
Categoría del usuario	Distingue si un usuario tiene la categoría de productor raíz o productor inicial	Tinyint	1
Id del usuario	Identificador único de cada usuario	varchar(20)	longitud de la cadena +1
Nombre	Nombre del usuario	varchar(20)	longitud de la cadena +1
Primer Apellido	Primer Apellido del usuario	varchar(20)	longitud de la cadena +1
Segundo Apellido	Segundo Apellido del usuario	varchar(20)	longitud de la cadena +1
Género	Género del usuario	varchar(15)	longitud de la cadena +1
Fecha de nacimiento	Fecha de nacimiento	DATE	3
Correo	Correo del usuario	varchar(20)	longitud de la cadena +1

Para la tabla 16, que describe los atributos de la entidad Usuario, se estima el peso total de un registro en la base de datos MongoDB, por medio de la operación 25. Se obtiene un total de 71 bytes por registro. Por consiguiente, esta estimación permite dimensionar el almacenamiento necesario para los datos de usuarios en el sistema.

$$1 + 11 + 11 + 11 + 11 + 7 + 3 + 16 = 71bytes \tag{25}$$

Para la tabla 17, que describe los atributos del Módulo 2 relacionados con los datos de fermentación, se estima el peso total de un registro en la base de datos MongoDB, en base a la operación 26. Se obtiene un total de 27 bytes por registro.

$$11 + 9 + 7 = 27bytes \tag{26}$$

Tabla 17: Atributos del módulo 2

Atributo	**Descripción**	**Tipo de Dato**	**Peso en Bytes**
Id del usuario	Identificador único de cada usuario al que pertenece la fermentación	varchar(20)	longitud de la cadena +1
Estado	Estado de la fermentación , si fue exitosa o no	varchar(20)	longitud de la cadena +1
Nivel	Nivel de la fermentación si fue alta, medio o baja	varchar(20)	longitud de la cadena +1

En base a los datos generados por las tablas 15, 16 y 17. La suma de los bytes del Módulo 1 (21516 bytes), Usuarios (71 bytes) y Módulo 2 (27 bytes) resulta en un total de 21614 bytes por usuario, conforme a la operación 27. Por consiguiente, este valor representa el tamaño del documento JSON que agrupa los datos de sensores, información del usuario y estado de la fermentación en la base de datos MongoDB. Dado que MongoDB permite documentos de hasta 16 MB (16777216 bytes), este total de aproximadamente 21 KB es totalmente permitido, lo que garantiza una gestión eficiente y escalable de los datos, incluso con múltiples usuarios y fermentaciones.

$$21516 + 71 + 27 = 21614 bytes \tag{27}$$

Tabla 18: Atributos del administrador

Atributo	**Descripción**	**Tipo de Dato**	**Peso en Bytes**
Id del usuario	Identificador único de cada usuario	varchar(20)	longitud de la cadena +1
Nombre	Nombre del usuario	varchar(20)	longitud de la cadena +1
Primer Apellido	Primer Apellido del usuario	varchar(20)	longitud de la cadena +1
Segundo Apellido	Segundo Apellido del usuario	varchar(20)	longitud de la cadena +1
Género	Género del usuario	varchar(15)	longitud de la cadena +1
Fecha de nacimiento	Fecha de nacimiento	DATE	3
Correo	Correo del usuario	varchar(20)	longitud de la cadena +1

Para la tabla 18, que describe los atributos de la entidad Administrador, con un peso calculado en la operación 28. se obtiene un total de 70 bytes por registro. Por consiguiente, esta estimación permite dimensionar el almacenamiento necesario para los datos de administradores en el sistema.

$$10 + 10 + 10 + 10 + 6 + 3 + 15 = 70 bytes \tag{28}$$

5.4. Análisis del Módulo 4

Basado en la Tabla No. 19, se selecciona React para el Módulo 4, dado que su enfoque en la construcción de interfaces de usuario interactivas mediante componentes y JSX permite una integración eficiente con los datos JSON enviados por el módulo 1 a través de MQTT. Su alto rendimiento, gracias al DOM Virtual, asegura actualizaciones rápidas de la UI para visualizar datos dinámicos de fermentación en tiempo real, mientras que su amplio ecosistema de librerías facilita la incorporación de herramientas para enrutamiento y gestión de estado, adaptándose a los requerimientos funcionales RF01 y no funcionales RNF04. Aunque requiere librerías adicionales, lo que incrementa el tamaño, su comunidad activa y extensa documentación compensan esta limitación, garantizando compatibilidad con el backend Python y MongoDB mediante APIs ligeras. En contraste, Angular es más rígido por su estructura MVC y dependencia de TypeScript, mientras que no se descarta el uso Vue.js, ya que es muy ligero, tiene un ecosistema más reducido que limita las soluciones prediseñadas. Por consiguiente, React es la opción óptima para desarrollar una interfaz escalable y reactiva en este sistema IoT.

Tabla 19: Comparación de Frameworks.

Tipo	**Nombre**	**Enfoque Principal**	**Ventaja**	**Desventaja**
Librería	**React** [86]	Construcción de Interfaces de Usuario (UI) interactivas basadas en componentes y empleando una sintaxis JSX(tipado dinámico).	Permite un alto rendimiento gracias al uso de **DOM Virtual**. Posse un amplio ecosistema de librerias gracias a su gran comunidad.	No es un framework completo; requiere librerías adicionales para enrutamiento y estado, lo que afecta en el espacio o tamaño necesario para implementar react.
Framework	**Angular** [87]	Es un marco de diseño Modelo-Vista-Controlador (MVC) y desarrollo de código abierto para aplicaciones a gran escala de una Sola Página (SPA).	Permite soluciónes completas (*full-featured*)al incorporar de forma nativa TypeScript(tipado estático) y una arquitectura modular basado en componentes.	Al ser un framework es algo robusto y rígida su implementación e integración,aparte depende del uso obligatorio de TypeScript.
Framework	**Vue.js** [88]	Implementa un modelo de programación declarativo basado en componentes mediante su modelo modular y reutilizable con HTML estándar, CSS y JavaScript.	Permite un diseño progresivo, ya que se puede integrar solo la librería central, siendo muy ligero y flexible pero con herramientas limitadas.	La menor escala de su ecosistema implica que hay menos soluciones prediseñadas (como plugins o props complejos) disponibles en comparación con React o Angular. Esto puede requerir el desarrollo manual de funcionalidades avanzadas, aumentando la complejidad del código.

Tabla 20: Librerias de graficos para React

Librería	Tipo de Gráficos	Flexibilidad y Personalización	Rendimiento	Comentarios / Uso Principal
Recharts [89]	Componentes React / SVG	Moderada; ideal para gráficos estándar con personalización básica.	Alto	Solución declarativa optimizada para React, adecuada para visualizaciones rápidas de datos en tiempo real, como métricas de fermentación.
React-Chartjs-2 [90]	Componentes React / Canvas	Media; personalización avanzada limitada por la API de Chart.js.	Medio-alto	Wrapper de Chart.js, eficiente para gráficos estadísticos simples y de alta frecuencia, como series temporales de sensores.
Nivo [91]	Componentes React / SVG, Canvas, HTML	Alta; ofrece modularidad y herramientas interactivas para gráficos complejos.	Alto	API moderna para visualizaciones profesionales, ideal para dashboards dinámicos con datos de IoT.
Victory [92]	Componentes React / SVG	Alta; API consistente y flexible para gráficos reutilizables y personalizados.	Medio	Enfocada en dashboards interactivos, adecuada para monitoreo de múltiples parámetros de fermentación.

5.5. Framework

Basado en la tabla comparativa, se selecciona Flask para el Módulo 4, ya que su naturaleza como microframework proporciona flexibilidad y extensibilidad, permitiendo una integración directa con Python y JSON para gestionar datos de fermentación enviados por ESP32 vía MQTT, almacenados en MongoDB. Su estructura MVC adaptable optimiza el desarrollo de una aplicación pequeña en un solo servidor, facilitando la conexión con React mediante APIs ligeras. Aunque requiere configuración manual, esto es aceptable para un sistema compacto. En contraste, Django es rígido y orientado a SQL, mientras que FastAPI, enfocado en APIs complejas, excede las necesidades actuales. Por ello, Flask es la mejor opción por su simplicidad y escalabilidad.

Tabla 21: Comparación de Frameworks para el módulo 4.

Framework	**Característica (Modularidad y Extensibilidad)**	**Estructura**	**Ventaja**	**Desventaja**
Django [93]	Posee una **alta modularidad** gracias a su sistema de .ªplicaciones"(*apps*), sin embargo, su extensibilidad es más enfocada a bases de datos SQL.	Sigue un patrón **MTV** (*Model-Template-View*) rígido que dicta la organización del código; por lo tanto, es un framework çon opinión".	Ofrece una **solución completa** (*full-stack*) con ORM, *admin panel*, enrutamiento y seguridad integrados, acelerando el desarrollo inicial.	Su **rigidez estructural** y su diseño intrínseco para SQL complican la integración completa y nativa de MongoDB.
Flask [94]	Al ser un microframework permit una **excelente modularidad** y **máxima extensibilidad**, lo que facilita el desarrollador al elegir e integrar librerías externas para cada funcionalidad.	Adopta un patrón **MVC** (*Model-View-Controller*) flexible o similar, donde el desarrollador define la organización.	Es una framework nativo de Python y JSON, permitiendo una integración directa y de bajo acoplamiento con bases de datos NoSQL basadas en documentos.	Requiere que el desarrollador **integre y configure** manualmente cada componente o librería externa, incrementando la complejidad de la configuración inicial.
FastAPI [95]	Ofrece **alta modularidad** al separar la lógica en archivos y rutas (*routers*). Su extensibilidad es simple, gracias al uso de tipado estándar de Python.	Se basa en el concepto de **funciones de ruta** y *Pydantic models* para la validación de datos. Su estructura está diseñada para la construcción de APIs.	Integración automática y fluida con Python y JSON, gracias al tipado de datos lo que complementa con la generación automática de documentación interactiva(*OpenAPI/ Swagger*).	Está **estrictamente enfocado en APIs**, por lo tanto, no incluye módulos para *templates* o frontend, lo cual debe ser manejado externamente.

5.6. Servidor

En esta sección se presenta la selección del broker MQTT y del servidor para el Módulo 4 del proyecto, que integra Python con Flask, MongoDB, React y los microcontroladores para gestionar datos de fermentación en tiempo real. La elección se fundamenta en los requerimientos de escalabilidad horizontal, flexibilidad para datos dinámicos JSON, baja latencia en la comunicación MQTT y optimización de costos para un despliegue inicial en un sistema IoT compacto. A ontinuación se evalúan opciones de brokers MQTT y servicios en la nube, considerando su compatibilidad con la arquitectura del sistema, facilidad de integración con Flask y React, y capacidad para soportar análisis y visualización de datos en tiempo real. Por consiguiente, se seleccionan las soluciones que mejor equilibran rendimiento, simplicidad y viabilidad económica.

5.6.1. Servidor en la nube o local

De la tabla Tabla No. 22 se selecciona un servidor en la nube como la mejor opción ya que sadisface a los requerimientos funcionales RF01 y no funcionales RNF04, dado su principal caracteristica de la escalabilidad al poder adaptar recursos según la demanda y permitir una gran flexibilidad ya que permite el acceso desde cualquier lugar, así como capacidades de respaldo y recuperación ante desastres. En contraste, un servidor local, aunque sin costos recurrentes, limita la escalabilidad, requiere mantenimiento manual y es vulnerable a fallos, lo que afecta la continuidad del monitoreo. Por consiguiente, un servidor en la nube ofrece mayor robustez y flexibilidad para este sistema.

Tabla 22: Comparación de Servidor en la nube o local.

Categoría	**Servidor Local** [96]	**Servidor en la Nube** [97]
Costo	Sin costos recurrentes, solo inversión inicial en hardware.	Costos mensuales bajo demanda ($10-40/mes para proyectos pequeños).
Escalabilidad	Limitada, requiere hardware adicional y configuración manual.	Alta, con autoescalado y balanceo de carga automáticos.
Mantenimiento	Responsabilidad del equipo, incluye actualizaciones y backups.	Gestionado por el proveedor, minimizando carga operativa.
Disponibilidad	Vulnerable a fallos de hardware o energía.	Alta disponibilidad (99.9 %+ SLA) con redundancia.
Integración	Configuración manual para MQTT, Flask, MongoDB, React.	Integración nativa con servicios como MongoDB Atlas y APIs.
Latencia	Baja en redes locales, ideal para pruebas iniciales.	Depende de internet, optimizable con regiones cercanas.

5.6.2. MQTT Servidor

Basado en la tabla comparativa Tabla No. 23, se selecciona **EMQX Cloud** para el Módulo 1, dado que su escalabilidad horizontal, soportada por clustering distribuido, permite conexiones simultáneas, alineándose con el requerimiento RNF01 de añadir dispositivos sin afectar los existentes. Su soporte para MQTT 5.0 y análisis en tiempo real optimiza el procesamiento de datos de fermentación enviados por los microcontroladores, mientras que su integración nativa con Python mediante `paho-mqtt` y JSON asegura un manejo eficiente en Flask y MongoDB. Además, su alta disponibilidad , seguridad avanzada con TLS/ACLS y despliegue en regiones de AWS, GCP o Azure garantizan fiabilidad y baja latencia global para visualizaciones en React. Aunque implica costos, su modelo “pay as you go” es asequible para proyectos pequeños, y su documentación extensa facilita la transición desde entornos locales. En contraste, HiveMQ Cloud presenta costos más altos (~$1,000/mes), NanoMQ y VerneMQ requieren mayor configuración técnica, y AWS IoT Core introduce dependencia de su ecosistema. Aunque Eclipse Mosquitto es ligero y gratuito, su escalabilidad limitada y dependencia de hardware lo hacen menos viable para la nube. Por consiguiente, EMQX Cloud equilibra rendimiento, simplicidad y viabilidad económica para este sistema IoT en la nube.

Tabla 23: Comparación de servicios MQTT.

Broker MQTT	Descripción	Ventaja	Desventaja	Costos
Eclipse Mosquitto [98]	Broker ligero de código abierto, soporta QoS 0-2, TLS y ACLs, ideal para despliegues locales.	Ligero y fácil de configurar, ideal para proyectos pequeños con baja latencia.	Escalabilidad limitada ya que depende mucho de los recursos del sistema.	Gratuito (licencia EPL/EDL).
EMQX [99]	Broker escalable con soporte MQTT 5.0, clustering distribuido y análisis en tiempo real.	Alta escalabilidad horizontal, soporta millones de conexiones con clustering.	Configuración compleja para entornos pequeños.	Open Source: Gratuito; Enterprise: Desde ~$0.10/conexión/mes (~$100/mes plan básico).
VerneMQ [100]	Broker basado en Erlang, enfocado en escalabilidad masiva y autenticación avanzada.	Escalabilidad masiva con clustering basado en Erlang, ideal para IoT.	Requiere experiencia técnica para optimización y mantenimiento.	Gratuito; soporte enterprise ~$5,000/año.
HiveMQ [101]	Broker empresarial con soporte MQTT 5.0, seguridad RBAC y compatibilidad con Kafka.	Seguridad avanzada y soporte MQTT 5.0, optimizado para entornos enterprise.	Costo elevado para despliegues comerciales.	Community: Gratuito; Enterprise: Desde ~$1,000/mes.
NanoMQ [102]	Broker ultraligero para edge computing, optimizado para IoT con MQTT 5.0.	Ligero y eficiente para edge computing, bajo consumo de recursos.	Menor ecosistema y soporte comunitario limitado.	Gratuito.
AWS IoT Core [103]	Servicio en la nube de AWS para IoT, soporta MQTT 5.0, integración con AWS Lambda y reglas para datos.	Alta escalabilidad y seguridad, integración nativa con ecosistema AWS.	Dependencia de AWS y costos variables según uso.	Desde $0.08/100,000 mensajes $5-20/mes para proyectos pequeños.

5.6.3. Servidor en la nube

Basado en los requerimientos funiconales RF01 y no funcionales RNF04, que involucran el despliegue de una página web con React y Flask para análisis de datos en Python, se selecciona AWS como el servidor en la nube más adecuado de la tabla Tabla No. 24, dado que su ecosistema amplio permite una integración fluida de EC2 para alojar Flask y procesamiento de datos, S3 para hosting estático de React jutno con Lambda. Además, ofrece escalabilidad horizontal automática mediante Auto Scaling y compatibilidad directa con MongoDB Atlas, asegurando baja latencia para flujos MQTT y visualizaciones en tiempo real, mientras que su free tier minimiza costos iniciales. En contraste, Google cloud computing, aunque competitivo en precios, tiene un ecosistema menos intuitivo para integraciones híbridas, y Azure, pese a su soporte para Microsoft, presenta costos más elevados y complejidad en configuraciones para Python. Por consiguiente, AWS equilibra eficiencia, escalabilidad y simplicidad para este sistema IoT.

Tabla 24: Comparación de Servidores en la nube.

Categoría [104]	**AWS**	**Google cloud**	**Azure**
Modelo	IaaS, PaaS, SaaS	IaaS, PaaS, SaaS	BaaS (Backend as a Service)
Herramientas	EC2, S3, Lambda, RDS, MongoDB Atlas	App Engine, Cloud Storage, Cloud Run, Firestore	App Service, Static Web Apps, Blob Storage, Cosmos DB
Seguridad	IAM, KMS, WAF, VPC	IAM, VPC Service Controls, Shield	Azure AD, Security Center, Sentinel
Ventajas	Alta escalabilidad, ecosistema amplio, free tier robusto	Escalabilidad flexible, precios competitivos, integración AI	Integración con Microsoft, soporte híbrido robusto
Desventajas	Complejidad en configuración, costos variables altos	Menor ecosistema, curva de aprendizaje técnica	Costos elevados, menos intuitivo para startups
Precio (mensual)	Bajo demanda, desde $15-30 (EC2 t3.micro $7, S3 $1, Atlas $9)	Bajo demanda , desde $10-25 (App Engine free tier, Storage $0.5/GB, Firestore $1)	Bajo demanda, desde $20-40 (App Service $13, Static free, Cosmos $24)

6. Capítulo: Diseño

En este capítulo se presenta el diseño detallado de los módulos que constituyen el sistema para el monitoreo de la fermentación en tiempo real. Se abordan la arquitectura general, los diagramas y la topología de comunicación entre dispositivos y servicios en la nube. Se detallan el diseño de cada componente previamente seleccionado en el capítulo 5 ,como Python para el análisis de datos, MongoDB para el almacenamiento de documentos JSON, React y React-Chartjs-2 para la visualización en tiempo real, microcontroladores XIAO ESP32 para adquisición de datos y EMQX Cloud como broker MQTT— así como la propuesta de despliegue en AWS para asegurar escalabilidad y disponibilidad.

El diseño cubre los Módulos 1 a 4 jutno con el diseño del broker MQTT y el servidor, en donde se especifica la estructura de la base de datos, los formatos de mensajes MQTT, las interfaces de usuario y las consideraciones de seguridad y rendimiento.Con el fin de cumplir con los requisitos funcionales y los no funcionales relativos a escalabilidad, flexibilidad ante cambios en los esquemas de datos y baja latencia en la comunicación. A lo largo del capítulo se incluyen diagramas lógicos, esquemas de colecciones y ejemplos de payloads JSON que facilitarán la implementación y validación.

6.1. Diseño Módulo 1

En esta sección se describen e ilustra los diagramas necesarios para llevar a cabo el funcionamiento de dicho módulo, así como la descripción detallada de cada uno de ellos, proporcionando la secuencia de operación del módulo y el diseño del circuito integrando todos los componentes previamente seleccionados en la sección 5.1, aparte se detalla un módelo 3D de los frascos integrando los sensores y el microcontrolor.

6.1.1. Diagrama Módulo 1

Como primera parte se define el diagrama del módulo 1, tal como se aprecia en la fig. 26. En dicha figura se ilustra el flujo qeu seguira el microcontrolador, desde la verificación de la conexión MQTT hasta el envío de datos y la gestión de alarmas, por lo que cada decisión y acción se traduce en una cadena JSON específica que viaja al broker EMQX Cloud. A continuación se describen las tres alarmas críticas, junto con la cadena exacta que publica el microcontrolador al broker:

1. **Alarma 1:** Se activa cuando cualquier sensor falla ya sea en modo monitoreo o prueba, generando la cadena siguiente cadena "Fallo sensor pH/CO2/temperaturaesté".

2. **Alarma 2:** Se activa cuando el microcontrolador esta en modo monitoreo y detectan que los datos iniciales estan fuera de rango, generando la siguiente cadena Çondiciones iniciales fuera de rango-"pH-"Temperatura"

3. **Alarma 3:** Se activa cuando el microcontrolador en modo monitoreo detecta que el pH supero el limite de 3.9, generando una cadena "pH fuera de rango-"pH"

4. **Alarma 4:** Se activa cuando el usuario desea terminar el proceso de monitoreo, generando una cadena "Fin de la fermentación"

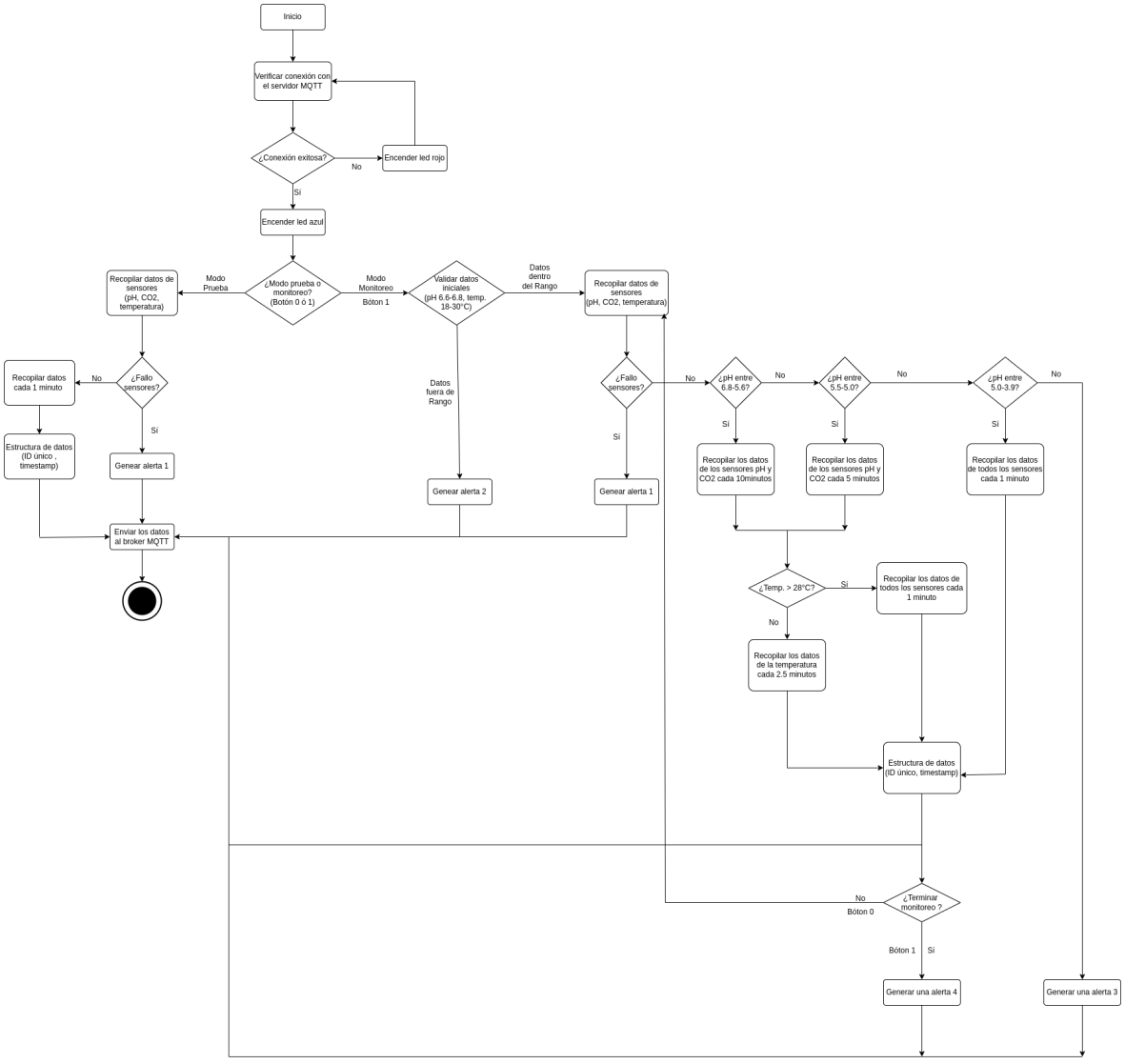


Figura 26: Diagrama del módulo 1

6.2. Diseño Módulo 3

En esta sección se describen e ilustra los diagramas necesarios para llevar a cabo el funcionamiento de dicho módulo, así como la descripción detallada de cada uno de ellos, proporcionando la secuencia de operación del módulo y el diseño del circuito integrando todos los componentes previamente seleccionados en la sección 5.1, aparte se detalla un módelo 3D de los frascos integrando los sensores y el microcontrolor.

6.2.1. Modelo entidad relación

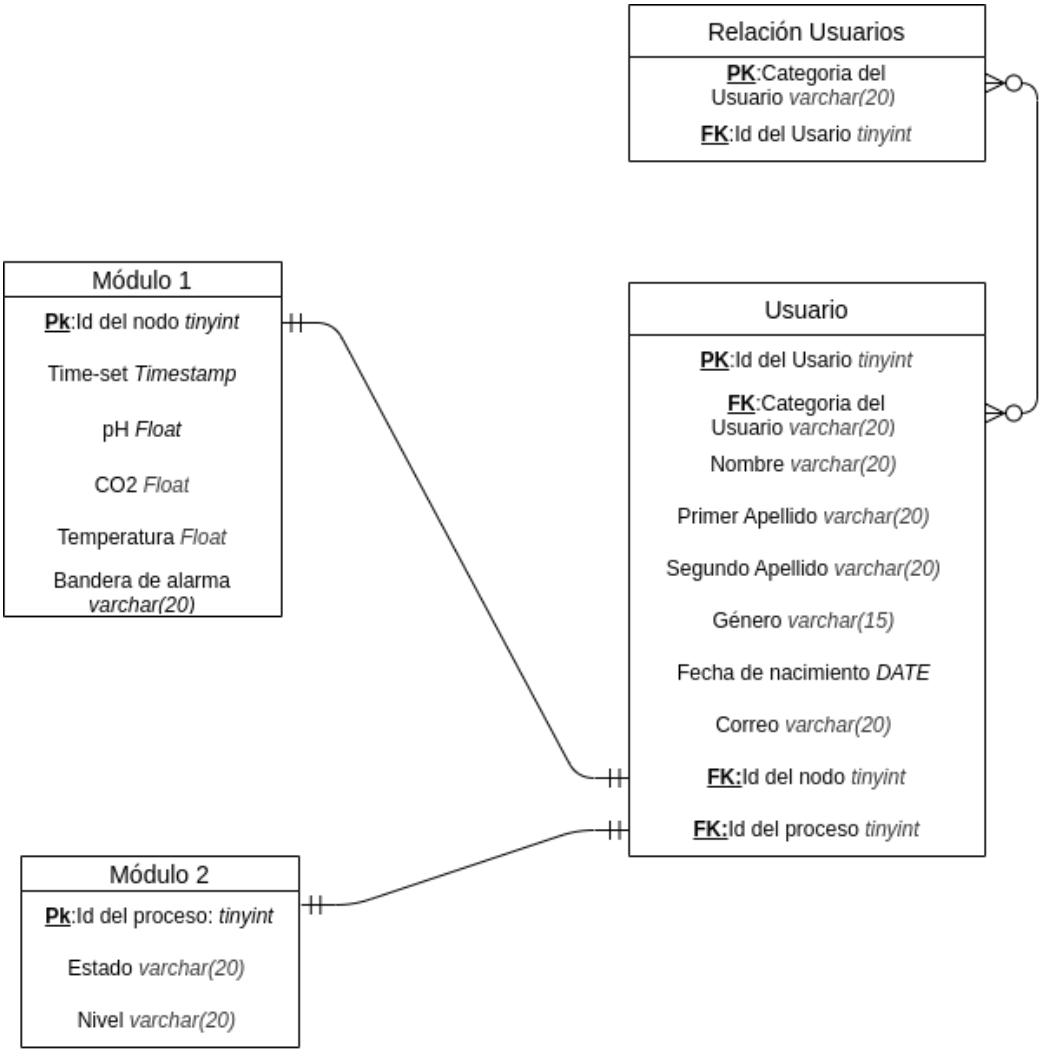


Figura 27: Modelo Relación.

Referencias

[1] D. S. Kumar Behera, E. Kayitesi, S. Kumar Panda, and A. F. Mulaba-Bafubiandi, "Kefir and Koumiss Origin, Health Benefits and Current Status of Knowledge," pp. 401–421, Mar. 2017.

[2] d. l. M. Luciana Gabriela;, A. L. KAIMEN, F. LÓPEZ, L. I. MORENO, C. ALFAGEME, and C. DROLAS, "DESCRIPCIÓN Y ANÁLISIS DE LAS TÉCNICAS DE PRODUCCIÓN DE KÉFIR," *Universidad de Buenos Aires, Facultad de Medicina, Escuela de Nutrición, Buenos Aires, Argentina*, 2019.

[3] J. H. Camacho Vera, F. Cervantes Escoto, A. Cesín Vargas, and M. I. Palacios Rangel, "Los alimentos artesanales y la modernidad alimentaria," *Centro de Investigación en Alimentación y Desarrollo, A.C.*, vol. 29, no. 53, 2019.

[4] L. César Ojeda, Á.-R. Gonzalo D., U. Carmen Julia Figuered, L. A. Islas, P. Lappe-Oliveras, G. Paul Nabhan, I. Torres-García, M. Vallejo, and A. Casas, "Traditional Fermented Beverages of Mexico: A Biocultural Unseen Foodscape," 2021.

[5] A. K. Mauro Cadme, "Biotecnología del kéfir: Una revisión sobre las aplicaciones y sus perspecti- vas futuras," *Universidad de las Fuerzas Armadas ESPE*, 2021.

[6] Á. L. López Ruiz, Á. Pedregosa Cabrero, F. d. A. Ruiz Morales2, L. P. Ureña Cámara, and J. C. Aguilar Jurado, "Recomendaciones en la Elaboración de Kéfir," *Consejería de Agricultura, Pesca, Agua y Desarrollo Rural. Instituto de Investigación y Formación Agraria y Pesquera*, pp. 1–12, 2023.

[7] A. Rueda Manzano, "Elaboración de Kéfir en México," Ph.D. dissertation, Universidad Autónoma Metropolitana, México,D.F., 1980.

[8] "DOF - Diario Oficial de la Federación," https://www.dof.gob.mx/nota_detalle.php?codigo=5254842&fecha=03/05/2012#gsc.tab=0.

[9] F. Fitrianingsih, H. Hafid, A. B. Kimestri, A. Sulfitrana, and R. D. S. Toba, "The Effect of Different Kefir Grain Starter Concentration on pH, Alcohol Percentage, and Organoleptic Properties of Ultra High Temperature (UHT) Milk Kefir:," in *International Conference on Improving Tropical Animal Production for Food Security (ITAPS 2021)*, Kendari, Indonesia, 2022.

[10] F. Arévalo Ortiz and G. Arias Arroyo, "Estudio comparativo del efecto de las temperaturas constante y variable ambiente en la producción de biomas proveniente de la microbiota del grano de kéfir," *Facultad de Farmacia y Bioquimica UNMSM*, 2007.

[11] J. Araujo-Osorio, "EVALUACIÓN DEL pH Y LA ACIDEZ EN KEFIR CON LECHES DE DIFERENTE CONTENIDO DE GRASA."

[12] C. Cuevas Manilla, "Un estudio sobre kéfir(Leche de búlgaros) como producto comercializable," *Universidad Autónoma Chapingo*, 1997.

[13] "DOF - Diario Oficial de la Federación," https://www.dof.gob.mx/nota_detalle.php?codigo=5133449&fecha=01/03/2010#gsc.tab=0.

[14] D. F. Paredes, A. Gonzalo, M. Fuentes Mora, and R. Sanchez, "Sistema Embebido para el seguimiento y control en línea de un fermentador aplicado a la conversión de azúcares en etanol," *INGAR/CONICET-UTN, Instituto de Desarrollo y Diseño.*

[15] H. N. Akgül, F. Y. Akgül, and T. Doğan, "Bulanık Mantık ile Kefir Üretiminin Modellenmesi," *Turkish Journal of Agriculture - Food Science and Technology*, vol. 2, no. 6, p. 251, Jun. 2014.

[16] D. F. Al Riza, R. Damayanti, and Y. Hendrawan, "RANCANG BANGUN FERMENTOR YOGURT DENGAN SISTEM KONTROL LOGIKA FUZZY MENGGUNAKAN MIKROKONTROLER ATMEGA32 (Yogurt Fermenter Design with Fuzzy Logic Control System Using Microcontroller ATMega32)," *Jurnal Agritech*, vol. 34, no. 04, p. 456, Feb. 2015.

[17] E. S. Hermosillo Díaz, "SISTEMA DE APRENDIZAJE AUTOMÁTICO PARA LA PREDICCIÓN DEL AVANCE DE FERMENTACIÓN EN CERVECERÍA ARTESANAL," Ph.D. dissertation, Instituto Tecnológico y de Estudios Superiores de Occidente, San Pedro Tlaquepaque, Jalisco, 2023.

[18] T. González Sánchez, "Diseño de un proceso automatizado de fermentación de tepache," Ph.D. dissertation, Tecnológico Nacional de México, Cuernavaca, Morelos, México, 2024.

[19] D. A. Burdge and I. G. L. Libourel, "Open Source Software to Control Bioflo Bioreactors," *PLoS ONE*, vol. 9, no. 3, p. e92108, Mar. 2014.

[20] A. O. Moreno, "Análisis de la innovación de proceso y producto de un sistema de control y monitoreo de la fermentación alcohólica en vino." *Centro de Enseñanza Técnica y Superior, CETYS Universidad*, 2020.

[21] B. Q. K. Denisse and G. S. Emilio, "Sistema telemático para el monitoreo y control de huertos urbanos basado en una red inalámbrica de sensores," *UPIITA-IPN*, 2024.

[22] A. Diaz Avila and R. Aguilar Ordaz, "Prototipo de sistema inálambrico de monitoreo y advertencia para visibilizar fugas de mónoxido de carbono en un entorno limitado mediante técnicas de realidad aumentada," Ph.D. dissertation, UPIITA-IPN, Ciudad de México, 2021.

[23] J. C. Rodríguez-Figueroa and J. Antonio, "AVANCES EN EL ESTUDIO DE LA BIOACTIVIDAD MULTIFUNCIONAL DEL KÉFIR."

[24] E. R. Farnworth, Ed., *Handbook of Fermented Functional Foods*, ser. Functional Foods and Nutraceuticals Series. Boca Raton, Fla.: CRC Press, 2003.

[25] G. I. Puerta Quintero, *FUNDAMENTOS DEL PROCESO DE FERMENTACIÓN EN EL BENEFICIO DEL CAFÉ*. Federación Nacional de Cfeteros de Colombia, 2010.

[26] B. O. BARRÓN GARCÍA, "ELABORACIÓN DE KÉFIR," Ph.D. dissertation, UPIBI-IPN, 2006.

[27] E. M. de Electricidad:Mecatrónica, "Sensores y actuadores."

[28] F. D. Gros and J. S. Casals, "Introducción a los sensores."

[29] D. S. AG & Co. KGaA., "Introducción a los Sistemas de Detección de Gases," 2009.

[30] "Procedimientos de trabajo recomendados para sensores de pH," 2015.

[31] O. Muñoz Ñungo, J. A. Munoz, and H. M. Hernández Sarabia, "Dynamic analysis and comparison of control techniques in the process of obtaining bioethanol," *Inge CuC*, vol. 18, no. 2, Sep. 2022.

[32] C. G. Morcillo, "Lógica Difusa Una introducción práctica," *Técnicas de Softcomputing*.

[33] M. Santos González, *Diseño de redes telemáticas*. Madrid: RA-MA Editorial, 2014.

[34] J. C. P. Viñé, "Introducción a la Telemática y a las Redes de Datos," 2000.

[35] D. Filmus, M. R. Almandoz, J. M. Kirschenbaum, and M. Estévez, "Microprocesadores y microcontroladores:Ficha 1 y 2," *Ministro de Educación, Ciencia y Tecnología*, 2004.

[36] R. Campa García and J. Pinedo Valdez, "HERRAMIENTAS PARA EL DESARROLLO DE BACKEND," *Instituto Tecnologico superior Zacatecas Sur*, 2024.

[37] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database system concepts*, 5th ed. Boston: McGraw-Hill Higher Education, 2006.

[38] P. Araneda, *Base de Datos El camino de los datos a la información*. Opendataclinica, 2022.

[39] S. Casteleyn, F. Daniel, P. Dolog, and M. Matera, *Engineering Web Applications*. Springer-Verlag, 2009.

[40] "¿Qué es un servidor?" https://www.ionos.mx/digitalguide/servidores/know-how/que-es-un-servidor-un-concepto-dos-definiciones/, Sep. 2025.

[41] H. I. S. , "Electrodo de pH con cuerpo de PVDF para productos lácteos y alimentos semisólidos," https://hannainst.com.mx/electrodo-de-ph-foodcare-con-cuerpo-de-pvdf-para-l-fc200b.

[42] M. Electronics Kft., "Catalogo General Instrumentos para análisis de agua," 2019.

[43] "ENV-40-PH," https://www.digikey.com.mx/en/products/detail/atlas-scientific/ENV-40-PH/16003032.

[44] "ENV-30-PH," https://www.digikey.com.mx/es/products/detail/atlas-scientific/ENV-30-PH/16003028.

[45] "PH-4502C Sensor de PH Liquido con electrodo E201C-BNC."

[46] "Sensor de temperatura DS18B20," https://www.taloselectronics.com/products/sensor-de-temperatura-ds18b20-a-prueba-de-agua.

[47] "E52-CA6DN 4M," https://www.digikey.com.mx/es/products/detail/omron-automation-and-safety/E52-CA6DN-4M/3947502.

[48] "Módulo sensor de calidad del aire MQ135," https://www.taloselectronics.com/products/modulo-sensor-de-calidad-del-aire-mq135.

[49] "MH-Z1911A NDIR CO2 Module," https://www.winsen-sensor.com/product/mh-z1911a.html?campaignid=11440257107&adgroupid=146930169300&feeditemid=&targetid=kwd-310105068160&device=c&creative=679918219655&keyword=mh %20z19&gad_source=1&gad_campaignid=11440257107&gclid=CjwK 3GBhAYEiwAjh9fUG7zjC2oOMbNlfavNd7swI_3Pwg8evlJUCiSxwnV-EiRiq_IuhWlLRoC3tgQAvD_BwE.

[50] "SCD30 Sensirion | Mouser," https://www.mouser.mx/ProductDetail/Sensirion/SCD30?qs=rrS6PyfT74fdywu4FxpYjQ%3D%3D.

[51] "MODULO MG811 SENSOR DE CO2 DIOXIDO DE CARBONO," https://ssdielect.com/gas/2462-md-mg81.html.

[52] "Db2 for Linux, UNIX and Windows," https://www.ibm.com/docs/en/db2/11.5.x?topic=concepts-json-documents, Jan. 2025.

[53] "csv — CSV File Reading and Writing," https://docs.python.org/3/library/csv.html.

[54] "Lenguaje de marcado extensible (XML) 1.0 (quinta edición)," https://www.w3.org/TR/xml/.

[55] J. C. Caldera Palma and W. E. Suzazo Sequeira, "Módulo II: Redes de datos." *Universidad Nacional de Ingeniería-Managua, Nicaragua.*

[56] "¿Cuáles son las distintas normas WLAN 802.11?" https://www.ionos.mx/digitalguide/servidores/know-how/ieee-80211/, Aug. 2023.

[57] "Bluetooth Technology Overview," https://www.bluetooth.com/learn-about-bluetooth/tech-overview/, Oct. 2025.

[58] Z. Allian, Inc., "ZigBee Specification," 2015.

[59] admin-z-wavealliance, "Z-Wave Long Range vs Z-Wave," Aug. 2022.

[60] "What are LoRa and LoRaWAN?" https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/.

[61] "MQTT Specification," https://mqtt.org/mqtt-specification/.

[62] "HTTP: Hypertext Transfer Protocol | MDN," https://developer.mozilla.org/en-US/docs/Web/HTTP, Jul. 2025.

[63] "The WebSocket API (WebSockets) - Web APIs | MDN," https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API, Sep. 2025.

[64] "Seeed Studio XIAO ESP32-C6," https://www.seeedstudio.com/Seeed-Studio-XIAO-ESP32C6-p-5884.html.

[65] E. Embajadores, "Espressif ESP32-DevKitC-32E - Módulo de desarrollo basado en ESP32-E SopC - ESP32-S3-BOX-3B," https://www.electronicaembajadores.com/es/Productos/Detalle/LCWFNM22/modulos-electronicos/esp32-adafruit-feather-sparkfun-thing-lora/espressif-esp32-devkitc-32e-modulo-de-desarrollo-basado-en-esp32-e-sopc-esp32-s3-box-3b/.

[66] B. K, "ESP32 C3 Super Mini," https://www-sudo-is.translate.goog/docs/esphome/boards/esp32c3supermini/?_x_tr_sl=en&_x_tr_tl=

[67] "ESP32-C6-DEVKITC-1-N8," https://www.digikey.com.mx/es/products/detail/espressif-systems/ESP32-C6-DEVKITC-1-N8/17728861.

[68] "Tarjeta de desarrollo Wi-Fi Bluetooth ESP32-C2," https://www.didacticaselectronicas.com/shop/esp32-c2-tarjeta-de-desarrollo-wi-fi-bluetooth-esp32-c2-22437.

[69] "Pico-series Microcontrollers - Raspberry Pi Documentation," https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html.

[70] "Baterí 802040 650mah 2.405wh," https://spanish.alibaba.com/product-detail/High-Quality-3-7v-802040-650mah-1601536243515.html.

[71] "Batería Li-Pol 600mAh, 3.7V, 602248 | AMPUL," https://ampul.eu/es/baterias/5742-bateria-li-pol-600mah-37v-602248.

[72] "Batería Li-Pol 600mAh, 3.7V, 303450 | AMPUL.eu," https://ampul.eu/es/baterias/4522-bateria-li-pol-600mah-37v-303450.

[73] R. Marcelo Villalobos, *Fundamentos de Programación C#.* Empresa Editora Macro EIRL, 2008.

[74] H. M. Deitel and P. J. Deitel, *Cómo Programar En C#*, segunda ed. Pearson Education, 2007.

[75] R. González Duque, *Python PARA TODOS.* Creative Commons.

[76] A. B. Downey, J. Elkner, and C. Meyers, *How to think like a computer scientist: learning with Python*, 1st ed. Wellesley, Mass: Green Tea Press, 2002.

[77] E. Paradis, *R para Principiantes.* Institut des Sciences de l' ´Evolution Universit Montpellier II, 2003.

[78] A. Oppel and R. Sheldon, *Fundamentos de SQL*, 3rd ed. México D.F: McGraw Hill, 2011.

[79] R. H. Gómez and A. M. I. Maqueda, "BASES DE DATOS NOSQL: ARQUITECTURA Y EJEMPLOS DE APLICACIÓN," *2014*.

[80] "Document Database - NoSQL," https://www.mongodb.com/es/resources/basics/databases/document-databases.

[81] "Introducción | Apache Cassandra Documentación," https://cassandra.apache.org/.

[82] "Redis data types," https://redis.io/docs/latest/develop/data-types/.

[83] "InfluxDB 3 Documentación básica," https://docs.influxdata.com/influxdb3/core/.

[84] "Amazon DynamoDB," https://aws.amazon.com/es/dynamodb/.

[85] "Tipos de Datos de Mysql," https://desarrolloweb.com/articulos/1054.php, 2003.

[86] "Quick Start – React," https://react.dev/learn.

[87] "Angular - Introducción a los documentos angulares," https://v17.angular.io/docs.

[88] "Vue.js," https://vuejs.org/.

[89] "Getting Started | Recharts," https://recharts.github.io/en-US/storybook.

[90] "React-chartjs-2," https://react-chartjs-2.js.org/.

[91] "Nivo_guide," https://nivo.rocks/about/.

[92] "React Native | Victory," https://commerce.nearform.com/open-source/victory/.

[93] "Django at a glance | Django documentation," https://docs.djangoproject.com/en/5.2/intro/overview/.

[94] "Bienvenido a Flask — Frasco Documentación (3.1.x)," https://flask.palletsprojects.com/en/stable/.

[95] "FastAPI," https://fastapi.tiangolo.com/.

[96] "Administración de redes de área local virtuales - Administración de Oracle Solaris: Interfaces y virtualización de redes," https://docs.oracle.com/cd/E26921_01/html/E25833/fpjve.html.

[97] *Cloud Computing: Una Guía de Aproximación Para El Empresario*. INCIBE, 2017.

[98] "Mosquitto.h," https://mosquitto.org/api/files/mosquitto-h.html.

[99] "Documentación EMQX," https://docs.emqx.com/en/.

[100] "Welcome | VerneMQ," https://docs.vernemq.com, May 2025.

[101] "Plataforma HiveMQ :: Documentación HiveMQ," https://docs.hivemq.com/hivemq/latest/user-guide/index.html.

[102] "NanoMQ Docs," https://nanomq.io.

[103] "Protección de la puerta de enlace de IoT y el dispositivo de puerta de enlace de IoT - AWS IoT Core - AWS," https://aws.amazon.com/es/iot-core/.

[104] ALEXANDER. SAMUEL GUAYAS ESPIN, "ANÁLISIS COMPARATIVO DE LAS PLATAFORMAS AMAZON CLOUD, GOOGLE CLOUD, AZURE CLOUD," Ph.D. dissertation, UNIVERSIDAD TÉCNICA DE BABAHOYO FACULTAD DE ADMINISTRACIÓN, FINANZAS E INFORMÁTICA, BABAHOYO- LOS RIOS-ECUADOR, 2023.